

Communication-Aware Model Distributed Inference via Latent Representation Compression

Peyman Gholami[§], Theodoros-Thirimachos Davarakis[†], Teng Li[§], Miquel Sirera Perelló[‡], Salil Reddy[‡], Ayberk Yarkin Yıldız[†], Anish Arora[‡], Atilla Eryilmaz[‡], Stratis Ioannidis[†], Chengzhang Li[‡], Hulya Seferoglu[§], Ness Shroff^{‡*}

ABSTRACT

We study optimization of distributed model inference over resource-constrained edge resources. We propose a framework that optimizes the trade-off between model accuracy and communication costs by controlling latent representation compression to meet strict Quality of Service (QoS) throughput targets. For settings with known channel state information (CSI), we derive a closed-form optimal solution for single tasks and reduce the multi-task problem to a convex optimization program characterized by a per-link water-filling strategy. We extend these to handle unpredictable environments via a stochastic dual descent algorithm that relies only on causal channel estimates. We provide Lyapunov-based proofs demonstrating that our approach strictly satisfies long-term delay constraints while achieving a bounded optimality gap. Our results offer a robust, scalable blueprint for maximizing the performance of pipelined AI tasks in dynamic, resource-constrained distributed systems. We verify the effectiveness of our proposed framework through simulations and experiments with real edge devices.

1 INTRODUCTION

Deploying machine learning (ML) algorithms over edge devices critical for applications with stringent latency requirements. It is also appealing when availability, privacy, as well as rising cloud computing costs matter [15]. This has motivated studies on model compression [10, 24, 30, 35], enabling model execution on memory, computation, or power constrained edge devices, and model distributed inference [21, 22, 26], which accelerates edge inference through parallelism. Nevertheless, successfully harnessing computational resources at the edge gives rise to significant challenges, precisely because model distributed inference entails additional communication overheads. Several recent works have proposed compressing neural network activations transmitted between devices [19, 20] precisely for to reduce such communication overheads. This, in turn comes at the cost of reduced inference accuracy.

To characterize this tradeoff, we study a setting in which the layers of multiple, diverse machine learning inference tasks are partitioned and subsequently distributed across a network of resource-constrained edge devices. Each task processes streams of inputs generated at the edge: these could be, e.g., data collected from IoT sensors, cameras, LiDAR scenes, text inputs to an LMM, etc. To speed-up inference, nodes parallelize execution through *pipelining*: a device that finishes processing a collection of layers can pass activations computed to a new device, and begin processing a new input. To meet throughput requirements in light of dynamic channel conditions, these communications can be *compressed*, at the cost of accuracy degradation. Our contributions are as follows:

- We formalize the tradeoff between machine learning task accuracy vs. throughput under pipeline parallelism over an arbitrary edge network topology. To the best of our knowledge, we are the first to introduce and study this problem.
- We first assume that channel-state information (CSI) is known, considering both single-task and multi-task settings. We show that the corresponding optimization problem is tractable when the accuracy objective is a monotone function of compression rates in the single task setting, and monotone and concave in the multi-task setting.
- Next, we study settings in which CSI is stochastic and a-priori unknown. Under the same assumptions of monotonicity and concavity, we show that a stochastic dual descent algorithm can asymptotically recover the optimal compression ratios when having access to noisy, near-unbiased estimates of the CSI in the single-task setting.
- For the multi-task setting with similar CSI estimates, the primal-step is non-convex. We show that a block-coordinate descent solver can be used as a tractable algorithm, and prove that, if it comes with additive approximation guarantees, the latter also translate to convergence guarantees for the end-to-end stochastic dual descent algorithm.
- We extensively evaluate our proposed CSI-aware and CSI-oblivious algorithms for a broad array of vision and language tasks, under top- k [1, 28], quantization [24, 30], and LLM-int8 [6] compression schemes. Our methods outperform competitors by maximizing accuracy while meeting throughput constraints.
- We deploy and evaluate our algorithms over three diverse testbeds, namely (a) a (highly-constrained) Raspberry PI testbed, (b) a Jetson Orin Nano testbed, and (c) PRESCIENT, a GPU dense network research testbed, demonstrating the applicability of our compression algorithms for scaling real-time inference deployments.

The remainder of this paper is structured as follows. We review related work in Section 2. In Section 3, we provide the system model, CSI-aware design, estimated CSI design, and accuracy-compression function & gradient estimation. Section 4 provides extensive evaluations of our proposed algorithms. Finally, we conclude in Section 5.

2 RELATED WORK

Model Distributed Inference. The deployment of large-scale neural networks on resource-constrained edge devices has catalyzed the development of distributed inference architectures [3, 14, 22]. This distributed paradigm mitigates the strict memory constraints of individual edge devices while simultaneously providing substantial speedups through pipeline parallelism. This architecture

^{*}§University of Illinois Chicago, [†]Northeastern University, [‡]The Ohio State University

has proven highly scalable, recently enabling the distributed inference of Large Language Models (LLMs) over consumer-grade internet connections [2, 26]. As a result, current literature is heavily focused on efficiently mapping and distributing computation workloads across available edge nodes [21, 23, 33]. However, despite successfully utilizing parallelism to reduce computation time, these frameworks generally assume a strict single-task environment. More critically, they rely on transmitting massive, uncompressed intermediate activations between model partitions. In edge environments characterized by dynamic or degraded channel capacities, communicating these high-dimensional tensors becomes a critical bottleneck, severely bounding the end-to-end throughput.

Communication-Efficient Inference and Activation Compression. To accelerate execution on resource-constrained edge devices, early efforts heavily focused on static model compression. Techniques such as weight pruning, knowledge distillation, and integer quantization [10, 13] effectively reduce the memory footprint and computational requirements of neural networks prior to deployment [35]. Extending this paradigm to distributed settings, recent work has explored communication-aware pruning [16] to explicitly co-optimize model sparsity and the resulting data transmission overhead. However, while these static methods are highly effective, they do not inherently resolve the dynamic network bottlenecks that arise when models are partitioned across multiple nodes. We explore an orthogonal dimension that can be used in addition to static methods such as, e.g., model pruning. We reduce the communication burden during model-distributed inference, by focusing on lightweight compression methods that operate directly on the intermediate outputs (activations) exchanged between workers at a partitioning point between layers (see Fig. 1).

Activation Compression. Several intermediate activation compression strategies have been proposed for both training and inference. Specifically, *Top-k sparsification* [1, 28, 29] reduces payload sizes by transmitting only the most significant activation values while masking the rest to zero, though this often requires transmitting additional spatial index metadata. Alternatively, *quantization* [4, 24, 30] mitigates communication overhead by mapping high-precision floating-point activations to lower bit-width representations. This is typically achieved through fixed-point quantization or dynamically adjusted mixed-precision [34]. Though the majority of these approaches accelerate training, recent literature has demonstrated the efficacy of targeted data compression schemes designed specifically to facilitate efficient and timely communication for network-edge classification tasks [19, 20]. We balance communication reduction against the accuracy of the downstream inference task. This can be achieved by introducing a dynamically adjustable latent compression factor that continuously scales the intermediate payload size in response to fluctuating, real-time channel capacities, ensuring that strict QoS throughput constraints are satisfied without unnecessary degradation of inference accuracy.

3 METHODOLOGY

3.1 System Model

We consider a model distributed inference system designed to process multiple inference tasks on the edge, as illustrated in Fig. 1.

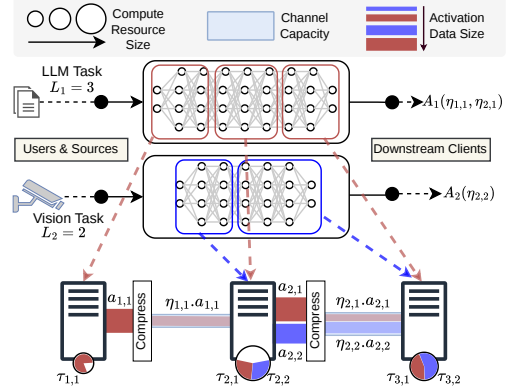


Figure 1: Distributed inference system processing two concurrent tasks (LLM, partitioned into $L_1 = 3$ stages; Vision, partitioned into $L_2 = 2$ stages). Computation cycles ($\tau_{i,k}$) are assigned to the computational nodes. Intermediate activations ($a_{i,k}$) are compressed by a factor $\eta_{i,k}$ prior to transmission, yielding effective payloads $\eta_{i,k} a_{i,k}$ and determining the final end-to-end inference accuracy A_k .

Several tasks are distributed through pipelining and executed continuously across interconnected computational nodes arranged in a general directed graph topology. Channel capacity between nodes is dynamic, and to meet throughput constraints, communication between consecutive nodes is compressed, leading to an accuracy-throughput tradeoff. We seek compression policies that maximize aggregate task utility across under throughput constraints.

Computational Nodes & Inference Tasks. We model the underlying network topology as a directed graph $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of computational nodes and $\mathcal{E}$ is the set of directed communication links. Time is slotted, and each edge $e \in \mathcal{E}$ has a time-varying *channel capacity* $c_e(t) \in \mathbb{R}_+$ at time slot $t \in \mathbb{N}$. Assume a set of K distinct *inference tasks*. The set of tasks available at time slot t is $\mathcal{K}(t) \subseteq [K] \triangleq \{1, \dots, K\}$. For every task $k \in \mathcal{K}(t)$, we assume a continuous stream of inference input samples (e.g., camera frames, LiDAR scenes, sensor measurements, etc.) to be processed for the duration of time slot t . To maximize hardware utilization, the system employs pipeline parallelism, overlapping computation and communication phases across all L_k stages.

Formally, each task $k \in \mathcal{K}(t)$ corresponds to a neural network whose layers are partitioned into L_k sequential *stages*. These stages are assigned to a processing path, given by the sequence $P_k = (v_{1,k}, \dots, v_{L_k,k})$: physical node $v_{i,k} \in \mathcal{V}$ processes the i -th pipeline stage of the task ($i \in [L_k]$). Let $V_k \subseteq \mathcal{V}$ and $E_k \subseteq \mathcal{E}$ denote the sets of vertices and edges traversed by P_k , respectively, where the i -th transmission hop occurs over edge $e_{i,k} = (v_{i,k}, v_{i+1,k})$ for $i \in [L_k - 1]$. For notational convenience, we denote the capacity of the physical link traversed by this i -th hop as $c_{i,k}(t) \triangleq c_{e_{i,k}}(t)$.

We denote by $\tau_{i,k} \in \mathbb{R}_+$ the compute cycle time required to process an input sample through the i -th stage of task k at node $v_{i,k}$. The output of the i -th stage (i.e., the activations of its last layer) has an uncompressed size $a_{i,k} \in \mathbb{R}_+$ (e.g., in Mbytes). To ensure Quality of Service (QoS), the system must satisfy a throughput constraint: sample inferences per second during time slot t meet or exceed a target rate $R_k(t) \in \mathbb{R}_+$ for task k . Finally, each task is described by an *accuracy-compression* function, which we describe next.

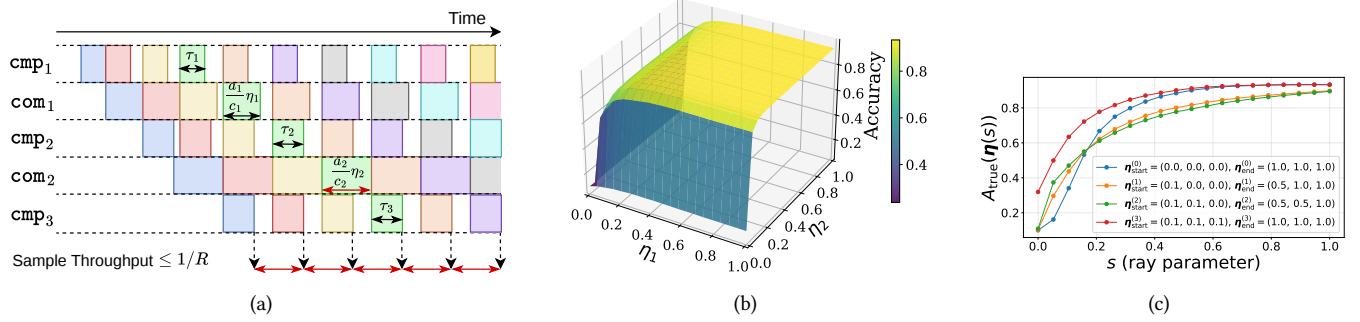


Figure 2: (a) Timing diagram of pipeline parallelism for distributed inference of a single task. Consider a task partitioned over 3 stages, distributed across 3 nodes. Colors indicate different samples. Computation (cmp_i) and communication (com_i) phases at each node i overlap. This leads to an end-to-end throughput bottlenecked by the maximum of stage processing times (τ_i) and compressed transmission delays ($\frac{a_i}{c_i(t)}\eta_i(t)$). If compression is sufficiently high such that computation dominates communication, and the computation is evenly partitioned across nodes (i.e., each τ_i equals $1/3$ of the serial execution time), the resulting parallel speedup is $3\times$. (b) The accuracy function $A(\eta)$ for ResNet-56 [11] on CIFAR10 with $L=3$ stages of top- k sparsification [1, 28] (two cutpoints), smoothened via KNN averaging. The function is monotone & concave above $[\eta_1, \eta_2] = [0.1, 0.1]$, and collapses below this threshold. (c) To verify that the observed behavior in (b) is present in higher dimensional η -spaces, we plot the directional profiles of $A(\eta)$ along random rays for the same ResNet-56 as in (b), with $L=4$ stages (three cutpoints). Pictured is $A(\eta(s))$ evaluated along 4 linear segments in three dimensional η -space, parameterised as $\eta(s) = \eta_{start} + s(\eta_{end} - \eta_{start})$, for $s \in [0, 1]$. Each curve traces $A(\eta(s))$ on a segment, as s increases from 0 to 1. We observe that (i) the accuracy collapses below a threshold (for certain segments), (ii) the concavity above the threshold and (iii) the overall monotonicity of A .

Accuracy-Compression Function. Since edge capacities fluctuate, to reduce communication overhead, a node may compress a stage's output (i.e., the activations of the last layer) prior to transmitting it to the next node. Several compression schemes are possible, e.g., top- k sparsification [1, 28], quantization [24, 30], etc. (see also Sec. 4.1). Given a compression scheme, we denote by $\eta_{i,k}(t) \in [\eta_{i,k}^{\min}, 1]$ the *compression rate* applied to the activations of the i -th stage of task k before transmission over edge $e_{k,i}$, resulting in an effective data size of $\eta_{i,k}(t)a_{i,k}$. Here, $\eta_{i,k}^{\min}$ denotes the minimum allowable compression rate for the i -th hop; compressing below this threshold causes the intermediate activations to collapse into noise, thereby degrading the inference accuracy to its absolute minimum (see also Fig. 2a). Let $\eta_k(t) = (\eta_{i,k}(t))_{1 \leq i < L_k-1}$ denote the vector of compression rates applied across the transmission hops of task k . We denote by $A_k(\eta_k(t)) \in \mathbb{R}_+$ the resulting *inference accuracy*. We assume that $A_k(\cdot)$ is given for each task k ; we describe how to estimate it from held-out data in Sec. 3.4.

Overall Objective. Given a mapping of partitioned tasks over an edge topology and a compression scheme (expressed via an accuracy-compression function per task), we wish to identify optimal compression factors that maximize aggregate accuracy, while meeting throughput constraints. In the remainder of this section, we formulate and propose solutions for this optimization problem under two distinct assumptions regarding channel state information (CSI), namely, a CSI-Aware and an Estimated-CSI setting.

3.2 CSI-Aware Design

In our channel state information (CSI) aware setting, we assume that channel capacities $c_e(t) \in \mathbb{R}_+$, $e \in \mathcal{E}$, are provided at the beginning time slot t . As a warmup, we first describe the optimization problem and the solution that arises when there is only a single task.

3.2.1 CSI-Aware Case – Single Task. When only a single task is present in the system, the optimization reduces to identifying optimal compression rates that meet throughput constraints. For convenience, we drop the task index k from our notation, and assume w.l.o.g. that the nodes i in the path P the task is assigned to are indexed as $1, 2, \dots, L$. Recall that a stream of inference requests occur at each timeslot using pipeline parallelism, that overlaps computation and communication phases across different samples. Under this model, the average throughput is constrained by its slowest component, either computation or communication (see also Fig. 2a). This leads to the following optimization problem:

Problem 1 (CSI-Aware Single-Task).

$$\text{Maximize: } A(\eta(t)) \quad (1a)$$

$$\text{subj. to: } \max_{i \in [L]} \tau_i, \max_{i \in [L-1]} \frac{a_i}{c_i(t)} \eta_i(t) \leq \frac{1}{R(t)}, \quad (1b)$$

$$\eta_i(t) \in \prod_{i=1}^{L-1} [\eta_i^{\min}, 1]. \quad (1c)$$

In short, (1b) states that *all* computation times $\{\tau_i\}_{i=1}^L$ and *all* transmission times $\{\frac{a_i}{c_i(t)}\eta_i(t)\}_{i=1}^{L-1}$ per stage must be bounded by the inverse of the target throughput $R(t)$. When the neural network is equipartitioned, reducing communication sufficiently makes execution computation-bound; in turn, this leads to a speedup by a factor L compared to serial execution (see Fig. 2a). More generally, varying compression rates η establishes a trade-off between maximum throughput $R(t)$ and accuracy $A(\eta(t))$.

It is easy to see that the optimal solution per stage is to exhaust available capacity, binding the target throughput rate at all stages:

Theorem 3.1 (Closed-Form Optimal Solution). *If Problem 1 is feasible, and the accuracy function $A(\eta(t))$ is non-decreasing with respect to each $\eta_i(t)$, the problem has the unique optimal solution: $\eta_i^*(t) = \min\left\{1, \frac{c_i(t)}{R(t)a_i}\right\}$, for all $i \in [L-1]$.*

The proof is in App. A. It is worth noting that an optimal solution can be computed so long as the A is monotone in $\eta(t)$, which is both natural and also observed experimentally (see Figs. 2b and 2c).

3.2.2 CSI-Aware – Multi-Task. In a multi-task setting, different tasks may exhibit varying sensitivities to activation compression and heterogeneous computational demands. For instance, a task that is highly sensitive to compression at stage i but robust at stage j can receive a larger share of its i -th transmission hop's bandwidth and a correspondingly smaller share for its j -th hop. Similarly, adaptive allocation of computational resources is essential for optimizing pipeline parallelism. Because the end-to-end throughput of a pipelined task is dictated by its slowest stage, optimizing the compute share at each node prevents stragglers and balances the processing delays across the entire pipeline.

Let $s_{i,k}^{\text{cmp}}(t) \in (0, 1]$ denote the fraction of computation capacity of node $v_{i,k}$ allocated to process the i -th stage of task k during time slot t . Similarly, let $s_{i,k}^{\text{com}}(t) \in (0, 1]$ denote the fraction of communication bandwidth of edge $e_{i,k}$ allocated to task k for its i -th transmission hop during time slot t . To ensure that physical resource limits are not exceeded, the total fraction of resources allocated to all active pipeline stages on any given node $v \in \mathcal{V}$, or transmissions across any given edge $e \in \mathcal{E}$, must be bounded at each time slot t . We express these capacity constraints by summing the assigned allocations directly over the applicable tasks and their respective stages for each physical resource:

$$\sum_{k \in \mathcal{K}(t)} \sum_{i: v_{i,k}=v} s_{i,k}^{\text{cmp}}(t) \leq 1, \quad \forall v \in \mathcal{V}, \quad (2)$$

$$\sum_{k \in \mathcal{K}(t)} \sum_{i: e_{i,k}=e} s_{i,k}^{\text{com}}(t) \leq 1, \quad \forall e \in \mathcal{E}. \quad (3)$$

We are interested in maximizing the aggregate accuracy across tasks k , weighted by some $w_k > 0$ (e.g., $1/K$). We thus arrive at the following optimization problem:

Problem 2 (CSI-Aware Multi-Task).

$$\text{Maximize: } \sum_{k=1}^K w_k A_k(\eta_k(t)) \quad (4a)$$

$$\text{subj. to: } \frac{\tau_{i,k}}{s_{i,k}^{\text{cmp}}(t)} \leq \frac{1}{R_k(t)}, \quad \forall k \in \mathcal{K}(t), i \in [L_k], \quad (4b)$$

$$\frac{a_{i,k} \eta_{i,k}(t)}{s_{i,k}^{\text{com}}(t) c_{i,k}(t)} \leq \frac{1}{R_k(t)}, \quad \forall k \in \mathcal{K}(t), i \in [L_k - 1], \quad (4c)$$

Constraints (2) and (3),

$$\eta_k(t) \in \prod_{i=1}^{L_k-1} [\eta_{i,k}^{\min}, 1], \quad \forall k \in \mathcal{K}(t), \quad (4d)$$

$$s_{i,k}^{\text{cmp}}(t), s_{i,k}^{\text{com}}(t) \in (0, 1]. \quad (4e)$$

Akin to (1b), constraints (4b) and (4c) enforce the fundamental bottleneck principle of pipeline parallelism by bounding the effective execution delays. The term $\tau_{i,k}/s_{i,k}^{\text{cmp}}(t)$ in (4b) represents the *dilated computation time*; allocating only a fraction $s_{i,k}^{\text{cmp}}(t)$ of the assigned node's computational capacity proportionally increases the time required to process that stage. Similarly, in (4c), the term $s_{i,k}^{\text{com}}(t) c_{i,k}(t)$ represents the effective, dedicated communication bandwidth allocated specifically to task k on its i -th transmission hop. Dividing the compressed payload $a_{i,k} \eta_{i,k}(t)$ by this bandwidth slice yields the actual transmission delay. To sustain a target throughput of $R_k(t)$, both this effective computation time and the

actual transmission delay must be bounded by $1/R_k(t)$, the maximum allowable pipeline cycle time.

To solve Problem 2, we first establish the conditions under which a feasible resource allocation exists.

Lemma 3.2 (Multi-Task Feasibility Condition). *Problem 2 is feasible if and only if $\sum_{k \in \mathcal{K}(t)} \sum_{i: v_{i,k}=v} \tau_{i,k} R_k(t) \leq 1$, for all $v \in \mathcal{V}$, and $\sum_{k \in \mathcal{K}(t)} \sum_{i: e_{i,k}=e} \frac{a_{i,k} \eta_{i,k}^{\min} R_k(t)}{c_{i,k}(t)} \leq 1$, for all $e \in \mathcal{E}$.*

The proof is in App. B. Next, we show Problem 2 can be reduced to a convex optimization program when $A_k(\cdot)$ are concave.

Theorem 3.3 (Reduction to Convex Optimization). *Assume Problem 2 is feasible. If the accuracy functions $A_k(\cdot)$ are concave and non-decreasing, Problem 2 is equivalent to the following convex optimization problem over the communication fraction variables:*

$$\text{Maximize: } \sum_{k=1}^K w_k A_k(\eta_k^*(s^{\text{com}}(t))) \quad (5a)$$

$$\text{subj. to: } \sum_{k \in \mathcal{K}(t)} \sum_{i: e_{i,k}=e} s_{i,k}^{\text{com}}(t) \leq 1, \quad \forall e \in \mathcal{E}, \quad (5b)$$

$$s_{i,k}^{\text{com}}(t) \geq \frac{a_{i,k} \eta_{i,k}^{\min} R_k(t)}{c_{i,k}(t)}, \quad \forall k \in \mathcal{K}(t), i \in [L_k - 1], \quad (5c)$$

where, for all $k \in \mathcal{K}(t)$, the compression rates $\eta_k(t)$ are given by:

$$\eta_{i,k}^*(s^{\text{com}}(t)) \triangleq \min \left(1, \frac{c_{i,k}(t)}{R_k(t) a_{i,k}} s_{i,k}^{\text{com}}(t) \right), \quad i \in [L_k], \quad (6)$$

and the computation fractions are $s_{i,k}^{\text{cmp}}(t) = \tau_{i,k} R_k(t)$, for $i \in [L_k]$.

The proof is in App. C. In summary, Thm. 3.3 implies that, if the accuracy-compression functions are concave and non-decreasing, we can compute the optimal solution to Problem 2 using standard convex optimization techniques. In fact, we show in App. D that Prob. (5) can be solved in polynomial time via a per-link water-filling with a QoS floor strategy [7].

3.3 Estimated-CSI Design

Next, we consider an online setting where the exact channel capacities $c(t)$ are (a) drawn from a stationary random process (e.g., i.i.d.) and (b) not known at the time of decision-making. Instead, the controller has access to an estimate $\hat{c}(t)$ based, e.g., on the transmission history of previous slots. The system must make decisions based on these estimates, while the actual pipeline delay is governed by the true realization $c(t)$.

For each task $k \in \mathcal{K}(t)$, we define a delay function:

$$D_k(\eta_k(t), s_k(t), c(t)) \triangleq \max \left\{ \max_{i \in [L_k]} \frac{\tau_{i,k}}{s_{i,k}^{\text{cmp}}(t)}, \max_{i \in [L_k-1]} \frac{a_{i,k} \eta_{i,k}(t)}{c_{i,k}(t) s_{i,k}^{\text{com}}(t)} \right\}, \quad (7)$$

that measures the maximum delay across all pipeline computation and communication stages under the given parameters. We denote by $D_{\text{act},k}(t) \triangleq D_k(\eta_k(t), s_k(t), c(t))$ the delay under the actual bandwidth capacities at time t , and $\hat{D}_k(t) \triangleq D_k(\eta_k(t), s_k(t), \hat{c}(t))$ the delay estimate under capacity estimates $\hat{c}(t)$. We introduce the following additional assumptions on our problem setup:

Assumption 1 (Bounded and Concave Objective): The accuracy objective function $A_k(\eta_k)$ for any task k is concave w.r.t. η_k and bounded, i.e., that $A_k(\eta_k) \in [A_k^{\min}, A_k^{\max}]$ for all feasible η_k .

Assumption 2 (Bounded Second Moment of Constraint Violation): For any time slot t and task k , the expected squared constraint violation is bounded, i.e., there exists a ξ^2 s.t. for all $k \in [K]$ and $t \geq 1$ we have $\mathbb{E} \left[\left(D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right)^2 \right] \leq \xi^2$.

Algorithm 1 Single-Task Estimated Stochastic Dual Descent

-
- 1: **Input:** Tradeoff parameter $\mu > 0$, QoS rates $R(t)$, dual lower bound $\epsilon > 0$.
 - 2: **Initialization:** Set the scalar Lagrange multiplier $\lambda_1 = \epsilon$.
 - 3: **for** time slot $t = 1, 2, \dots$ **do**
 - 4: **Estimation:** Obtain the channel estimate vector $\hat{c}(t) = [\hat{c}_1(t), \dots, \hat{c}_{L-1}(t)]$.
 - 5: **Primal Step (Compression Optimization):** Select $\eta(t)$ to maximize the partial Lagrangian using the **estimated** delay cost:

$$\eta(t) = \arg \max_{\eta \in \prod_{i=1}^{L-1} [\eta_i^{\min}, 1]} (A(\eta) - \mu \cdot \lambda_t \cdot \hat{D}(\eta, 1, \hat{c}(t))).$$

Note: This is a convex optimization problem.
 - 6: **Execute:** Run the pipeline with configuration $\eta(t)$. The system experiences the **true** bottleneck delay $D_{\text{act}}(t) = D(\eta(t), c(t))$.
 - 7: **Dual Step (Multiplier Update):** Update the Lagrange multiplier based on the **actual** realized constraint violation:

$$\lambda_{t+1} = \max\{\epsilon, \lambda_t + D_{\text{act}}(t) - 1/R(t)\}.$$
 - 8: **end for**
-

Assumption 3 (Estimation Error Characteristics): For any time slot t and task $k \in [K]$, let $\Delta_k(t) = D_{\text{act},k}(t) - \hat{D}_k(t)$ denote the estimation error. Let $\mathcal{F}(t)$ denote the system history (filtration) up to the beginning of time slot t , encompassing all past channel realizations and algorithmic decisions. Conditioned on the filtration $\mathcal{F}(t)$, the expected estimation error is bounded; i.e., there exists a constant $\delta > 0$ s.t.: $\mathbb{E}[\Delta_k(t) | \mathcal{F}(t)] \leq \delta$, for all $k \in [K]$ and $t \geq 1$. Additionally, the expected positive part of the error is also bounded, i.e., there exists a constant $\delta^+ > 0$ s.t. $\mathbb{E}[\max\{0, \Delta_k(t)\} | \mathcal{F}(t)] \leq \delta^+$, for all $k \in [K]$ and $t \geq 1$.

Assumption 4 (Robust System Feasibility): There exists a valid resource allocation sequence $\{s_k(t)\}_{t \geq 1}$ such that the expected baseline bottleneck delay for any task k , evaluated at its maximum compression configuration $\eta_k^{\min}$, satisfies its throughput constraint: $\mathbb{E}[D_k(\eta_k^{\min}, s_k(t), c(t))] \leq \frac{1}{R_k(t)} - \gamma$, for all $k \in [K]$, $t \geq 1$, for some positive margin $\gamma > 0$. We assume this feasibility margin dominates the worst-case channel estimation error, i.e., $\gamma > \delta^+$.

3.3.1 CSI-Oblivious – Single-Task. Dropping the task index k for the single-task setting, let $\hat{c}(t)$ denote the estimated channel capacity available at the beginning of slot t ; the resource allocation vector can be trivially fixed to a vector of ones (i.e., $s(t) = 1$), as the task consumes all computational and bandwidth resources. We aim to maximize the time average of accuracy subject to the time average delay constraint being satisfied under the true channel realizations:

Problem 3 (No-CSI Single-Task).

$$\max_{\{\eta(t)\}_{t \geq 1}} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[A(\eta(t))] \quad (8a)$$

$$\text{subj. to: } \frac{1}{T} \sum_{t=1}^T \mathbb{E}\left[D_{\text{act}}(t) - \frac{1}{R(t)}\right] \leq 0, \quad (8b)$$

$$\eta(t) \in \prod_{i=1}^{L-1} [\eta_i^{\min}, 1], \quad \forall t \geq 1. \quad (8c)$$

We solve this problem using a primal-dual algorithm (Alg. 1). In short, compression rates can be set by optimizing a Lagrangian using the estimated delay; under Assumption 1, the corresponding problem is convex and thereby tractable. Moreover, under assumptions we have made on the quality of the channel estimator, we prove in App. E that the following theorem holds:

Theorem 3.4 (No-CSI Single Task Performance Bounds). *Under Assumptions 1–4, Alg. 1 yields the following performance guarantees: **Constraint Satisfaction:** The long-term average expected delay strictly satisfies the target threshold, i.e.:*

$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}\left[D_{\text{act}}(t) - \frac{1}{R(t)}\right] \leq 0$. **Dual Variable Stability Bound:** The long-term average expected dual variable is bounded by $\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[\lambda_t] \leq \frac{A_{\max} - A_{\min}}{\mu(\gamma - \delta^+)} + \frac{\xi^2 + \epsilon^2}{2(\gamma - \delta^+)}$. **Optimality Gap:** Let B_{dual} denote the above dual variable stability bound. The long-term average expected accuracy satisfies: $\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[A(\eta(t))] \geq A^* - \frac{\mu(\xi^2 + \epsilon^2)}{2} - \mu B_{\text{dual}}(\delta - \delta^*)$, where A^* is the optimal objective value achievable over all stationary algorithms that can satisfy the desired constraints in Problem 3, and δ^* is the expected estimation error under the optimal stationary policy.

The performance bounds established above reveal a fundamental $[O(1/\mu), O(\mu)]$ trade-off governed by the control parameter μ . This parameter serves as the primary mechanism for balancing application utility (inference accuracy) against dual variable stability (delay constraint satisfaction). As μ increases, the algorithm places a heavier penalizing weight on the delay constraint violations, ensuring stricter compliance; mathematically, the upper bound on the time-averaged dual variables—which track the cumulative delay violations—shrinks proportionally to $O(1/\mu)$. However, this focus on delay constraints comes at the expense of application performance: a larger μ widens the optimality gap proportionally to $O(\mu)$, leading to lower overall inference accuracy. Conversely, a smaller μ prioritizes maximizing the accuracy objective, but results in a looser upper bound on the accumulated delay violations. We discuss how to select μ from problem parameters in App. G.

3.3.2 Estimated-CSI – Multi-Task. Next, we extend the online setting to multiple concurrent tasks. Recall that, in this environment, the controller faces a coupled optimization challenge: it must jointly set compression rates while dynamically partitioning shared computational and communication resources, while relying entirely on the channel estimates $\hat{c}(t)$. Let $\mathcal{T}_k(T) \triangleq \{t \leq T : k \in \mathcal{K}(t)\}$ be the set of time slots up to horizon T during which task k is active. Our objective is again to maximize the weighted sum of time-averaged accuracies across all tasks, strictly subject to individual time-averaged delay constraints:

Problem 4 (No-CSI for Multiple Tasks).

$$\max_{\substack{\{\eta(t)\}_{t \geq 1}, \\ \{s^{\text{com}}(t), s^{\text{cmp}}(t)\}_{t \geq 1}}} \frac{1}{T} \sum_{t=1}^T \sum_{k \in \mathcal{K}(t)} w_k \mathbb{E}[A_k(\eta_k(t))] \quad (9)$$

$$\text{subj. to: } \frac{1}{|\mathcal{T}_k(T)|} \sum_{t \in \mathcal{T}_k(T)} \mathbb{E}\left[D_{\text{act},k}(t) - \frac{1}{R_k(t)}\right] \leq 0, \quad \forall k \in [K], \quad (10)$$

Constraints (2), (3), (4d) and (4e), $\forall t \geq 1$.

Our algorithm for this problem is described in Alg. 2. The joint optimization of resource allocation $(s^{\text{cmp}}, s^{\text{com}})$ and compression factors (η) is inherently non-convex due to the coupling in the delay term η/s^{com} . To maintain tractability, we employ a Block Coordinate Descent (BCD) strategy, which iteratively fixes one set of variables to solve resulting convex sub-problem. Notice that in the primal optimization step (line 5), the dual variables λ_k act as the weights for bottleneck delays. If an active task were to have $\lambda_k = 0$, the optimizer would allocate it close to zero resources, effectively starving it. To prevent this, we introduce $\epsilon > 0$ as a mandatory floor for the dual variables. To provide guarantees for this algorithm, we

Algorithm 2 Multi-Task Estimated Stochastic Dual Descent

- 1: **Input:** Tradeoff parameter $\mu > 0$, task weights w_k , QoS rates $R_k(t)$, max BCD iterations J , dual lower bound $\epsilon > 0$.
- 2: **Initialization:** Set the dual variables $\lambda_k(1) = \epsilon$ for all $k \in [K]$.
- 3: **for** time slot $t = 1, 2, \dots$ **do**
- 4: **Estimation:** Obtain the channel estimate vector $\hat{c}(t) = [\hat{c}_e(t)]_{e \in \mathcal{E}}$.
- 5: **Primal Step (Resource and Compression Optimization):** Select $\eta(t)$ and $s(t)$ for active tasks $\mathcal{K}(t)$ to maximize the partial Lagrangian using the **estimated** delay cost:

$$\begin{aligned} & \max \sum_{k \in \mathcal{K}(t)} [w_k A_k(\eta_k) - \mu \cdot \lambda_k(t) \cdot \hat{D}_k(\eta_k, s_k, \hat{c}(t))] \\ & \text{subj. to: } \sum_{k \in \mathcal{K}(t)} \sum_{i: e_{i,k}=e} s_{i,k}^{\text{com}} \leq 1, \quad \forall e \in \mathcal{E}, \\ & \quad \sum_{k \in \mathcal{K}(t)} \sum_{i: v_{i,k}=v} s_{i,k}^{\text{cmp}} \leq 1, \quad \forall v \in \mathcal{V}. \end{aligned}$$

Optimization Method (Block Coordinate Descent): For iteration $j = 1, \dots, J$:

Phase A (Resource Optimization): Fix $\eta = \eta^{(j-1)}$. Update $s^{(j)}$ by solving the resulting convex sub-problem.

Phase B (Compression Optimization): Fix $s = s^{(j)}$. Update $\eta^{(j)}$ by solving the resulting concave sub-problem.

Assign final operational decisions $\eta(t) = \eta^{(J)}$ and $s(t) = s^{(J)}$.
- 6: **Execute:** Run active tasks with assigned configurations. The system experiences the **true** bottleneck delays $D_{\text{act},k}(t) = D_k(\eta_k(t), s_k(t), c(t))$ for all $k \in \mathcal{K}(t)$.
- 7: **Dual Step (Multiplier Update):** Update the Lagrange multipliers based on the **actual** realized constraint violations, freezing inactive tasks:

$$\lambda_k(t+1) = \begin{cases} \max \left\{ \epsilon, \lambda_k(t) + D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right\}, & \text{if } k \in \mathcal{K}(t) \\ \lambda_k(t), & \text{otherwise} \end{cases}$$
- 8: **end for**

need the following additional assumption regarding the stationary point found by the BCD process:

Assumption 5 (Primal Optimization Gap): The expected additive gap between the stationary point solution found by the block coordinate descent and the true global maximum of the partial Lagrangian in Alg. 2 is bounded by C_{gap} .

Theorem 3.5 (No-CSI Multi Task Performance Bounds). *Under Assumptions 1–5, Alg. 2 yields the following performance guarantees: **Constraint Satisfaction:** The long-term average expected delay strictly satisfies the target threshold during the time slots when the task is active, for all tasks $k \in [K]$, $\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t \in \mathcal{T}_k(T)} \mathbb{E} \left[D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right] \leq 0$. **Dual Variable Stability Bound:** The sum of the average expected active dual variables across all tasks is bounded as: $\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{k \in \mathcal{K}(t)} \mathbb{E}[\lambda_k(t)] \leq \frac{\sum_{k=1}^K \rho_k w_k (A_{k,\text{max}} - A_{k,\text{min}}) + C_{\text{gap}}}{\mu(\gamma - \delta^*)} + \frac{\sum_{k=1}^K \rho_k (\xi^2 + \epsilon^2)}{2(\gamma - \delta^*)}$. **Optimality Gap:** Let B_{dual} denote the above dual variable stability bound. The long-term weighted average expected accuracy satisfies: $\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{k \in \mathcal{K}(t)} w_k \mathbb{E}[A_k(\eta_k(t))] \geq A^* - \frac{\mu}{2} \sum_{k=1}^K \rho_k (\xi^2 + \epsilon^2) - C_{\text{gap}} - \mu B_{\text{dual}}(\delta - \delta^*)$, where $\rho_k \triangleq \lim_{T \rightarrow \infty} \frac{|\mathcal{T}_k(T)|}{T}$. A^* is the optimal objective value achievable over all stationary algorithms that can satisfy the desired constraints in Problem 4, and δ^* is the expected estimation error when the stationary optimal policy is applied.*

The proof is in App. F. As in the single-task case (Thm. 3.4), we observe the fundamental $[O(1/\mu), O(\mu)]$ trade-off, where the parameter μ balances aggregate accuracy against delay compliance. However, Theorem 3.5 highlights the critical role of task-specific weights w_k and activity frequencies ρ_k in a shared-resource environment, normalizing the bounds by the duty cycle and priority of each task. The multi-task bounds explicitly capture the impact of the non-convex BCD algorithm; because the solver converges to a

Table 1: Inference tasks used in the evaluation. Each task is specified by a dataset-model pair and its utility metric is A_k .

Task ID	Dataset	Model	Metric (A_k)	$\eta_{\min}$
<i>Vision tasks</i>				
MLP-MN	MNIST [18]	MLP	Accuracy	0.125
RN56-CF10	CIFAR-10 [17]	ResNet-56 [11]	Accuracy	0.125
<i>Language tasks</i>				
G2-2B-SGPT	ShareGPT [25]	Gemma-2 2B [8]	Perplexity	0.25
G2-7B-SGPT		Gemma-2 7B [8]	Perplexity	0.25
LI3-8B-MMLU	MMLU [12]	Llama-3.1 8B [9]	Accuracy	0.25
LI3-8B-WT	WikiText [27]	Llama-3.1 8B [9]	Perplexity	0.25
FT5-SST2	SST2 [31]	Flan-T5-Base [5]	Accuracy	0.125

stationary point, the primal optimization gap C_{gap} emerges in both the dual stability bound and the long-term optimality gap.

3.4 Accuracy-Compression Function

The accuracy function $A(\eta)$ can be empirically evaluated on a held-out dataset $\mathcal{D}$ of N_{eval} samples. For each candidate compression vector η , the intermediate activations of the distributed pipeline are compressed according to η , passed to the next layer, and the resulting end-to-end inference quality (e.g., classification accuracy or perplexity) is measured on $\mathcal{D}$. Because this evaluation involves discrete operations such as top- k sparsification [1, 28] or quantization [24, 30], the resulting function $A(\eta)$ may be non-smooth or even discontinuous. We explored two approaches to smoothen this empirical evaluation. The first is *fitting*: a smooth function such as a polynomial, a multi-layer perceptron, etc., can be regressed from input/output samples of the accuracy function. The fitted function can be readily used to produce gradient estimates $\nabla A(\eta)$ while the held-out dataset $\mathcal{D}$ is not needed during optimization. A drawback is that different fitted functions may introduce biases. An alternative approach is to use *kernel-smoothening*, e.g., KNN with an inverse distance weighting kernel, a Gaussian kernel, etc. This is less prone to biases, but is more computationally intensive, and requires repeated evaluations of $A(\cdot)$ over the held-out dataset $\mathcal{D}$.

Interestingly, in the case of Gaussian kernel smoothing, one can produce a sampling estimator of the gradient $\nabla A(\eta)$ directly from randomized $A(\eta)$ function calls using Stein’s lemma [32]. Specifically, for a Gaussian perturbation $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ with smoothing bandwidth $\sigma > 0$, the gradient of the Gaussian-convolved function $\tilde{A}(\eta) = \mathbb{E}_{\mathbf{z}}[A(\eta + \sigma \mathbf{z})]$ satisfies: $\nabla_{\eta} \tilde{A}(\eta) = \frac{1}{\sigma} \mathbb{E}_{\mathbf{z}}[A(\eta + \sigma \mathbf{z}) \cdot \mathbf{z}]$, and this can be estimated *directly from sampled function evaluations*, without the need for either integration (w.r.t. the Gaussian kernel) or differentiation (w.r.t. η). We have implemented and explored both fitting and this kernel-smoothening via Stein’s lemma in our experiments; we provide more details in App. H, including quality of fit experiments for various ML tasks.

4 EVALUATION

4.1 Experimental Setup

Inference Tasks and Compression Schemes Vision and language tasks we study are summarized in Table 1. To compute $A_k(\eta_k)$, we use fitting and Stein kernel smoothening (see App. H) on 3 compression schemes: *Top- k sparsification* (T), whereby only a fraction η of the largest-magnitude activation coordinates are transmitted [1, 28]; *uniform quantization* (Q), which applies symmetric

Table 2: Offline experiment scenario summary. Where τ differs across compression schemes within the same scenario, it reflects per-scheme computational profiles measured on the Jetson testbed.

Scenario	Task ID	$c_{\min} \sim c_{\max}$ (MB)	R_k (Hz)	T	L_k	τ (ms)	a (MiB)
<i>Single-task</i>							
L-SG2B	G2-2B-SGPT	0.46–2.08	8	30	4	[8, 10, 8, 10]	[0.176, 0.176, 0.176]
L-SG2B7N	G2-2B-SGPT	3.4–16	6	48	7	[3.4, 3.4, 3.4, 2.3, 2.3, 2.3, 3.4]	[2.1, 2.1, 2.1, 2.1, 2.1, 2.1]
L-SG7B	G2-7B-SGPT	0.575–2.6	7	36	4	[13, 13, 13, 13]	[0.252, 0.252, 0.252]
V-TRS	RN56-CF10	0.06–0.3	10	50	4	[1, 1, 1, 1]	[0.066, 0.033, 0.016]
V-LOO	RN56-CF10	0.2–0.6	10	50	4	[1, 1, 1, 1]	[0.066, 0.033, 0.016]
L-5shot	L13-8B-MMLU	10–40	10	50	3	[14.7, 16.2, 14.8]	[4.2, 4.2]
L-WT8B	L13-8B-WT	5–9	4	30	4	[11.4, 11.4, 11.4, 11.4]	[4.19, 4.19, 4.19]
V-ST	RN56-CF10	1.08–1.79	1	100	4	T: [120, 61, 36, 19] Q: [131, 56, 36, 18] I8: [334, 159, 81, 21]	[6.6, 3.3, 1.6]
L-ST	FT5-SST2	1.35–1.95	2	100	4	T: [61, 52, 53, 105] Q: [43, 37, 38, 103] I8: [64, 47, 48, 104]	[1.2, 0.39, 0.39]
<i>Multi-task</i>							
L-SGPT3	G2-2B-SGPT	4.2–7	6	4		[8, 10, 8, 10]	[0.176, 0.176, 0.176]
	G2-2B-SGPT		5	20	4	[8, 10, 8, 10]	[0.176, 0.176, 0.176]
	G2-2B-SGPT		4	4		[8, 10, 8, 10]	[0.176, 0.176, 0.176]
V3-LOO	RN56-CF10	0.2–0.6	10	100	4	[1, 1, 1, 1]	[0.033, 0.049, 0.008]
	RN56-CF10		10	4		[1, 1, 1, 1]	[0.098, 0.016, 0.025]
	RN56-CF10		0.5	100	4	T: [120, 61, 36, 19] Q: [131, 56, 36, 18] I8: [334, 159, 81, 21]	[6.6, 3.3, 1.6]
LV-MT	FT5-SST2	1.08–1.79	1.5	100	4	T: [61, 52, 53, 105] Q: [43, 37, 38, 103] I8: [64, 47, 48, 104]	[1.2, 0.39, 0.39]

uniform quantization and maps activations to a reduced-precision fixed-point grid [24, 30]; and *LLM.int8* (I8), a hybrid scheme that isolates the largest-magnitude values for higher-precision storage while quantizing the remainder at lower precision [6]. In each scheme, $\eta \in [0, 1]$ is this retained fraction and controls how aggressive the transform is. Additional details for tasks, datasets, and compression schemes are in App. I.¹

Offline Experiments. We conduct 12 distinct offline experiments on linear topologies with the tasks from Table 2, with corresponding experiment parameters. Simulations run for T timeslots, with the link capacities sampled u.a.r. from $[c_{\min}, c_{\max}]$. When τ varies across schemes, it reflects per-scheme profiles characterized on *Jetson* testbed. Additional details are provided in App. I.

Testbed Experiments We also conduct 14 experiments on 3 testbeds:

(i) a CPU-only Raspberry Pi edge deployment for small LLMs, (ii) a Jetson Edge GPU deployment for LLM and vision workloads, and (iii) a multi-GPU testbed for Large-model and multi-task deployments with controllable communication bottlenecks. Experiment settings summarized in Table 3; further testbed deployment details are provided in App. J. Our focus is on No-CSI methods on testbed experiments, as channel is volatile and difficult to estimate.

Compression Rate Selection Algorithms For single-task experiments, we evaluate the following policies: Max-compression (MAX) is a baseline forcing $\eta_i = \eta_i^{\min}$, a strict lower bound on accuracy. No-compression (NONE) is a baseline transmitting uncompressed data ($\eta_i = 1$) to establish an upper bound on accuracy, ignoring constraints. Uniform-compression (UNI) is a CSI-aware policy enforcing uniform compression across stages, bottlenecking the system to the weakest link. Myopic-capacity (MYO) is a No-CSI policy assuming static networks by treating the most recent channel observation as the current capacity. Conservative-capacity (CONS) is a No-CSI pessimistic baseline assuming future capacities will match the lowest historically observed value. Moving-average (MA) is a No-CSI baseline assuming a moving average of past capacities to smooth

¹We pledge to make our code publicly available once the paper is accepted.

Table 3: Testbed experiment scenario summary. Where τ differs across compression schemes within the same scenario, it reflects per-scheme computational profiles measured on that testbed.

Scenario	Task ID	Network	R_k (Hz)	T	L_k	τ (ms)	a (MiB)
<i>PRESCIENT</i>							
P-V-G	RN56-CF10	1Gbps LAN	5.4	50	3	[8.796, 7.871, 2.239]	[6.25, 1.5625]
P-V-E	RN56-CF10	W-Edge	4.3	50	3	[8.796, 7.871, 2.239]	[6.25, 1.5625]
P-L1-G	L13-8B-WT	1Gbps LAN	4.0	50	3	[21.078, 29.998, 19.108]	[4.0, 4.0]
P-L1-E	L13-8B-WT	W-Edge	3.0	50	3	[21.078, 29.998, 19.108]	[4.0, 4.0]
P-L2-E	L13-8B-MMLU	W-Edge	2.5	50	3	[24.856, 40.983, 31.539]	[16, 16]
P-VV-G	RN56-CF10	1Gbps LAN	2.7	50	3	[8.796, 7.871, 2.239]	[6.25, 1.5625]
P-VV-E	RN56-CF10	W-Edge	1.2	50	3	[8.796, 7.871, 2.239]	[6.25, 1.5625]
P-VL-E	RN56-CF10	W-Edge	1.1	50	3	[8.796, 7.871, 2.239]	[6.25, 1.5625]
P-VL-W	L13-8B-WT	MANET	0.7	50	3	[21.078, 29.998, 19.108]	[4.0, 4.0]
P-VL-E	L13-8B-WT	W-Edge	0.5	50	3	[21.078, 29.998, 19.108]	[4.0, 4.0]
P-VL-E	L13-8B-WT	W-Edge	0.9	50	3	[21.078, 29.998, 19.108]	[4.0, 4.0]
<i>Jetson</i>							
J-V-S	RN56-CF10	Wi-Fi AP	T: 0.530 Q: 0.527 I8: 0.525	50	4	T: [120, 61, 36, 19] Q: [131, 56, 36, 18] I8: [334, 159, 81, 21]	[6.6, 3.3, 1.6]
J-L-S	FT5-SST2	Wi-Fi AP	T: 5.51 I8: 3.97 Q: 2.74	50	4	T: [61, 52, 53, 105] Q: [43, 37, 38, 103] I8: [64, 47, 48, 104]	[1.2, 0.39, 0.39]
J-LV-M	RN56-CF10	Wi-Fi AP	T: 0.273 I8: 0.273 Q: 0.820	20	4	T: [120, 61, 36, 19] Q: [131, 56, 36, 18] I8: [334, 159, 81, 21]	[6.6, 3.3, 1.6]
	FT5-SST2	Wi-Fi AP	T: 0.821 I8: 0.817	20	4	T: [61, 52, 53, 105] Q: [43, 37, 38, 103] I8: [64, 47, 48, 104]	[1.2, 0.39, 0.39]
<i>Raspberry Pi</i>							
R-L-SG2B	G2-2B-SGPT	WiFi 802.11ac	5	100	7	[378, 172, 145, 246, 155, 177, 143]	[4, 4, 4, 4, 4, 4, 4]

transient network noise. Finally, we evaluate our proposed methods: CSI-Aware Optimal (OURS-CSI) is our CSI-aware optimizer computing the closed-form optimal solution of Theorem 3.1. Alg. 1 with LCB (OURS-NOCSI) is our No-CSI optimizer using Alg. 1 with a 90% Lower Confidence Bound (LCB) as channel estimate $\hat{c}$.

In multi-task environments, beyond Max-compression (MAX) and No-compression (NONE), we implement policies that select both compression and resource allocations. Equal-share (EQ) is a CSI-aware baseline splitting resources equally, then applies optimal single-task compression. Proportional-share (PROP) is a CSI-aware baseline allocating resources proportionally based on inherent task computational loads and data sizes, before applying optimal single-task compression. Strict-priority (PRIO) is a CSI-aware baseline meeting minimum requirements for all tasks, then greedily funneling leftover resources to the highest-priority tasks. Decoupled-equal-split (D-EQUAL) is a No-CSI baseline splitting resources equally and running Alg. 1 for each task in isolation. Decoupled- λ -proportional (D-L-PROP) is a No-CSI baseline dynamically shifting resources proportional to dual variables ($\lambda_k(t)$) to explicitly aid struggling tasks. Moving-average (MA) is a No-CSI baseline feeding smoothed, moving-average channel estimates into the optimal CSI-aware multi-task solver. Lastly, we evaluate our proposed multi-task frameworks: CSI-Aware Optimal (OURS-CSI) is our CSI-aware optimizer solving the convex optimization in theorem 3.3. Alg. 2 with LCB (OURS-NOCSI) is our No-CSI optimizer using Algorithm 2 with a 90% LCB as channel estimate $\hat{c}$.

Evaluation Metrics. We evaluate system performance over the experiment horizon using three time-averaged metrics. *Aggregate Utility* ($\bar{U}$) measures the overall normalized weighted accuracy across active tasks: $\bar{U} = \frac{1}{T} \sum_{t=1}^T \frac{\sum_{k \in \mathcal{K}(t)} w_k A_k(\eta_k(t))}{\sum_{k \in \mathcal{K}(t)} w_k}$.

Average Delay ($\bar{D}$) represents the average bottleneck delay: $\bar{D} = \frac{1}{KT} \sum_{t=1}^T \sum_{k \in \mathcal{K}(t)} D_{act,k}(t)$. Finally, *Excess Delay* ($\bar{\Delta D}$)

Table 4: Offline results for single (upper) and multi-task (lower). We report agg. utility $\bar{U}$, avg. delay $\bar{D}$, and excess delay $\bar{\Delta D}$, across methods, grouped into reference, CSI-aware, and No-CSI methods. T, Q, and I8 denote Top-k, quantization, and LLM.int8, respectively. Infeasible entries ($\bar{\Delta D}/\bar{D} > 0.05$) are shaded light red. Bold and underlined values denote the best and second-best accuracies, respectively, among feasible entries.

Single-task Scenarios																										
Scenario	Metric	Reference				CSI-aware						No-CSI														
		NONE			MAX	UNI			OURS-CSI			MYO			CONS			MA			OURS-NOCSI					
		T	Q	I8		T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8			
L-SG2B	$\bar{U}$	1.00	0.802	0.365	0.889	0.968	0.989	0.994	0.985	<u>0.995</u>	0.997	0.982	0.992	0.996	0.925	0.961	<u>0.984</u>	0.992	0.996	0.998	0.957	0.991	0.980			
	$\bar{D}$	222	55.5	55.5	55.5	124	124	124	124	<u>124</u>	124	183	183	183	96.8	96.8	<u>96.8</u>	203	203	203	121	119	99.1			
	$\bar{\Delta D}$	97.1	-69.5	-69.5	-69.5	-1.46	-1.46	-1.46	-1.46	<u>-1.46</u>	-1.46	58.0	58.0	58.0	-28.2	-28.2	<u>-28.2</u>	77.8	77.8	77.8	-4.15	-5.67	-25.9			
L-SG2B7N	$\bar{U}$	1.00	0.444	0.155	0.344	0.824	0.805	0.949	<u>0.951</u>	0.909	0.985	0.944	0.891	0.981	0.750	0.378	<u>0.876</u>	0.979	0.980	0.994	0.825	0.797	0.949			
	$\bar{D}$	440	96.9	96.9	96.9	167	167	167	<u>167</u>	167	167	344	344	344	151	151	<u>151</u>	337	337	337	175	182	173			
	$\bar{\Delta D}$	274	-69.8	-69.8	-69.8	0.00	0.00	0.00	<u>0.00</u>	0.00	0.00	178	178	178	-15.7	-15.7	<u>-15.7</u>	171	171	171	7.86	15.3	5.94			
L-SG7B	$\bar{U}$	1.00	0.537	0.882	0.851	0.930	<u>0.994</u>	0.992	0.966	0.996	<u>0.994</u>	0.963	0.995	0.994	0.869	<u>0.989</u>	0.986	0.988	0.996	0.997	0.968	0.989	0.986			
	$\bar{D}$	269	67.3	67.3	67.3	141	<u>141</u>	141	141	141	<u>141</u>	230	230	230	115	<u>115</u>	115	234	234	234	146	113	113			
	$\bar{\Delta D}$	126	-75.5	-75.5	-75.5	-1.37	<u>-1.37</u>	-1.37	-1.37	-1.37	<u>-1.37</u>	87.2	87.2	87.2	-27.7	<u>-27.7</u>	-27.7	91.3	91.3	91.3	3.53	-29.5	-30.3			
V-TRS	$\bar{U}$	0.933	0.415	0.769	0.873	0.677	0.827	0.893	0.708	<u>0.926</u>	0.933	0.703	0.921	0.932	0.786	0.899	<u>0.903</u>	0.757	0.912	0.913	0.889	0.923	0.933			
	$\bar{D}$	476	59.5	59.5	59.5	101	101	101	101	<u>101</u>	101	175	175	175	75.5	75.5	<u>75.5</u>	164	164	164	85.3	104	119			
	$\bar{\Delta D}$	376	-40.5	-40.5	-40.5	1.23	1.23	1.23	1.23	<u>1.23</u>	1.23	75.4	75.4	75.4	-24.5	-24.5	<u>-24.5</u>	63.7	63.7	63.7	-14.7	3.99	19.0			
V-LOO	$\bar{U}$	0.933	0.415	0.769	0.873	0.894	0.913	<u>0.923</u>	0.900	0.933	<u>0.933</u>	0.898	0.923	0.933	0.831	0.903	<u>0.913</u>	0.910	0.930	0.933	0.901	0.913	0.933			
	$\bar{D}$	189	23.6	23.6	23.6	100	100	<u>100</u>	100	100	100	122	122	122	72.5	72.5	<u>72.5</u>	118	118	118	106	89.0	98.5			
	$\bar{\Delta D}$	88.7	-76.4	-76.4	-76.4	0.00	0.00	<u>0.00</u>	0.00	0.00	0.00	21.5	21.5	21.5	-27.5	-27.5	<u>-27.5</u>	18.3	18.3	18.3	6.06	-11.0	-1.50			
L-5shot	$\bar{U}$	0.595	0.420	0.255	0.543	0.531	0.479	<u>0.583</u>	0.545	0.499	0.590	0.575	0.559	0.592	0.467	0.355	<u>0.543</u>	0.493	0.417	0.555	0.487	0.425	0.555			
	$\bar{D}$	233	23.9	23.9	23.9	100	100	<u>100</u>	100	100	100	155	155	155	68.7	68.7	<u>68.7</u>	145	145	145	106	115	91.1			
	$\bar{\Delta D}$	133	-76.1	-76.1	-76.1	0.00	0.00	<u>0.00</u>	0.00	0.00	0.00	55.0	55.0	55.0	-31.3	-31.3	<u>-31.3</u>	44.9	44.9	44.9	5.74	15.1	-8.89			
L-WT8B	$\bar{U}$	1.00	0.727	0.206	0.818	0.866	0.894	<u>0.972</u>	0.909	0.922	0.988	0.904	0.917	0.984	0.833	0.843	<u>0.943</u>	0.910	0.942	0.990	0.856	0.904	0.962			
	$\bar{D}$	709	177	177	177	250	250	<u>250</u>	250	250	250	316	316	316	230	230	<u>230</u>	295	295	295	252	261	242			
	$\bar{\Delta D}$	459	-72.6	-72.6	-72.6	0.00	0.00	<u>0.00</u>	0.00	0.00	0.00	65.5	65.5	65.5	-20.2	-20.2	<u>-20.2</u>	44.8	44.8	44.8	1.79	10.8	-7.82			
V-ST	$\bar{U}$	0.933	0.414	0.869	0.931	0.749	<u>0.932</u>	<u>0.932</u>	0.781	0.933	<u>0.933</u>	0.780	<u>0.932</u>	0.932	0.738	0.917	0.932	0.780	0.932	0.932	0.786	0.925	0.931			
	$\bar{D}$	3810	476	473	477	1000	<u>1000</u>	<u>1000</u>	1000	1000	1000	1091	<u>1034</u>	1091	902	885	902	1084	1034	1084	1034	1017	477			
	$\bar{\Delta D}$	2810	-524	-527	-523	0.00	<u>0.00</u>	<u>0.00</u>	0.00	0.00	0.00	91.4	<u>33.7</u>	91.4	-97.9	-115	-97.9	83.5	34.3	83.5	34.3	16.5	-523			
L-ST	$\bar{U}$	0.930	0.850	0.506	0.930	0.930	<u>0.480</u>	0.930	0.930	<u>0.480</u>	0.930	0.930	0.480	0.930	0.930	0.480	0.930	0.930	0.480	0.930	<u>0.930</u>	0.501	0.930			
	$\bar{D}$	683	105	104	105	500	<u>500</u>	500	500	<u>500</u>	500	500	500	500	434	436	434	500	498	500	<u>304</u>	104	105			
	$\bar{\Delta D}$	183	-395	-396	-395	0.00	<u>0.00</u>	0.00	0.00	<u>0.00</u>	0.00	0.310	-0.004	0.0003	-65.7	-63.6	-65.7	-0.150	-1.52	-0.0002	<u>-196</u>	-396	-395			
Multi-task Scenarios																										
Scenario	Metric	Reference				CSI-aware								No-CSI												
		NONE			MAX	EQ			PROP			PRIO		OURS-CSI		D-EQUAL			D-L-PROP			OURS-NOCSI				
		T	Q	I8		T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8			
L-SGPT3	$\bar{U}$	1.00	0.888	$1.00 \cdot 10^{-4}$	0.364	0.967	0.581	0.379	<u>0.966</u>	0.626	0.375	0.951	0.491	0.543	0.964	$1.00 \cdot 10^{-4}$	0.364	0.935	$1.00 \cdot 10^{-4}$	0.364	<u>0.934</u>	$1.00 \cdot 10^{-4}$	0.364	<u>0.955</u>	$1.00 \cdot 10^{-4}$	0.364
	$\bar{D}$	352	87.9	87.9	87.9	131	131	131	<u>131</u>	131	131	131	131	131	131	92.1	92.1	104	87.9	87.9	<u>103</u>	87.9	87.9	133	87.8	87.8
	$\bar{\Delta D}$	663	-128	-128	-128	0.880	0.880	0.880	<u>1.38</u>	1.38	1.38	0.00	0.00	0.00	0.00	-115	-115	-80.3	-128	-128	<u>-128</u>	-128	-128	<u>7.50</u>	-128	-128
V3-LOO	$\bar{U}$	0.933	0.415	0.869	0.933	0.738	0.931	0.933	0.781	<u>0.933</u>	<u>0.933</u>	0.704	0.925	<u>0.933</u>	0.767	0.932	0.933	0.707	0.931	0.933	0.711	0.932	<u>0.933</u>	0.737	0.932	0.933
	$\bar{D}$	416	52.0	52.0	52.0	101	101	101	100	<u>100</u>	<u>100</u>	100	100	<u>100</u>	99.8	57.9	46.2	106	66.1	52.0	105	67.0	<u>51.9</u>	114	71.7	43.9
	$\bar{\Delta D}$	316	-48.0	-48.0	-48.0	0.620	0.620	0.620	0.00	<u>0.00</u>	<u>0.00</u>	0.00	0.00	<u>0.00</u>	-0.240	-42.1	-53.8	5.58	-34.0	-48.0	4.99	-33.0	<u>-48.1</u>	13.8	-28.4	-56.1
LV-MT	$\bar{U}$	0.931	0.632	0.684	0.930	0.859	0.706	0.931	0.880	0.721	0.931	0.877	0.724	<u>0.931</u>	0.892	0.714	0.931	0.919	0.718	<u>0.930</u>	0.919	0.719	<u>0.930</u>	0.883	0.690	0.930
	$\bar{D}$	4498	582	577	581	1335	1335	1335	1334	1334	1335	1333	1333	<u>1333</u>	1332	1331	797	2427	1281	<u>584</u>	2427	1281	<u>584</u>	1345	738	488
	$\bar{\Delta D}$	3164	-752	-757	-752	1.46	1.45	1.45	0.665	0.608	1.41	0.00	0.00	<u>0.00</u>	-1.61	-1.99	-536	1094	-52.7	<u>-749</u>	1094	-52.7	<u>-749</u>	11.2	-645	-845

evaluates QoS compliance by tracking the difference between actual bottleneck delay and the target threshold: $\bar{\Delta D} = \frac{1}{KT} \sum_{t=1}^T \sum_{k \in \mathcal{K}(t)} (D_{act,k}(t) - 1/R_k(t))$. We consider an excess delay higher than 5% of the average target delay as violating feasibility.

4.2 Experimental Results

Offline Experiments. Table 4 shows our offline experiment results. Across all scenarios, no-compression (NONE) is infeasible. In single-task experiments, among CSI-aware methods, OURS-CSI consistently achieves the highest feasible utility, whereas UNI bottlenecks all links to the worst-case channel, sacrificing up to 0.03

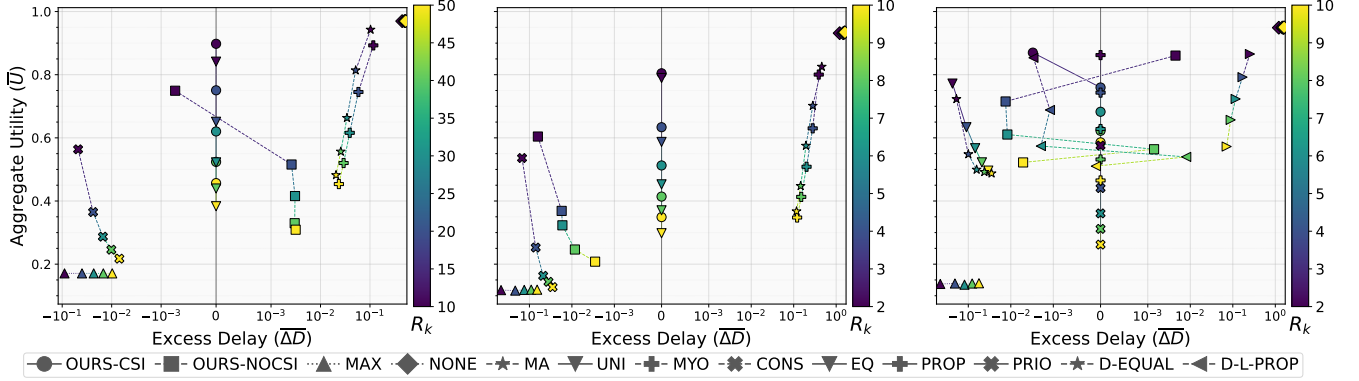


Figure 3: Aggregate utility versus excess delay for single-task and multi-task scenarios under varying target throughputs (R_k) and Top- k compression. Each line represents the trajectory of a specific algorithm. The left panel displays single-task MLP-MN, the middle panel shows single-task RN56-CF10, and the right panel illustrates a multi-task scenario combining both workloads with equal R_k .

utility under I8. In the No-CSI group, MYO and MA are infeasible in nearly every scenario, while CONS achieves feasibility through pessimistic estimation at a substantial accuracy cost. OURS-NOCSI balances both: it meets the delay constraint in most scenarios while maintaining utility competitive with the CSI-aware policy, trailing CONS in excess delay but exceeding it by up to 0.09 in utility.

For multi-task scenarios, resource contention makes the allocation strategy as important as the compression policy. OURS-CSI achieves the best feasible utility in every scenario, and on LV-MT with I8, it simultaneously attains the highest utility and lowest delay, outperforming other CSI-aware baselines by a wide margin. EQ and PROP remain feasible but cannot exploit asymmetric accuracy sensitivities across tasks. In the No-CSI group, OURS-NOCSI achieves the best feasible utility in three of four scenarios under I8, while D-EQUAL and D-L-PROP become infeasible under high task heterogeneity due to lacking jointly dual adaptation.

Testbed Experiments. The results in Table 5 confirm the offline trends under realistic conditions, focusing on No-CSI as the channel is hard to estimate (we provide a detailed discussion of the challenges of CSI-aware deployment across testbeds in App. K). MYO and MA remain infeasible across most scenarios, consistent with the offline findings, while CONS achieves feasibility through pessimistic estimation at reduced utility. OURS-NOCSI with I8 is the only No-CSI method achieving both feasibility and competitive utility on the challenging LLM scenarios (P-L1-G: 0.942, P-L1-E: 0.857), where all other No-CSI methods are infeasible. On the Jetson testbed, CONS and OURS-NOCSI are the only methods consistently meeting the delay constraint, with OURS-NOCSI matching utility at lower excess delay. On the hardest scenario (P-VV-G), every method except OURS-NOCSI is infeasible—only asynchronous per-task dual adaptation finds a feasible operating point (0.759 utility with I8). On P-VV-E, OURS-NOCSI with I8 achieves the best feasible utility (0.765) at the lowest delay, demonstrating efficient resource usage under imperfect channel knowledge. The cross-modal P-VL-E further confirms this: OURS-NOCSI with I8 achieves the highest utility (0.844) with the most negative excess delay (−440 ms). On the Jetson multi-task scenario, D-L-PROP and OURS-NOCSI with Q are the only feasible entries, both above 0.91 utility.

Varying Target Throughputs Fig. 3 (left) isolates a single-task setup deploying an MLP on the MNIST dataset across a 3-node routing path, whereas Fig. 3 (middle) examines a ResNet-56 model processing CIFAR-10 data over a 4-node path. Fig. 3 (right) illustrates a multi-task scenario across a 4-node path, where the MLP and ResNet models share the first three nodes and two connecting links. Both tasks share an identical target throughput, with utility weights set to 0.7 for the MLP and 1.0 for the ResNet. The accuracy-compression functions in these setups are based on Stein’s approach for the MLP task and the fitting approach for the ResNet task. In Fig. 3 (left) and (middle), NONE and MAX establish performance extremes. With perfect channel knowledge, OURS-CSI strictly enforces the target rate while maximizing utility, outperforming UNI. For no-CSI case, OURS-NOCSI tightly tracks the zero excess delay axis, avoiding severe constraint violations of MYO and MA, as well as unnecessary utility degradation of over-pessimistic CONS. Fig. 3 (right) demonstrates multi-task resource contention between MLP and ResNet-56 pipelines. OURS-CSI has non-positive excess delay while experiencing optimal utility via multi-dimensional water-filling, whereas static CSI-aware heuristics (EQ, PROP) and greedy policies (PRIO) fail to catch up. Crucially, OURS-NOCSI achieves a highly competitive delay-utility profile via block coordinate descent. By jointly optimizing the tasks, it outperforms No-CSI baselines like D-EQUAL and D-L-PROP, which optimize tasks in isolation. Furthermore, the MA baseline incurs high constraint violations because its weak estimation fails to track the highly variable channel.

5 CONCLUSION

We attempt to balance communication reduction against the accuracy of the downstream inference task. We achieve this by introducing a dynamically adjustable latent compression factor that continuously scales the intermediate payload size in response to fluctuating, real-time channel capacities, ensuring that strict QoS throughput constraints are satisfied without unnecessary degradation of inference accuracy.

REFERENCES

- [1] Song Bian, Dacheng Li, Hongyi Wang, Eric P Xing, and Shivaram Venkataraman. 2024. Does compressing activations help model parallel training?. In *MLSys*.

Table 5: Testbed results for single-task (upper block) and multi-task (lower block), excluding the CSI-aware methods. We report aggregate utility $\bar{U}$, average delay $\bar{D}$, and excess delay $\bar{\Delta D}$, across methods, grouped into reference baselines and No-CSI methods. Compression schemes T, Q, and I8 denote Top- k , quantization, and LLM.int8, respectively. Infeasible entries ($\bar{\Delta D}/\bar{D} > 0.05$) are shaded light red. Bold and underlined values denote the best and second-best results, respectively, among feasible entries within the No-CSI group.

Single-task Scenarios																			
Scenario	Metric	Reference						No-CSI											
		NONE			MAX			MYO			CONS			MA			OURS-NOCSI		
		T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8
R-L-SGB	$\bar{U}$	1.00	0.331	0.242	0.552			0.686	0.470	0.742	0.607	0.443	0.702	0.718	0.506	0.798	0.676	0.669	0.713
	$\bar{D}$	320	102	91.1	97.0			307	285	293	154	143	162	288	265	284	86.0	81.1	128
	$\bar{\Delta D}$	120	-97.9	-909	-993			107	84.9	92.7	-45.7	-57.3	-38.0	88.2	65.1	84.3	-16.5	-119	-17.8
P-V-G	$\bar{U}$	0.927	0.120	0.114	0.751	0.900	0.927	0.934	0.880	0.927	0.935	0.907	0.927	0.934	0.855	0.848	0.876	0.892	0.894
	$\bar{D}$	236.334	226.023	169.156	221.468	285.990	175.103	278.655	287.929	128.763	278.887	286.529	179.709	278.993	183.209	183.649	187.401	187.013	187.013
	$\bar{\Delta D}$	9.061	-1.250	-58.117	-5.805	58.718	58.718	44.773	60.656	-48.510	51.615	59.256	-47.564	51.720	-44.064	-43.624	-39.871	-40.470	-40.470
P-V-E	$\bar{U}$	0.927	0.120	0.114	0.751	0.551	0.933	0.908	0.543	0.933	0.909	0.546	0.933	0.909	0.533	0.684	0.845	0.872	0.875
	$\bar{D}$	676.002	197.228	197.498	209.463	285.748	264.321	255.146	284.461	268.449	248.806	285.830	263.583	252.235	235.449	222.658	205.917	205.917	205.917
	$\bar{\Delta D}$	437.907	-40.867	-40.957	-28.632	47.653	26.226	17.051	46.366	30.354	10.711	47.735	25.487	14.139	-2.646	-15.437	-32.178	-32.178	-32.178
P-L1-G	$\bar{U}$	1.000	0.192	0.028	0.682	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.639	0.066	0.942	0.942	0.942
	$\bar{D}$	296.553	281.092	236.484	273.836	338.583	294.891	307.931	338.732	287.429	316.593	343.194	294.311	331.681	262.893	260.805	236.245	236.245	236.245
	$\bar{\Delta D}$	33.395	18.534	-26.674	10.678	75.425	31.733	44.773	75.975	24.271	53.435	80.036	31.153	68.523	-21.062	-2.553	-26.913	-26.913	-26.913
P-L1-E	$\bar{U}$	1.000	0.192	0.028	0.677	0.957	0.013	0.989	0.956	0.013	0.987	0.957	0.013	0.989	0.691	0.087	0.857	0.857	0.857
	$\bar{D}$	790.384	286.550	335.390	291.654	862.651	457.479	434.263	433.741	433.214	457.973	458.281	448.635	433.632	322.722	314.389	343.769	343.769	343.769
	$\bar{\Delta D}$	420.014	-83.820	-34.981	-78.716	92.281	87.109	63.892	83.370	82.843	67.602	87.911	78.265	65.262	-32.649	-55.982	-26.601	-26.601	-26.601
P-L2-E	$\bar{U}$	0.467	0.267	0.283	0.283	0.408	0.300	0.477	0.417	0.300	0.481	0.417	0.300	0.478	0.269	0.282	0.394	0.394	0.394
	$\bar{D}$	901.330	274.393	280.272	297.253	385.275	445.917	874.369	390.544	451.298	862.662	386.328	655.560	863.350	290.246	309.594	358.331	358.331	358.331
	$\bar{\Delta D}$	401.330	-225.607	-219.728	-202.446	-114.725	145.917	374.369	-109.452	151.298	362.662	-113.672	155.560	363.350	-209.754	-190.406	-141.609	-141.609	-141.609
J-V-S	$\bar{U}$	0.933	0.414	0.870	0.930	0.894	0.933	0.932	0.856	0.933	0.932	0.893	0.933	0.932	0.894	0.925	0.930	0.930	0.930
	$\bar{D}$	3772	476	469	510	1955	1954	1960	1620	1780	1173	1941	1942	1992	1900	1270	972	972	972
	$\bar{\Delta D}$	1878	-1410	-1430	-1400	68.3	66.1	54.5	-267	-114	-733	53.4	47.5	86.9	12.5	-624	-934	-934	-934
J-L-S	$\bar{U}$	0.934	0.863	0.484	0.939	0.939	0.486	0.939	0.942	0.486	0.939	0.939	0.486	0.939	0.939	0.540	0.939	0.939	0.939
	$\bar{D}$	237	98.1	95.5	96.2	235	179	249	148	153	129	230	178	243	149	95.9	95.9	95.9	95.9
	$\bar{\Delta D}$	65.2	-95.2	-76.1	-156	41.5	7.86	-3.39	-45.8	-18.1	-123	36.2	6.60	-0.04	-44.0	-75.7	-153	-153	-153

Multi-task Scenarios																			
Scenario	Metric	Reference						No-CSI											
		NONE			MAX			D-EQUAL			D-L-PROP			MA			OURS-NOCSI		
		T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8	T	Q	I8
P-VV-G	$\bar{U}$	0.924	0.094	0.086	0.758	0.094	0.086	0.758	0.094	0.086	0.758	0.094	0.086	0.758	0.109	0.109	0.759	0.759	0.759
	$\bar{D}$	430	415	376	383	390	351	394	388	343	393	393	343	396	286	283	294	294	294
	$\bar{\Delta D}$	193	163	85.6	98.9	113	34.9	120	109	18.4	120	119	20.2	125	-95.1	-101	-79.5	-79.5	-79.5
P-VV-E	$\bar{U}$	0.924	0.094	0.086	0.758	0.094	0.086	0.763	0.094	0.086	0.763	0.094	0.086	0.763	0.389	0.325	0.765	0.765	0.765
	$\bar{D}$	1397	465	412	469	424	432	487	436	433	483	426	438	493	358	344	369	369	369
	$\bar{\Delta D}$	1842	-21.8	-128	-13.6	-104	-89.1	21.1	-80.6	-86.8	13.2	-101	-95.8	34	-236	-264	-215	-215	-215
P-VL-E	$\bar{U}$	0.962	0.153	0.0593	0.737	0.153	0.0593	0.737	0.153	0.0593	0.737	0.153	0.0593	0.737	0.45	0.232	0.844	0.844	0.844
	$\bar{D}$	1350	492	431	487	475	480	471	471	449	483	474	452	487	388	380	446	446	446
	$\bar{\Delta D}$	1368	-349	-471	-360	-384	-374	-392	-391	-436	-368	-384	-429	-360	-557	-574	-441	-441	-441
P-VL-W	$\bar{U}$	0.947	0.147	0.059	0.707	0.147	0.059	0.707	0.147	0.059	0.707	0.147	0.059	0.707	0.184	0.0718	0.716	0.716	0.716
	$\bar{D}$	6107	803	962	1249	766	940	1266	767	951	1252	769	968	1228	865	996	1225	1225	1225
	$\bar{\Delta D}$	9357	-1250	-934	-358	-1324	-976	-325	-1323	-956	-352	-1318	-922	-401	-1126	-864	-405	-405	-405
P-LL-E	$\bar{U}$	1.000	0.211	0.0327	0.716	0.211	0.0327	0.716	0.211	0.0327	0.716	0.211	0.0327	0.716	0.594	0.0327	0.898	0.898	0.898
	$\bar{D}$	1492	554	519	518	573	488	512	577	491	510	576	477	511	491	495	518	518	518
	$\bar{\Delta D}$	1555	-321	-390	-393	-283	-453	-405	-274	-446	-409	-277	-475	-407	-446	-438	-393	-393	-393
J-LV-M	$\bar{U}$	0.926	0.418	0.860	0.929	0.707	0.915	0.932	0.836	0.915	0.932	0.928	0.927	0.933	0.747	0.913	0.932	0.932	0.932
	$\bar{D}$	4330	1090	957	1680	2440	1230	2880	3590	1210	2670	4880	3990	13800	2400	1450	2730	2730	2730
	$\bar{\Delta D}$	1900	-1350	-1480	-760	9.55	-1210	443	1160	-1230	234	2450	1550	11400	-30.3	-986	285	285	285

- [2] Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin, and Colin A Raffel. 2023. Distributed Inference and Fine-tuning of Large Language Models Over The Internet. In *ACL*.
- [3] Jiasi Chen and Xukan Ran. 2019. Deep learning with edge computing: A review. *Proc. IEEE* 107, 8 (2019), 1655–1674.
- [4] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijay Srinivasan, and Kailash Garg. 2018. PACT: Parameterized clipping activation for quantized neural networks. In *ICLR*.
- [5] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *JMLR* 25, 70 (2024), 1–53.
- [6] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *NeurIPS*.
- [7] Osama Elgarhy and Luca Reggiani. 2018. Application of the Water Filling Algorithm to the Sum Rate Problem with Minimum Rate and Power Constraint. In *RTUWO*.
- [8] Gemma Team. 2024. Gemma 2: Improving Open Language Models at a Practical Size. *arXiv preprint arXiv:2408.00118* (2024). <https://arxiv.org/abs/2408.00118>
- [9] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024). <https://arxiv.org/abs/2407.21783>

- [10] Song Han, Huizi Mao, and William J Dally. 2016. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *ICLR*.
- [11] Kaiming He, X. Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. In *CVPR*.
- [12] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *ICLR*.
- [13] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015).
- [14] Chenghao Hu and Baochun Li. 2022. Distributed Inference with Deep Learning Models across Heterogeneous Edge Devices. In *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*. 330–339. <https://doi.org/10.1109/INFOCOM48880.2022.9796896>
- [15] Haochen Hua, Yutong Li, Tonghe Wang, Nanqing Dong, Wei Li, and Junwei Cao. 2023. Edge computing with artificial intelligence: A machine learning perspective. *Comput. Surveys* 55, 9 (2023), 1–35.
- [16] Tong Jian, Debashri Roy, Batool Salehi, Nasim Soltani, Kaushik Chowdhury, and Stratis Ioannidis. 2023. Communication-aware DNN pruning. In *INFOCOM*.
- [17] Alex Krizhevsky. 2009. Learning Multiple Layers of Features from Tiny Images.
- [18] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. <https://doi.org/10.1109/5.726791>
- [19] Chengzhang Li, Peizhong Ju, Atilla Eryilmaz, and Ness Shroff. 2024. Efficient Multi-dimensional Compression for Network-edge Classification. In *MobiHoc*.
- [20] Chengzhang Li, Peizhong Ju, Atilla Eryilmaz, and Ness Shroff. 2025. Two Levels Are All You Need: Simplifying Data Compression for Timely Edge Classification. In *MobiHoc*.
- [21] Pengzhen Li, Erdem Koyuncu, and Hulya Seferoglu. 2024. Adaptive and Resilient Model-Distributed Inference in Edge Computing Systems. *IEEE Transactions on Mobile Computing (or appropriate venue)* (2024).
- [22] Pengzhen Li, Hulya Seferoglu, and Erdem Koyuncu. 2023. Model-distributed inference in multi-source edge networks. In *2023 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW)*. IEEE, 1–5.
- [23] Teng Li and Hulya Seferoglu. 2025. Priority-aware model-distributed inference at edge networks. In *2025 IEEE International Conference on Machine Learning for Communication and Networking (ICMLCN)*. IEEE, 1–6.
- [24] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. In *MLSys*.
- [25] LMSYS Org. 2023. ShareGPT. <https://sharegpt.com/>. [Accessed 2026-03-31].
- [26] Davide Macario, Hulya Seferoglu, and Erdem Koyuncu. 2025. Model-distributed inference for large language models at the edge. In *LANMAN*.
- [27] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
- [28] Zhangqiang Ming, Yuchong Hu, Wenxiang Zhou, Xinjue Zheng, Chenxuan Yao, and Dan Feng. 2024. ADTopk: all-dimension top-k compression for high-performance data-parallel DNN training. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*. 135–147.
- [29] Sameera Ramasinghe, Thalaisyasingam Ajanthan, Gil Avraham, Yan Zuo, and Alexander Long. [n. d.]. Beyond top-k: Structured sparsification for compression in pipeline parallel. In *ICLR 2025 Workshop on Modularity for Collaborative, Decentralized, and Continual Deep Learning*.
- [30] Xuan Shen, Peiyang Dong, Lei Lu, Zhenglun Kong, Zhengang Li, Ming Lin, Chao Wu, and Yanzhi Wang. 2024. Agile-quant: Activation-guided quantization for faster inference of LLMs on the edge. In *AAAI*.
- [31] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In *EMNLP*.
- [32] Charles M. Stein. 1981. Estimation of the Mean of a Multivariate Normal Distribution. *The Annals of Statistics* 9, 6 (1981), 1135–1151. <https://doi.org/10.1214/aos/1176345632>
- [33] Yang Wang, Qingbao Wu, Shiru Huang, Emin Gün Sirer, et al. 2021. PipeEdge: Pipeline parallelism for large-scale model inference on heterogeneous edge devices. In *2021 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 214–227.
- [34] Chien-Ting Wu, Yin-Wen Lin, Wei-Hsin Chen, et al. 2023. Federated split learning for edge intelligence: A comprehensive review. *IEEE Communications Surveys & Tutorials* 25, 2 (2023), 1234–1260.
- [35] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. 2024. A Survey on Model Compression for Large Language Models. *Transactions of the Association for Computational Linguistics* 12 (2024), 1556–1577.

A PROOF OF THEOREM 3.1

PROOF. We begin by determining the feasibility conditions for Prob. 1:

Lemma A.1 (Single-Task Feasibility Condition). *Problem 1 is feasible if and only if:*

$$\tau_i \leq \frac{1}{R(t)}, \quad \forall i \in [L], \quad \text{and} \quad \frac{a_i \eta_i^{\min}}{c_i(t)} \leq \frac{1}{R(t)}, \quad \forall i \in [L-1]. \quad (11)$$

PROOF. We prove this in both directions.

Necessity: Assume the problem is feasible. Let $\eta(t)$ be a feasible solution. The constraint in (1b) dictates that the maximum of a set is bounded by $\frac{1}{R(t)}$, meaning every individual element in the set must be bounded by $\frac{1}{R(t)}$. Consequently, for the computation delays, it directly establishes (11). For the communication delays, we must have $\frac{a_i}{c_{e_i}(t)} \eta_i(t) \leq \frac{1}{R(t)}$ for all $i \in [L-1]$. Because the solution is drawn from the valid domain, $\eta_i(t) \geq \eta_i^{\min}$. Substituting this lower bound preserves the inequality:

$$\frac{a_i \eta_i^{\min}}{c_{e_i}(t)} \leq \frac{a_i \eta_i(t)}{c_{e_i}(t)} \leq \frac{1}{R(t)}, \quad \forall i \in [L-1].$$

Sufficiency: Assume condition (11) holds. We demonstrate feasibility by constructing a valid solution. Let $\eta_i(t) = \eta_i^{\min}$ for all $i \in [L-1]$. By definition, this vector satisfies the domain constraint $\eta(t) \in \prod_{i=1}^{L-1} [\eta_i^{\min}, 1]$. Condition (11) ensures that $\tau_i \leq \frac{1}{R(t)}$ for all $i \in [L]$ and $\frac{a_i}{c_{e_i}(t)} \eta_i^{\min} \leq \frac{1}{R(t)}$ for all $i \in [L-1]$. Since every individual computation and communication delay term under this configuration is bounded by $\frac{1}{R(t)}$, their maximum is also bounded by $\frac{1}{R(t)}$. Thus, $\eta(t) = \eta^{\min}$ satisfies (1b), proving the feasible set is non-empty. $\square$

To prove Theorem 3.1, observe that, since the problem is feasible, the inequalities regarding τ_i are inherently satisfied and independent of the choice of $\eta(t)$. Thus, we can rewrite the constraints entirely in terms of $\eta_i(t)$. The upper bound imposed by the target delay $\frac{1}{R(t)}$ is $\eta_i(t) \leq \frac{c_{e_i}(t)}{R(t)a_i}$. Combined with the natural domain constraint $\eta_i(t) \leq 1$, the feasible region for each compression factor is:

$$\eta_i(t) \in \left[\eta_i^{\min}, \min \left\{ 1, \frac{c_{e_i}(t)}{R(t)a_i} \right\} \right]$$

Because $A(\eta(t))$ is monotonically non-decreasing with respect to $\eta_i(t)$, maximizing the objective function strictly equates to maximizing each $\eta_i(t)$ independently. Therefore, the optimal value $\eta_i^*(t)$ is the upper bound of its feasible interval. $\square$

B PROOF OF LEMMA 3.2

PROOF. We prove this by demonstrating both necessity and sufficiency.

Necessity: Assume Problem 2 is feasible. Thus, there exist sets of variables $s^{\text{comp}}(t)$, $s^{\text{comm}}(t)$, and $\eta(t)$ satisfying all constraints. From (4b), we rearrange to find $s_{i,k}^{\text{comp}}(t) \geq \tau_{i,k} R_k(t)$. Summing this inequality over all active tasks $k \in \mathcal{K}(t)$ and their corresponding stages i assigned to any physical node $v \in \mathcal{V}$ (where $v_{i,k} = v$), and applying the capacity constraint (2) yields:

$$1 \geq \sum_{k \in \mathcal{K}(t)} \sum_{i: v_{i,k}=v} s_{i,k}^{\text{comp}}(t) \geq \sum_{k \in \mathcal{K}(t)} \sum_{i: v_{i,k}=v} \tau_{i,k} R_k(t),$$

which establishes the first condition. Similarly, rearranging (4c) yields $s_{i,k}^{\text{comm}}(t) \geq \frac{a_{i,k} \eta_{i,k}(t) R_k(t)}{c_{i,k}(t)}$. By the domain constraint (4d), $\eta_{i,k}(t) \geq \eta_{i,k}^{\min}$. Substituting this lower bound and summing over all active tasks $k \in \mathcal{K}(t)$ and their corresponding hops i assigned to any physical edge $e \in \mathcal{E}$ (where $e_{i,k} = e$) gives:

$$1 \geq \sum_{k \in \mathcal{K}(t)} \sum_{i: e_{i,k}=e} s_{i,k}^{\text{comm}}(t) \geq \sum_{k \in \mathcal{K}(t)} \sum_{i: e_{i,k}=e} \frac{a_{i,k} \eta_{i,k}^{\min} R_k(t)}{c_{i,k}(t)}.$$

Sufficiency: Assume the conditions in the lemma hold. We construct a valid solution by setting the variables to their minimum required limits: let $s_{i,k}^{\text{comp}}(t) = \tau_{i,k} R_k(t)$, $\eta_{i,k}(t) = \eta_{i,k}^{\min}$, and $s_{i,k}^{\text{comm}}(t) = \frac{a_{i,k} \eta_{i,k}^{\min} R_k(t)}{c_{i,k}(t)}$ for all corresponding i, k . By definition, $\eta_{i,k}(t)$ satisfies (4d). These allocations exactly satisfy the latency constraints (4b) and (4c) as equalities. Furthermore, by the assumed conditions in the lemma statement, the sums of these allocated shares do not exceed 1, thus satisfying the node and edge capacity constraints (2) and (3). Therefore, a feasible solution exists. $\square$

C PROOF OF THEOREM 3.3

PROOF. Because the objective function (4) is a monotonically non-decreasing function of $\eta_k(t)$, maximizing the objective equates to maximizing each $\eta_{i,k}(t)$ independently, constrained only by the allocated bandwidth $s_{i,k}^{\text{comm}}(t)$. From (4c) and (4d), the feasible upper bound for compression is:

$$\eta_{i,k}(t) \leq \min \left(1, \frac{c_{i,k}(t)}{R_k(t) a_{i,k}} s_{i,k}^{\text{comm}}(t) \right).$$

Since we aim to maximize accuracy, the optimal compression factor is exactly this upper bound. We can define this relationship as a function of the allocated bandwidth:

$$\eta_{i,k}^*(s_{i,k}^{\text{comm}}(t)) \triangleq \min \left(1, \frac{c_{i,k}(t)}{R_k(t)a_{i,k}} s_{i,k}^{\text{comm}}(t) \right).$$

The function $\eta_{i,k}^*$ is the pointwise minimum of two affine functions, and is therefore concave. Substituting $\eta_{i,k}(t) = \eta_{i,k}^*(s_{i,k}^{\text{comm}}(t))$ into the objective function yields $\sum_{k=1}^K w_k A_k(\eta_{i,k}^*(s_{i,k}^{\text{comm}}(t)))$. Because A_k is concave and non-decreasing, the composition of A_k with the concave function $\eta_{i,k}^*$ is also concave. A positively weighted sum of concave functions remains concave.

Furthermore, the computation allocation variables $s^{\text{comp}}(t)$ are decoupled from the objective and can be trivially satisfied by assigning $s_{i,k}^{\text{comp}}(t) = \tau_{i,k} R_k(t)$ (guaranteed valid by Lemma 3.2). Thus, the optimization simplifies to maximizing a concave objective function subject to linear inequality constraints on the communication allocations:

$$\begin{aligned} \text{Maximize: } & \sum_{k=1}^K w_k A_k \left(\dots, \min \left(1, \frac{c_{i,k}(t)}{R_k(t)a_{i,k}} s_{i,k}^{\text{comm}}(t) \right), \dots \right) \\ \text{subj. to: } & \sum_{k \in \mathcal{K}(t)} \sum_{i: e_{i,k}=e} s_{i,k}^{\text{comm}}(t) \leq 1, \quad \forall e \in \mathcal{E}, \\ & s_{i,k}^{\text{comm}}(t) \geq \frac{a_{i,k} \eta_{i,k}^{\min} R_k(t)}{c_{i,k}(t)}, \quad \forall k \in \mathcal{K}(t), \forall i \in [L_k - 1]. \end{aligned}$$

Maximizing a concave function over a convex set defined by linear constraints constitutes a standard convex optimization problem, completing the proof. $\square$

D WATER-FILLING ALGORITHM

Optimality Conditions (KKT). To gain insight into the convex optimization problem formulated in Theorem 3.3, we apply the Karush-Kuhn-Tucker (KKT) conditions. We attach a Lagrange multiplier $\lambda_e \geq 0$ to the communication bandwidth capacity constraint of each physical edge $e \in \mathcal{E}$, and a multiplier $\mu_{i,k} \geq 0$ to the minimum QoS bandwidth constraint for the i -th transmission hop of task k .

Let $s_{i,k}^{\min}(t) = \frac{a_{i,k} \eta_{i,k}^{\min} R_k(t)}{c_{i,k}(t)}$ denote the minimum bandwidth fraction required for task k at its i -th hop during time slot t . The Lagrangian is:

$$\mathcal{L}(\eta(t), s^{\text{comp}}(t), s^{\text{com}}(t), \lambda, \mu) = \sum_{k=1}^K w_k A_k(\eta_k(t)) - \sum_{e \in \mathcal{E}} \lambda_e \left(\sum_{k \in \mathcal{K}(t)} \sum_{i: e_{i,k}=e} s_{i,k}^{\text{com}}(t) - 1 \right) - \sum_{k \in \mathcal{K}(t)} \sum_{i=1}^{L_k-1} \mu_{i,k} (s_{i,k}^{\min}(t) - s_{i,k}^{\text{com}}(t)). \quad (12)$$

Assuming the solution lies in the active region where $\eta_{i,k}(t) < 1$, we take the partial derivative of the Lagrangian with respect to the bandwidth allocation $s_{i,k}^{\text{com}}(t)$ and apply the chain rule. Note that differentiating the capacity penalty for a specific $s_{i,k}^{\text{com}}(t)$ isolates the shadow price $\lambda_{e_{i,k}}$ of the specific edge traversed by that hop:

$$\frac{\partial \mathcal{L}}{\partial s_{i,k}^{\text{com}}(t)} = w_k \frac{\partial A_k}{\partial \eta_{i,k}(t)} \cdot \frac{\partial \eta_{i,k}(t)}{\partial s_{i,k}^{\text{com}}(t)} - \lambda_{e_{i,k}} + \mu_{i,k} = 0. \quad (13)$$

Substituting the partial derivative $\frac{\partial \eta_{i,k}(t)}{\partial s_{i,k}^{\text{com}}(t)} = \frac{c_{i,k}(t)}{R_k(t)a_{i,k}}$, we obtain the stationarity condition:

$$\frac{w_k c_{i,k}(t)}{R_k(t)a_{i,k}} \frac{\partial A_k}{\partial \eta_{i,k}(t)} + \mu_{i,k} = \lambda_{e_{i,k}}, \quad \forall k \in \mathcal{K}(t), \forall i \in [L_k - 1]. \quad (14)$$

Furthermore, the KKT complementary slackness conditions require that:

$$\lambda_e \left(\sum_{k \in \mathcal{K}(t)} \sum_{i: e_{i,k}=e} s_{i,k}^{\text{com}}(t) - 1 \right) = 0, \quad \forall e \in \mathcal{E}, \quad (15)$$

$$\mu_{i,k} (s_{i,k}^{\min}(t) - s_{i,k}^{\text{com}}(t)) = 0, \quad \forall k \in \mathcal{K}(t), \forall i \in [L_k - 1]. \quad (16)$$

Algorithm. Equations (14) through (16) reveal a **Per-Link Water-Filling with a QoS Floor** strategy.

- Each physical edge e possesses a shadow price λ_e , representing the marginal utility of additional bandwidth at that link.
- If a task k is allocated strictly more than its minimum required bandwidth ($s_{i,k}^{\text{com}}(t) > s_{i,k}^{\min}(t)$) for hop i , the complementary slackness condition (16) dictates that $\mu_{i,k} = 0$. For these tasks, the solver equalizes their effective marginal accuracy gains to the traversed edge's shadow price $\lambda_{e_{i,k}}$.
- If a task's marginal utility is too low to compete for additional bandwidth, the optimization algorithm will attempt to starve it. However, the constraint halts the reduction at $s_{i,k}^{\min}(t)$. At this boundary, $\mu_{i,k} > 0$ compensates for the task's lack of competitiveness, artificially satisfying the stationarity condition (14) and pegging the task to its minimum viable bandwidth.

E PROOF OF THEOREM 3.4

PROOF. Part A: Proof of Constraint Satisfaction

From the dual update rule $\lambda_{t+1} = \max \left\{ \epsilon, \lambda_t + D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right\}$, we establish the fundamental inequality:

$$\lambda_{t+1} \geq \lambda_t + D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)}$$

Rearranging to isolate the instantaneous constraint violation yields:

$$D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \leq \lambda_{t+1} - \lambda_t$$

Taking the expectation and averaging over $t = 1, \dots, T$ produces a telescoping sum:

$$\frac{1}{T} \sum_{t=1}^T \mathbb{E} \left[D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right] \leq \frac{\mathbb{E}[\lambda_{T+1}] - \mathbb{E}[\lambda_1]}{T}$$

As rigorously established in Part B, the expected dual variable $\mathbb{E}[\lambda_t]$ is uniformly bounded by a constant. Consequently, $\lim_{T \rightarrow \infty} \frac{\mathbb{E}[\lambda_{T+1}]}{T} = 0$. Taking the limit as $T \rightarrow \infty$ gives:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E} \left[D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right] \leq 0$$

This confirms that the long-term expected delay constraint is strictly satisfied.

Part B: Bounding the Multiplier

Define the Lyapunov function as $L(\lambda_t) = \frac{1}{2}\lambda_t^2$. We utilize the projection property $(\max\{\epsilon, x\})^2 \leq x^2 + \epsilon^2$ for all $x \in \mathbb{R}$. Applying this to the dual update rule yields:

$$\lambda_{t+1}^2 \leq \left(\lambda_t + D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right)^2 + \epsilon^2$$

Expanding the square and rearranging provides the Lyapunov drift bound:

$$\frac{1}{2}\lambda_{t+1}^2 - \frac{1}{2}\lambda_t^2 \leq \lambda_t \left(D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right) + \frac{1}{2} \left(D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right)^2 + \frac{1}{2}\epsilon^2$$

To introduce the utility-delay trade-off, we subtract the scaled utility $\frac{1}{\mu}A(\boldsymbol{\eta}(t))$ from both sides. Let $\Delta(\boldsymbol{\eta}(t)) = D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \hat{D}(\boldsymbol{\eta}(t), \hat{\mathbf{c}}(t))$ denote the estimation error. Adding and subtracting the estimator term $\lambda_t \hat{D}(\boldsymbol{\eta}(t), \hat{\mathbf{c}}(t))$ on the right-hand side yields:

$$\begin{aligned} \frac{1}{2}\lambda_{t+1}^2 - \frac{1}{2}\lambda_t^2 - \frac{1}{\mu}A(\boldsymbol{\eta}(t)) &\leq -\frac{1}{\mu}A(\boldsymbol{\eta}(t)) + \lambda_t \left(\hat{D}(\boldsymbol{\eta}(t), \hat{\mathbf{c}}(t)) - \frac{1}{R(t)} \right) \\ &\quad + \lambda_t \Delta(\boldsymbol{\eta}(t)) + \frac{1}{2} \left(D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right)^2 + \frac{1}{2}\epsilon^2 \end{aligned}$$

By design, our algorithm minimizes the primary terms $-\frac{1}{\mu}A(\boldsymbol{\eta}(t)) + \lambda_t \left(\hat{D}(\boldsymbol{\eta}(t), \hat{\mathbf{c}}(t)) - \frac{1}{R(t)} \right)$ with respect to any alternative choice of $\boldsymbol{\eta}(t)$. We can thus upper-bound this expression by evaluating it at the strictly feasible, maximum compression baseline policy $\boldsymbol{\eta}^{\min}$:

$$\begin{aligned} \frac{1}{2}\lambda_{t+1}^2 - \frac{1}{2}\lambda_t^2 - \frac{1}{\mu}A(\boldsymbol{\eta}(t)) &\leq -\frac{1}{\mu}A(\boldsymbol{\eta}^{\min}) + \lambda_t \left(\hat{D}(\boldsymbol{\eta}^{\min}, \hat{\mathbf{c}}(t)) - \frac{1}{R(t)} \right) \\ &\quad + \lambda_t \Delta(\boldsymbol{\eta}(t)) + \frac{1}{2} \left(D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right)^2 + \frac{1}{2}\epsilon^2 \end{aligned}$$

Substituting the identity $\hat{D}(\boldsymbol{\eta}^{\min}, \hat{\mathbf{c}}(t)) = D(\boldsymbol{\eta}^{\min}, \mathbf{c}(t)) - \Delta(\boldsymbol{\eta}^{\min})$ and rearranging terms yields:

$$\begin{aligned} \frac{1}{2}\lambda_{t+1}^2 - \frac{1}{2}\lambda_t^2 - \frac{1}{\mu}A(\boldsymbol{\eta}(t)) &\leq -\frac{1}{\mu}A(\boldsymbol{\eta}^{\min}) + \lambda_t \left(D(\boldsymbol{\eta}^{\min}, \mathbf{c}(t)) - \frac{1}{R(t)} \right) \\ &\quad + \lambda_t \left(\Delta(\boldsymbol{\eta}(t)) - \Delta(\boldsymbol{\eta}^{\min}) \right) + \frac{1}{2} \left(D(\boldsymbol{\eta}(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right)^2 + \frac{1}{2}\epsilon^2 \end{aligned}$$

To bound the error difference term $\Delta(\boldsymbol{\eta}(t)) - \Delta(\boldsymbol{\eta}^{\min})$, we analyze the bottleneck mechanics. Because $\boldsymbol{\eta}(t) \geq \boldsymbol{\eta}^{\min}$, we consider the exhaustive set of physically possible bottleneck transitions:

- **Case 1:** If both configurations are compute-bottlenecked, $\Delta(\boldsymbol{\eta}(t)) = 0$ and $\Delta(\boldsymbol{\eta}^{\min}) = 0$, yielding a difference of 0.
- **Case 2:** If $\boldsymbol{\eta}^{\min}$ is compute-bottlenecked but $\boldsymbol{\eta}(t)$ is communication-bottlenecked, $\Delta(\boldsymbol{\eta}^{\min}) = 0$, making the difference exactly $\Delta(\boldsymbol{\eta}(t))$.

- **Case 3:** If both are communication-bottlenecked, the delay is monotonic with η . If the estimator underestimates the delay ($\Delta \geq 0$), then $\Delta(\eta^{\min}) \geq 0$, yielding a difference $\leq \Delta(\eta(t))$. If the estimator overestimates the delay ($\Delta \leq 0$), the difference is strictly ≤ 0 .

In all cases, the difference is strictly bounded by the positive error margin: $\Delta(\eta(t)) - \Delta(\eta^{\min}) \leq \max\{0, \Delta(\eta(t))\}$. Based on Assumption 3, we bound the conditional expectation of this positive error by δ^+ .

Taking the expectation of the full drift inequality, applying Assumption 4 ($\mathbb{E}[D(\eta^{\min}, \mathbf{c}(t)) - \frac{1}{R(t)}] \leq -\gamma$), Assumption 3 ($\mathbb{E}[\max\{0, \Delta(\eta(t))\} | \lambda_t] \leq \delta^+$), and Assumption 2 ($\mathbb{E}[(D(\eta(t), \mathbf{c}(t)) - \frac{1}{R(t)})^2] \leq \xi^2$):

$$(\gamma - \delta^+) \mathbb{E}[\lambda_t] \leq \frac{\mathbb{E}[A(\eta(t))] - A(\eta^{\min})}{\mu} + \frac{1}{2}(\xi^2 + \epsilon^2) + \frac{1}{2}\mathbb{E}[\lambda_t^2] - \frac{1}{2}\mathbb{E}[\lambda_{t+1}^2]$$

Averaging over T slots, applying Assumption 1 ($A(\eta) \leq A_{\max}$), and taking the limit $T \rightarrow \infty$ establishes the multiplier bound:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[\lambda_t] \leq \frac{A_{\max} - A_{\min}}{\mu(\gamma - \delta^+)} + \frac{\xi^2 + \epsilon^2}{2(\gamma - \delta^+)} \quad (17)$$

Part C: Bounding the Optimality Gap

We return to the fundamental drift inequality (prior to evaluating $\eta^{\min}$) and instead evaluate it under the strictly optimal, stationary policy $\eta^*(t)$. This policy makes decisions independent of λ_t , achieves the optimal expected accuracy A^* , and satisfies the desired long-term constraint $\mathbb{E}[D(\eta^*(t), \mathbf{c}(t)) - \frac{1}{R(t)}] \leq 0$. Substituting $\eta^*(t)$ and expanding the estimate yields:

$$\begin{aligned} & \frac{1}{2}\lambda_{t+1}^2 - \frac{1}{2}\lambda_t^2 - \frac{1}{\mu}A(\eta(t)) - \lambda_t \Delta(\eta(t)) - \frac{1}{2} \left(D(\eta(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right)^2 - \frac{1}{2}\epsilon^2 \\ & \leq -\frac{1}{\mu}A(\eta^*(t)) + \lambda_t \left(\hat{D}(\eta^*(t), \hat{\mathbf{c}}(t)) - \frac{1}{R(t)} \right) \\ & = -\frac{1}{\mu}A(\eta^*(t)) + \lambda_t \left(D(\eta^*(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right) - \lambda_t \Delta(\eta^*(t)) \end{aligned}$$

Moving $-\lambda_t \Delta(\eta(t))$ to the right-hand side isolates the error difference $\lambda_t (\Delta(\eta(t)) - \Delta(\eta^*(t)))$. Because $\eta^*(t)$ is a stationary policy independent of λ_t , the expectation of its cross-terms factorizes cleanly.

First, for the queue term:

$$\begin{aligned} \mathbb{E} \left[\lambda_t \left(D(\eta^*(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right) \right] &= \mathbb{E}_{\lambda_t} \left[\lambda_t \mathbb{E} \left[D(\eta^*(t), \mathbf{c}(t)) - \frac{1}{R(t)} \mid \lambda_t \right] \right] \\ &= \mathbb{E}_{\lambda_t} \left[\lambda_t \mathbb{E} \left[D(\eta^*(t), \mathbf{c}(t)) - \frac{1}{R(t)} \right] \right] \\ &\leq \mathbb{E}_{\lambda_t} [\lambda_t \cdot 0] = 0 \end{aligned}$$

Second, for the error difference term, we apply the definitions from Assumption 3. For the dynamic policy, we use the raw expected error bound $\mathbb{E}[\Delta(\eta(t)) | \lambda_t] \leq \delta$. For the stationary optimal policy, we use $\mathbb{E}[\Delta(\eta^*(t))] \leq \delta^*$. Because η^* is independent of λ_t , the expectation factorizes:

$$\begin{aligned} \mathbb{E}[\lambda_t (\Delta(\eta(t)) - \Delta(\eta^*(t)))] &= \mathbb{E}[\lambda_t \Delta(\eta(t))] - \mathbb{E}[\lambda_t \Delta(\eta^*(t))] \\ &= \mathbb{E}_{\lambda_t} [\lambda_t \mathbb{E}[\Delta(\eta(t)) | \lambda_t]] - \mathbb{E}[\lambda_t] \mathbb{E}[\Delta(\eta^*(t))] \\ &\leq \delta \mathbb{E}[\lambda_t] - \delta^* \mathbb{E}[\lambda_t] = (\delta - \delta^*) \mathbb{E}[\lambda_t] \end{aligned}$$

Taking the expectation of the full drift inequality and applying these decoupled bounds gives:

$$\mathbb{E}[A(\eta(t))] \geq A^* - \mu(\delta - \delta^*) \mathbb{E}[\lambda_t] - \frac{\mu}{2}(\xi^2 + \epsilon^2) + \frac{\mu}{2}\mathbb{E}[\lambda_t^2] - \frac{\mu}{2}\mathbb{E}[\lambda_{t+1}^2]$$

Averaging over T slots:

$$\frac{1}{T} \sum_{t=1}^T \mathbb{E}[A(\eta(t))] \geq A^* - \frac{\mu(\delta - \delta^*)}{T} \sum_{t=1}^T \mathbb{E}[\lambda_t] - \frac{\mu}{2}(\xi^2 + \epsilon^2) + \frac{\mu}{2T}\mathbb{E}[\lambda_1^2] - \frac{\mu}{2T}\mathbb{E}[\lambda_{T+1}^2]$$

Since $\mathbb{E}[\lambda_{T+1}^2] \geq 0$ and $\frac{1}{T}\mathbb{E}[\lambda_1^2] \rightarrow 0$, taking the limit as $T \rightarrow \infty$ gives:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[A(\eta(t))] \geq A^* - \frac{\mu(\xi^2 + \epsilon^2)}{2} - \mu(\delta - \delta^*) \left(\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[\lambda_t] \right)$$

By substituting the long-term multiplier bound derived in Part B, we establish the final optimality gap. $\square$

F PROOF OF THEOREM 3.5

PROOF. Part A: Proof of Constraint Satisfaction

From the dual update rule, for time slots when task k is active ($t \in \mathcal{T}_k$), the update is:

$$\lambda_k(t+1) = \max \left\{ \epsilon, \lambda_k(t) + D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right\}$$

For inactive slots ($t \notin \mathcal{T}_k$), $\lambda_k(t+1) = \lambda_k(t)$. Rearranging the active update to isolate the instantaneous constraint violation yields:

$$D_{\text{act},k}(t) - \frac{1}{R_k(t)} \leq \lambda_k(t+1) - \lambda_k(t)$$

Taking the expectation and summing over all active time slots up to T (the set $\mathcal{T}_k(T)$) produces a telescoping sum for each task k :

$$\sum_{t \in \mathcal{T}_k(T)} \mathbb{E} \left[D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right] \leq \mathbb{E}[\lambda_k(T+1)] - \mathbb{E}[\lambda_k(1)]$$

Dividing by the cardinality $|\mathcal{T}_k(T)|$ yields:

$$\frac{1}{|\mathcal{T}_k(T)|} \sum_{t \in \mathcal{T}_k(T)} \mathbb{E} \left[D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right] \leq \frac{\mathbb{E}[\lambda_k(T+1)] - \mathbb{E}[\lambda_k(1)]}{|\mathcal{T}_k(T)|}$$

As established in Part B, the expected dual variable $\mathbb{E}[\lambda_k(t)]$ is uniformly bounded by a constant. Assuming the task is active infinitely often, the number of active time slots approaches infinity ($|\mathcal{T}_k(T)| \rightarrow \infty$) as the global time horizon $T \rightarrow \infty$. Taking the limit with respect to the active time slots yields:

$$\lim_{|\mathcal{T}_k(T)| \rightarrow \infty} \frac{1}{|\mathcal{T}_k(T)|} \sum_{t \in \mathcal{T}_k(T)} \mathbb{E} \left[D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right] \leq 0$$

This proves the long-term expected delay constraint is strictly satisfied specifically for the time slots where the task is active..

Part B: Bounding the Multipliers

Define the aggregate Lyapunov function as $L(\lambda(t)) = \frac{1}{2} \sum_{k=1}^K \lambda_k^2(t)$. Applying the projection property $(\max\{\epsilon, x\})^2 \leq x^2 + \epsilon^2$ to the active tasks ($k \in \mathcal{K}(t)$), and noting that the drift is zero for inactive tasks, we bound the unified single-slot aggregate expected drift $\Delta L(t) \triangleq \mathbb{E}[L(\lambda(t+1))] - \mathbb{E}[L(\lambda(t))]$:

$$\Delta L(t) \leq \mathbb{E} \left[\sum_{k \in \mathcal{K}(t)} \left(\lambda_k(t) \left(D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right) + \frac{1}{2} \left(D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right)^2 + \frac{1}{2} \epsilon^2 \right) \right]$$

To introduce the utility-delay trade-off, we subtract the aggregate scaled expected utility $\frac{1}{\mu} \sum_{k \in \mathcal{K}(t)} w_k \mathbb{E}[A_k(\eta_k(t))]$ from both sides. We further add and subtract the estimator term $\lambda_k(t) \hat{D}_k(t)$ inside the expectation:

$$\begin{aligned} \Delta L(t) - \frac{1}{\mu} \sum_{k \in \mathcal{K}(t)} w_k \mathbb{E}[A_k(\eta_k(t))] &\leq \mathbb{E} \left[\sum_{k \in \mathcal{K}(t)} \left(-\frac{1}{\mu} w_k A_k(\eta_k(t)) + \lambda_k(t) \left(\hat{D}_k(t) - \frac{1}{R_k(t)} \right) \right) \right] \\ &\quad + \sum_{k \in \mathcal{K}(t)} \lambda_k(t) \Delta_k(t) + \sum_{k \in \mathcal{K}(t)} \left(\frac{1}{2} \left(D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right)^2 + \frac{1}{2} \epsilon^2 \right) \end{aligned}$$

Based on Assumption 5, the block coordinate descent algorithm maximizes the partial Lagrangian within an expected additive gap C_{gap} . We upper-bound the primary algorithmic terms by evaluating them against a strictly feasible, robust baseline configuration $(\eta^{\min}, \mathbf{s}^{\min})$. Let $D_k^{\min}(t) \triangleq D_k(\eta_k^{\min}, \mathbf{s}_k^{\min}, \mathbf{c}(t))$ and $\hat{D}_k^{\min}(t)$ be its estimate. Substituting this bound back into the drift inequality, and replacing $\hat{D}_k^{\min}(t)$ with $D_k^{\min}(t) - \Delta_k^{\min}(t)$, we isolate the error difference term:

$$\begin{aligned} \Delta L(t) - \frac{1}{\mu} \sum_{k \in \mathcal{K}(t)} w_k \mathbb{E}[A_k(\eta_k(t))] &\leq \mathbb{E} \left[\sum_{k \in \mathcal{K}(t)} \left(-\frac{1}{\mu} w_k A_k(\eta^{\min}) + \lambda_k(t) \left(D_k^{\min}(t) - \frac{1}{R_k(t)} \right) \right) \right] \\ &\quad + \sum_{k \in \mathcal{K}(t)} \lambda_k(t) \left(\Delta_k(t) - \Delta_k^{\min}(t) \right) \\ &\quad + \sum_{k \in \mathcal{K}(t)} \left(\frac{1}{2} \left(D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right)^2 + \frac{1}{2} \epsilon^2 \right) \Big] + \frac{C_{\text{gap}}}{\mu} \end{aligned}$$

The bottleneck mechanics dictate that the error difference is strictly bounded by the positive error margin: $\Delta_k(t) - \Delta_k^{\min}(t) \leq \max\{0, \Delta_k(t)\}$. By conditioning on the state variables and applying Assumption 3 ($\mathbb{E}[\max\{0, \Delta_k(t)\} \mid \lambda_k(t)] \leq \delta^+$) and Assumption 4 ($\mathbb{E}[D_k^{\min}(t) - \frac{1}{R_k(t)}] \leq -\gamma$), we can bound the terms:

$$\begin{aligned} \mathbb{E}\left[\lambda_k(t) \left(\Delta_k(t) - \Delta_k^{\min}(t)\right)\right] &\leq \mathbb{E}_{\lambda_k}\left[\lambda_k(t) \mathbb{E}[\max\{0, \Delta_k(t)\} \mid \lambda_k(t)]\right] \leq \delta^+ \mathbb{E}[\lambda_k(t)] \\ \mathbb{E}\left[\lambda_k(t) \left(D_k^{\min}(t) - \frac{1}{R_k(t)}\right)\right] &= \mathbb{E}_{\lambda_k}\left[\lambda_k(t) \mathbb{E}\left[D_k^{\min}(t) - \frac{1}{R_k(t)} \mid \lambda_k(t)\right]\right] \leq -\gamma \mathbb{E}[\lambda_k(t)] \end{aligned}$$

Applying these bounds along with Assumption 2 ($\mathbb{E}[(D_{\text{act},k}(t) - \frac{1}{R_k(t)})^2] \leq \xi^2$), and rearranging yields:

$$(\gamma - \delta^+) \sum_{k \in \mathcal{K}(t)} \mathbb{E}[\lambda_k(t)] \leq \sum_{k \in \mathcal{K}(t)} \frac{w_k}{\mu} \left(\mathbb{E}[A_k(\boldsymbol{\eta}_k(t))] - A_k(\boldsymbol{\eta}^{\min}) \right) + \sum_{k \in \mathcal{K}(t)} \frac{1}{2} (\xi^2 + \epsilon^2) + \frac{C_{\text{gap}}}{\mu} - \Delta L(t)$$

Summing over T slots, dropping the positive term $\mathbb{E}[L(\lambda(T+1))]$, and dividing by T . Note that $\frac{1}{T} \sum_{t=1}^T \sum_{k \in \mathcal{K}(t)} (\cdot) = \sum_{k=1}^K \frac{|\mathcal{T}_k(T)|}{T} (\cdot)$. Defining the asymptotic frequency of task k being active as $\rho_k = \lim_{T \rightarrow \infty} \frac{|\mathcal{T}_k(T)|}{T}$, we take the limit $T \rightarrow \infty$ to establish the aggregate multiplier bound:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{k \in \mathcal{K}(t)} \mathbb{E}[\lambda_k(t)] \leq \frac{\sum_{k=1}^K \rho_k w_k (A_{k,\max} - A_{k,\min}) + C_{\text{gap}}}{\mu(\gamma - \delta^+)} + \frac{\sum_{k=1}^K \rho_k (\xi^2 + \epsilon^2)}{2(\gamma - \delta^+)} \quad (18)$$

Part C: Bounding the Optimality Gap

We return to the fundamental drift inequality prior to evaluating the robust baseline. Instead, we evaluate it under the optimal stationary multi-task policy ($\boldsymbol{\eta}^*(t), \mathbf{s}^*(t)$), which satisfies $\mathbb{E}\left[D_k^*(t) - \frac{1}{R_k(t)}\right] \leq 0$, where $D_k^*(t) \triangleq D_k(\boldsymbol{\eta}_k^*(t), \mathbf{s}_k^*(t), \mathbf{c}(t))$. Incorporating the approximation gap C_{gap} , we obtain:

$$\begin{aligned} \Delta L(t) - \frac{1}{\mu} \sum_{k \in \mathcal{K}(t)} w_k \mathbb{E}[A_k(\boldsymbol{\eta}_k(t))] &\leq \mathbb{E}\left[\sum_{k \in \mathcal{K}(t)} \left(-\frac{1}{\mu} w_k A_k^*(t) + \lambda_k(t) \left(D_k^*(t) - \frac{1}{R_k(t)} \right) \right) \right. \\ &\quad + \sum_{k \in \mathcal{K}(t)} \lambda_k(t) (\Delta_k(t) - \Delta_k^*(t)) \\ &\quad \left. + \sum_{k \in \mathcal{K}(t)} \left(\frac{1}{2} \left(D_{\text{act},k}(t) - \frac{1}{R_k(t)} \right)^2 + \frac{1}{2} \epsilon^2 \right) \right] + \frac{C_{\text{gap}}}{\mu} \end{aligned}$$

Because the optimal stationary policy makes decisions independent of the queue state, the expectation of the queue cross-term cleanly factorizes by conditioning on the state variables:

$$\mathbb{E}\left[\lambda_k(t) \left(D_k^*(t) - \frac{1}{R_k(t)} \right)\right] = \mathbb{E}_{\lambda_k}\left[\lambda_k(t) \mathbb{E}\left[D_k^*(t) - \frac{1}{R_k(t)} \mid \lambda_k(t)\right]\right] \leq \mathbb{E}[\lambda_k(t)] \cdot 0 = 0$$

For the error difference term, we apply the raw expected error definitions from Assumption 3 ($\mathbb{E}[\Delta_k(t) \mid \lambda_k(t)] \leq \delta$ and $\mathbb{E}[\Delta_k^*(t)] = \delta^*$):

$$\begin{aligned} \mathbb{E}[\lambda_k(t) (\Delta_k(t) - \Delta_k^*(t))] &= \mathbb{E}[\lambda_k(t) \Delta_k(t)] - \mathbb{E}[\lambda_k(t) \Delta_k^*(t)] \\ &= \mathbb{E}_{\lambda_k}\left[\lambda_k(t) \mathbb{E}[\Delta_k(t) \mid \lambda_k(t)]\right] - \mathbb{E}_{\lambda_k}\left[\lambda_k(t) \mathbb{E}[\Delta_k^*(t)]\right] \\ &\leq \delta \mathbb{E}[\lambda_k(t)] - \delta^* \mathbb{E}[\lambda_k(t)] = (\delta - \delta^*) \mathbb{E}[\lambda_k(t)] \end{aligned}$$

Taking the full expectation of the drift inequality, applying these decoupled bounds:

$$\frac{1}{\mu} \sum_{k \in \mathcal{K}(t)} w_k \mathbb{E}[A_k(\boldsymbol{\eta}_k(t))] \geq \frac{1}{\mu} \sum_{k \in \mathcal{K}(t)} w_k A_k^* - \sum_{k \in \mathcal{K}(t)} (\delta - \delta^*) \mathbb{E}[\lambda_k(t)] - \sum_{k \in \mathcal{K}(t)} \frac{1}{2} (\xi^2 + \epsilon^2) - \frac{C_{\text{gap}}}{\mu} + \Delta L(t)$$

Averaging the above inequality over T slots, dropping the stabilizing Lyapunov term $\mathbb{E}[L(\lambda(1))]/T$, multiplying by μ , and taking the limit $T \rightarrow \infty$. We define $A^* = \sum_{k=1}^K \rho_k w_k A_k^*$ as the optimal time-averaged expected baseline utility:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{k \in \mathcal{K}(t)} w_k \mathbb{E}[A_k(\boldsymbol{\eta}_k(t))] \geq A^* - \mu(\delta - \delta^*) \left(\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{k \in \mathcal{K}(t)} \mathbb{E}[\lambda_k(t)] \right) - \frac{\mu}{2} \sum_{k=1}^K \rho_k (\xi^2 + \epsilon^2) - C_{\text{gap}}$$

□

G PRINCIPLED SELECTION OF THE TRADEOFF PARAMETER μ

The operational performance of the *Estimated Stochastic Dual Descent* algorithm is inherently dictated by the tradeoff parameter μ . This parameter serves as the pivotal weight that balances the maximization of the application-level utility, $A(\boldsymbol{\eta})$, against the strict enforcement of the long-term Quality of Service (QoS) delay constraints, represented by the virtual queue of dual variables $\boldsymbol{\lambda}$. Based on the Lyapunov stability results established in Theorem 3.5 and Theorem 3.6, the time-averaged expected delay violation is governed by the fundamental $[O(\mu), O(1/\mu)]$ tradeoff and is upper-bounded as follows:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[\lambda_t] \leq \frac{A_{\max} - A_{\min}}{\mu(\gamma - \delta^+)} + \Psi, \quad (19)$$

where Ψ captures the constant noise terms.

To guarantee system stability, our objective is to ensure that the expected accumulated delay violations remain bounded by a reasonable constant order of magnitude $\Theta(1)$, rather than growing infinitely or being unnecessarily suppressed to zero at the severe expense of inference accuracy. Achieving this target requires a principled derivation for μ . This derivation relies on two critical system parameters: γ , the expected robust safety margin between the target QoS cycle time $1/R$ and the baseline bottleneck delay evaluated at maximum compression $\boldsymbol{\eta}^{\min}$; and δ , the expected absolute impact of the channel estimation error on the realized bottleneck delay.

Because these parameters govern different mechanics of the system, they must be determined through distinct methodologies:

- **Analytical Calculation of γ :** Because the feasibility margin is defined strictly at a static, maximum compression configuration ($\boldsymbol{\eta}^{\min}$), it acts as an open-loop physical property of the hardware and network. If the base processing times, activation sizes, and the underlying channel capacity distributions are known, γ can be calculated directly. Furthermore, in multi-task scenarios where resource contention complicates exact calculation, a conservative lower bound for γ can be derived analytically by assuming a baseline equal resource sharing policy across all active tasks and subsequently evaluating the expected bottleneck delays at $\boldsymbol{\eta}^{\min}$.
- **Empirical Measurement of δ :** Unlike γ , the expected delay error δ depends on the dynamic compression factor $\boldsymbol{\eta}(t)$. Because $\boldsymbol{\eta}(t)$ is actively selected by the algorithm in a closed-loop manner based on the capacity estimator $\hat{c}(t)$, the error impact is deeply coupled to both the selection policy and the estimator's behavior. Therefore, δ must be measured empirically. This is achieved during a warmup phase or simulation by executing the policy, logging the true channel realizations $c(t)$ alongside the causal predictions $\hat{c}(t)$, and averaging the resulting delay discrepancies.

A critical component of our experimental setup is the deployment of a 90% Lower Confidence Bound (LCB) for causal channel estimation. By adopting a deliberately pessimistic view of the network state, the estimator systematically under-predicts the available channel capacity. Consequently, the estimated bottleneck delay safely overestimates the actual realized delay. In the context of our Lyapunov drift analysis, this pessimistic over-estimation drives the expected error impact δ to be strictly negative ($\delta^+ < 0$). Mathematically, this broadens the effective stability denominator from $(\gamma - \delta^+)$ to $(\gamma + |\delta^+|)$. This augmented, artificial buffer ensures that the virtual queues remain stable and strictly satisfy the QoS constraints, even when the system is subjected to high-variance network environments.

Leveraging this expanded stability margin, we define a heuristic upper bound for μ to ensure the delay violations remain firmly within the $\Theta(1)$ budget:

$$U = \frac{A_{\max} - A_{\min}}{\gamma}. \quad (20)$$

To systematically isolate the optimal operating point on the Pareto frontier—defined as the minimum μ that strictly satisfies the QoS delay constraints while maximizing accuracy—we employ an empirical binary search strategy over the interval $(0, U]$. To minimize profiling overhead, we conduct a multiple-step binary search initialized at the midpoint:

$$\mu = \frac{U}{2}. \quad (21)$$

Following each trial, the system evaluates the resulting time-averaged delay compliance. The search direction for the subsequent iteration is dictated by the system's stability:

- **Delay Satisfied (Over-penalized):** If the constraints are strictly met, the system can safely prioritize inference accuracy further. We update the upper bound of the search space and test a smaller penalty (e.g., transitioning from $U/2$ to $U/4$).
- **Delay Violated (Under-penalized):** If the delay constraints are breached, the dual variables lack sufficient weight to stabilize the virtual queues. We update the lower bound of the search space and test a larger penalty (e.g., transitioning from $U/2$ to $3U/4$).

Based on our experiments, executing this binary search for three iterations rapidly converges on a highly tuned μ . In Fig. 4, we observe the sensitivity of the result to the value of μ in an online Jetson experiment (J-V-ST) for three different values of μ . We observe that in the left figure, related to approximating the gradient using a fitting model approach, decreasing the tradeoff parameter μ from 1 to 0.01 results in an increase in aggregate utility (from approximately 0.81 to 0.90). However, this improvement comes at the direct cost of QoS compliance. As μ decreases, the system prioritizes utility over the virtual queues, causing the excess delay to cross the feasibility threshold (the zero vertical line) from a safe negative value into the positive domain, indicating delay violations. For the fitting model, only $\mu = 1$ successfully maintains a strictly negative excess delay. In contrast, the right panel illustrates the performance when employing the Stein's estimator. While the overarching μ -driven tradeoff curve remains structurally similar, the specific operating points shift. Notably, $\mu = 0.1$ emerges as a highly

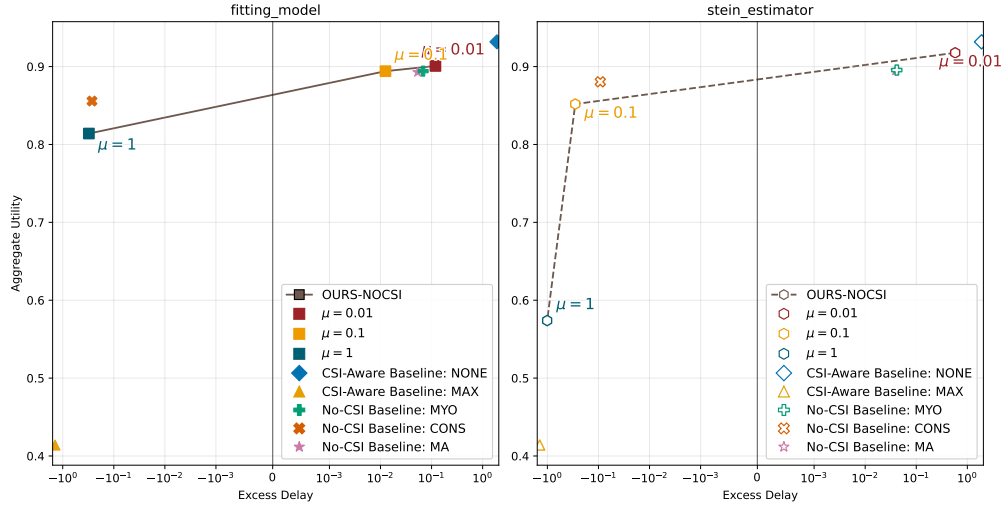


Figure 4: Testbed Top-K single-task results under μ sweep. The left panel uses the fitting-based estimator, and the right panel uses Stein’s estimator.

optimal configuration for the Stein’s estimator; it achieves a strong utility of approximately 0.85 while still maintaining a strictly negative excess delay (around -10^{-1}). Further decreasing the parameter to $\mu = 0.01$ yields marginal utility gains (up to roughly 0.92) but severely breaches the delay constraints, resulting in a large positive excess delay.

Ultimately, both panels empirically validate the $[O(\mu), O(1/\mu)]$ bounds derived in our Lyapunov analysis. Sweeping μ traces out a distinct Pareto frontier for the OURS-NOCSI approach, clearly bridging the gap between conservative, low-utility baseline executions and aggressive, delay-violating performance. This visualizes and underscores the necessity of our empirical binary search methodology, as the exact optimal μ required to maximize utility while safely adhering to the zero-excess-delay constraint heavily depends on the specific gradient estimation strategy deployed.

H ESTIMATING THE ACCURACY-COMPRESSION FUNCTION

H.1 Stein Gradient Oracle

We describe the zeroth-order gradient oracle used in our method, which estimates $\nabla f(\mathbf{x})$ using only scalar evaluations of f . The construction is grounded in Stein’s lemma.

H.1.1 Stein’s Lemma. We repeat here Stein’s Lemma, which we can use to produce a gradient estimator for the accuracy function via appropriate sampling:

Lemma H.1 (Stein, 1981 [32]). *Let $\mathbf{z} \sim \mathcal{N}(0, I_d)$ and let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be almost everywhere differentiable with $\mathbb{E}[\|\nabla f(\mathbf{x} + \sigma\mathbf{z})\|] < \infty$ and $\mathbb{E}[f(\mathbf{x} + \sigma\mathbf{z})^2] < \infty$. Then for any $\mathbf{x} \in \mathbb{R}^d$ and $\sigma > 0$:*

$$\mathbb{E}_{\mathbf{z}} [\mathbf{z} f(\mathbf{x} + \sigma\mathbf{z})] = \sigma \mathbb{E}_{\mathbf{z}} [\nabla f(\mathbf{x} + \sigma\mathbf{z})]. \quad (22)$$

The integrability conditions ensure that the boundary term arising in the integration by parts proof vanishes, i.e. $\lim_{\|\mathbf{z}\| \rightarrow \infty} f(\mathbf{x} + \sigma\mathbf{z}) \phi(\mathbf{z}) = 0$, where ϕ is the standard Gaussian density. These conditions are satisfied whenever f grows slower than exponentially, which covers all cases of practical interest.

H.1.2 Gradient Estimator of $A(\eta)$ using Stein’s Lemma. Equation (22) implies that the gradient of the Gaussian-smoothed function,

$$f_{\sigma}(\mathbf{x}) = \mathbb{E}_{\mathbf{z} \sim \mathcal{N}(0, I_d)} [f(\mathbf{x} + \sigma\mathbf{z})], \quad (23)$$

can be written purely in terms of function evaluations:

$$\nabla f_{\sigma}(\mathbf{x}) = \frac{1}{\sigma} \mathbb{E}_{\mathbf{z}} [\mathbf{z} f(\mathbf{x} + \sigma\mathbf{z})]. \quad (24)$$

As $\sigma \rightarrow 0$, $f_{\sigma} \rightarrow f$ pointwise, so for small σ this approximates $\nabla f(\mathbf{x})$. In practice we estimate the expectation via N antithetic sample pairs $(\mathbf{z}_i, -\mathbf{z}_i)$, which cancels the zeroth-order term $f(\mathbf{x})$ and reduces variance:

$$\hat{\nabla} f(\mathbf{x}) = \frac{1}{2N\sigma} \sum_{i=1}^N \mathbf{z}_i [f(\mathbf{x} + \sigma\mathbf{z}_i) - f(\mathbf{x} - \sigma\mathbf{z}_i)], \quad \mathbf{z}_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, I_d). \quad (25)$$

Algorithm 3 Stein Gradient Oracle**Require:** Point $\mathbf{x} \in \mathbb{R}^d$, smoothing parameter $\sigma > 0$, number of samples N **Ensure:** Gradient estimate $\hat{\nabla}f(\mathbf{x}) \in \mathbb{R}^d$

```

1:  $\mathbf{g} \leftarrow \mathbf{0}_d$ 
2: for  $i = 1$  to  $N$  do
3:   Sample  $\mathbf{z}_i \sim \mathcal{N}(\mathbf{0}, I_d)$ 
4:    $\mathbf{g} \leftarrow \mathbf{g} + \mathbf{z}_i [f(\mathbf{x} + \sigma \mathbf{z}_i) - f(\mathbf{x} - \sigma \mathbf{z}_i)]$ 
5: end for
6: return  $\hat{\nabla}f(\mathbf{x}) = \mathbf{g} / (2N\sigma)$ 

```

The estimator requires $2N$ function evaluations regardless of the input dimension d , and all queries are independent so they can be executed in parallel. The smoothing parameter σ controls a bias-variance trade-off: larger σ reduces variance but increases the bias from smoothing, while smaller σ reduces bias at the cost of higher variance.

H.2 Accuracy Function Analysis: Concavity and $\eta_{\min}$ Detection

This appendix describes the methodology used to (i) characterise the shape of the accuracy function $A(\boldsymbol{\eta})$ across the compression-ratio space, (ii) automatically detect the per-link threshold $\eta_{\min}^\ell$ below which the activation representations collapse to noise and the predictor should not be fitted, and (iii) produce three-dimensional surface plots of $A(\boldsymbol{\eta})$. Let $\boldsymbol{\eta} = (\eta_0, \eta_1, \dots, \eta_{L-1}) \in [0, 1]^L$ denote the per-link compression ratios, where $\eta_\ell = 1$ corresponds to no compression on link ℓ and $\eta_\ell = 0$ corresponds to maximum compression. The true accuracy function $A_{\text{true}}(\boldsymbol{\eta})$ is defined as the task accuracy (e.g. top-1 classification accuracy for ResNet, or MMLU accuracy for the LLM backend) when each link ℓ operates at ratio η_ℓ . Empirically, A_{true} exhibits two qualitatively distinct regimes (see Figure 8):

- (1) **Collapse regime** ($\eta_\ell < \eta_{\min}^\ell$): the compression is so severe that intermediate activations carry no useful information. A is essentially flat and close to chance level; small increases in η_ℓ produce no consistent improvement.
- (2) **Informative regime** ($\eta_\ell \geq \eta_{\min}^\ell$): A is monotonically non-decreasing and *concave* in η_ℓ . Our surrogate predictor is only well-specified in this regime.

Operating below $\eta_{\min}^\ell$ wastes compute (all configurations are equally bad) and can mislead the optimiser. We therefore identify $\eta_{\min}^\ell$ automatically before training the predictor and enforce it as a hard lower bound on the search domain.

H.2.1 $\eta_{\min}$ Detection. We run three complementary analyses, each implemented as 1-D sweeps along individual link axes or as multi-dimensional directional profiles. In each case, we used a knn regressor to smoothly approximate the true accuracy function. The knn regressor was trained on N^2 samples of the accuracy function, each averaged over R evaluations to reduce noise.

Test 1: Per-axis Monotonicity Scan. For each link ℓ , we construct a 1-D slice of the accuracy function by sweeping η_ℓ over a uniform grid of N points in $[0, 1]$ while holding all other links fixed at a reference value $\eta_{\text{mid}} = 0.5$. Let $p_j = A(\eta_j^{(\ell)})$ denote the accuracy at grid point j . We apply a light three-point moving average $\tilde{p}_j = \frac{1}{3}(p_{j-1} + p_j + p_{j+1})$ and compute finite-difference slopes

$$s_j = \frac{\tilde{p}_{j+1} - \tilde{p}_j}{\eta_{j+1}^{(\ell)} - \eta_j^{(\ell)}}. \quad (26)$$

The per-link monotonicity threshold is defined as the smallest grid value above which the slope remains consistently positive:

$$\hat{\eta}_{\min, \text{mono}}^\ell = \eta_{j^*-1}^{(\ell)}, \quad j^* = \min\{j : s_j \geq \tau_{\text{slope}}\}, \quad (27)$$

where $\tau_{\text{slope}} > 0$ is a small noise tolerance (default 5×10^{-3}).

Test 2: Concavity Scan via Second Differences. Using the same 1-D profiles p_j , we compute second (centred) differences

$$\Delta_j^2 = p_{j-1} + p_{j+1} - 2p_j, \quad j = 1, \dots, N-2. \quad (28)$$

Strict concavity of A requires $\Delta_j^2 \leq 0$ for all j . A significantly positive second difference indicates a convex neighbourhood in the accuracy curve. We scan downward from $\eta = 1$ and identify the last (highest) index at which concavity is violated:

$$\hat{\eta}_{\min, \text{conc}}^\ell = \eta_{j^{**}}^{(\ell)}, \quad j^{**} = \max\{j : \Delta_j^2 > \tau_{\text{conc}}\}, \quad (29)$$

where $\tau_{\text{conc}} > 0$ is a small positive tolerance to allow for measurement noise (default 5×10^{-3}). If no violation is detected, $\hat{\eta}_{\min, \text{conc}}^\ell = 0$. Plots of Δ_j^2 are shown in Figure 6.

Table 6: Default hyperparameters for concavity analysis and 3-D surface plots.

Parameter	Description	ResNet	LLM
N	Grid points per axis (1-D sweeps)	20	12
R	Repeated evaluations per point	5	1
D	Random rays (directional test)	10	5
τ_{slope}	Monotonicity noise tolerance	5×10^{-3}	
τ_{conc}	Concavity noise tolerance	5×10^{-3}	
δ	Safety margin	0.05	
N^2	Grid points (3-D surface)	400	400
R	Repeats per point (3-D)	3	3
k	KNN neighbours	8	
σ	Stein bandwidth	0.05	
M	Stein MC samples per point	10	

Test 3: Directional Profiles. We repeat the above process across different directions. We sample D random rays in η -space. Each ray is parameterised as

$$\eta(s) = \eta_{\text{start}}^{(d)} + s(1 - \eta_{\text{start}}^{(d)}), \quad s \in [0, 1], \quad (30)$$

so that $\eta(0) = \eta_{\text{start}}^{(d)}$ and $\eta(1) = 1$. The first ray always starts from $\mathbf{0}$; the remaining $D - 1$ start from points sampled uniformly in $[0, 0.5]^L$. Accuracy is evaluated along each ray at N equispaced values of s , and the profiles are plotted together in Figure 7. A vertical marker is drawn on each profile at the ray parameter s^* corresponding to the point where the ray enters the informative regime.

The final per-link threshold combines the two quantitative tests with an additive safety margin $\delta \geq 0$ (default 0.05):

$$\hat{\eta}_{\text{min,final}}^\ell = \min\left(1, \max(\hat{\eta}_{\text{min,mono}}^\ell, \hat{\eta}_{\text{min,conc}}^\ell) + \delta\right). \quad (31)$$

The global threshold used as the lower bound for all subsequent experiments is

$$\eta_{\text{min}} = \max_\ell \hat{\eta}_{\text{min,final}}^\ell. \quad (32)$$

Axis-sweep profiles with all three threshold lines overlaid are shown in Figure 5.

H.3 Three-Dimensional Accuracy Surface Visualisation

To justify the use of convex optimization algorithms, we provide a picture of $A(\eta)$ that demonstrates its concave region. To do that, we use the ResNet-56 model with three cutpoints, fix link 0 at $\eta_0 = 1$ (uncompressed) and produce 3-D surface plots over the (η_1, η_2) plane on an $N \times N$ grid (total N^2 evaluations, R repeats each). Two surfaces are rendered: a) the KNN-smoothed surface, where a k -nearest-neighbour regressor with inverse-distance weighting is fitted to the N^2 observations, yielding a smooth surface that removes per-point noise while preserving the global shape (Figure 8). b) the Stein-smoothed surface, where The Stein (Gaussian-smoothing) view of A ,

$$\tilde{A}(\eta) = \mathbb{E}_{z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} [A(\eta + \sigma z)], \quad (33)$$

estimated by Monte Carlo with M perturbation samples per grid point and bandwidth σ matching the value used in the gradient oracle (Figure 9). This surface is what the optimiser effectively maximises during training. Table 6 summarises the default hyperparameters used in our experiments.

H.4 Regression Fitting

As an alternative to the Stein-based approach, we also estimate the accuracy-compression function $A_k(\eta_k)$ and its gradients using a fitting-based method. The basic idea is to conduct offline experiments on a partitioned model, establish the relationship between η_k and the resulting inference accuracy, and then fit a surrogate model to this relationship.

Specifically, for each task k and scenario, we first build an offline partitioned model with a scenario-specific set of fixed cut points. For every cut point i , we predefine a candidate set of compression factors within $[\eta_{i,k}^{\text{min}}, 1]$ that covers the feasible range as broadly as possible. We then enumerate all combinations of compression factors across the cut points. For each configuration of η_k , we run offline inference on the hold-out dataset using the corresponding partitioned model and record the resulting accuracy $A_k(\eta_k)$. In this way, we obtain N_f sampled compression-accuracy pairs,

$$\left(\eta_k^{(n)}, A_k^{(n)}\right), \quad n = 1, 2, \dots, N_f,$$

where $\eta_k^{(n)}$ denotes the compression vector of the n -th sample for task k , and $A_k^{(n)}$ is the corresponding measured accuracy.

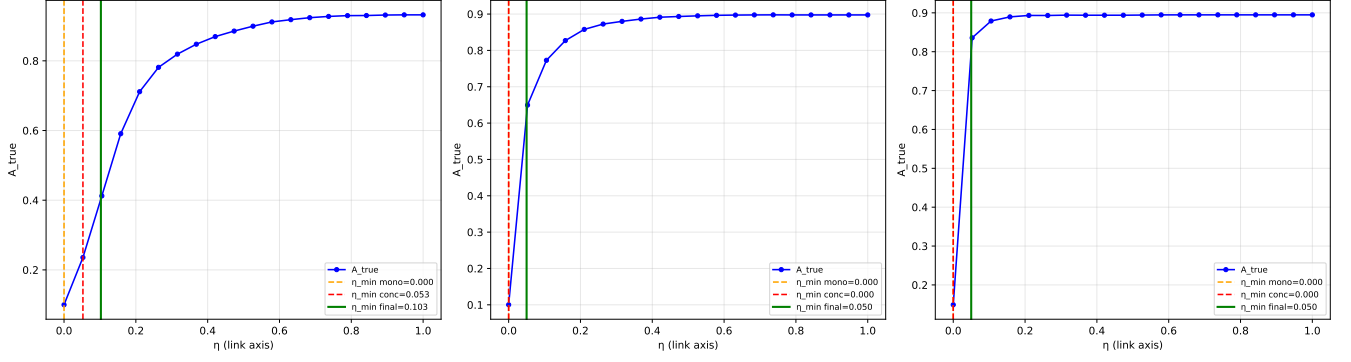


Figure 5: Per-axis accuracy profiles with $\eta_{\min}$ thresholds. Each panel shows the 1-D accuracy slice along one link axis (all other links held at $\eta_{\text{mid}} = 0.5$). Orange dashed: monotonicity threshold $\hat{\eta}_{\min, \text{mono}}^{\ell}$; red dashed: concavity threshold $\hat{\eta}_{\min, \text{conc}}^{\ell}$; green solid: final recommended threshold $\hat{\eta}_{\min, \text{final}}^{\ell}$ (equation (32)).

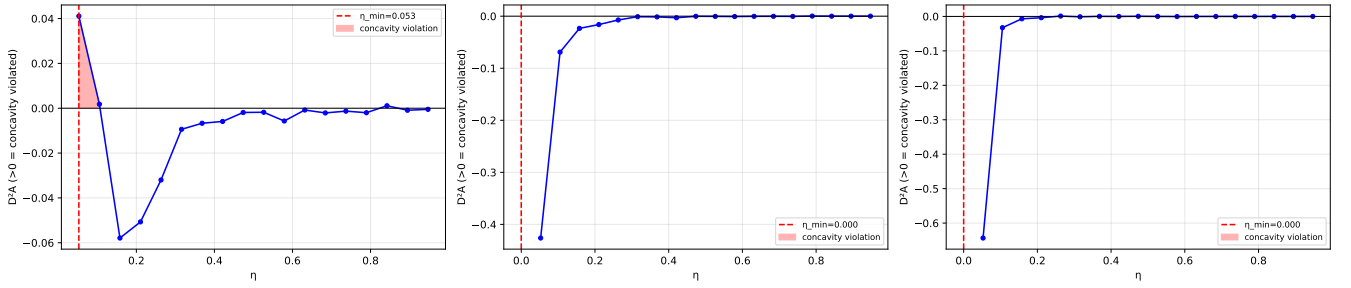


Figure 6: Second differences $\Delta^2 A$ along each link axis. Positive values (shaded red) indicate violations of concavity. The red dashed vertical line marks $\hat{\eta}_{\min, \text{conc}}^{\ell}$, i.e. the largest η at which a significant violation occurs.

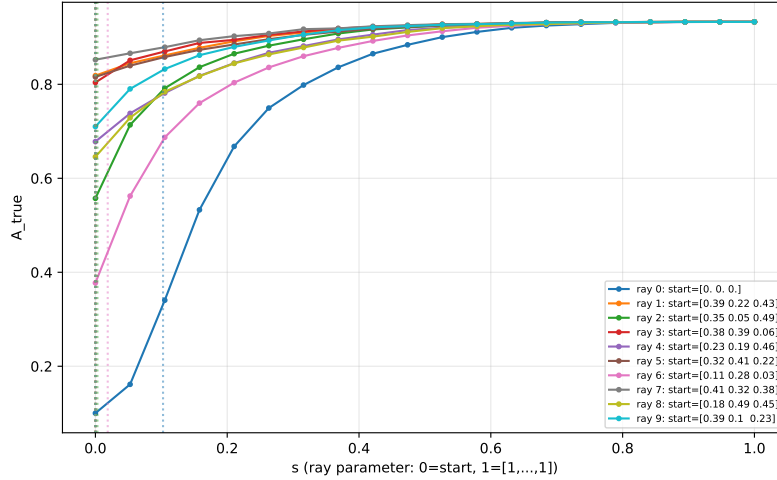


Figure 7: Directional accuracy profiles along random rays. Each coloured curve traces $A(\eta(s))$ as s increases from a random starting point to 1. The dotted vertical line marks the ray parameter s^* at which the ray enters the informative regime (all links above their respective $\hat{\eta}_{\min, \text{final}}^{\ell}$). The consistent increase in slope at s^* confirms that the collapse is correlated across links.

We then fit a surrogate model $\hat{A}_k(\eta_k)$ based on the sampled compression-accuracy pairs $(\eta_k^{(n)}, A_k^{(n)})$. To train $\hat{A}_k(\eta_k)$, we use *scikit-learn*, a widely used Python machine learning library that provides efficient tools for regression fitting. Since the shape of the accuracy-compression

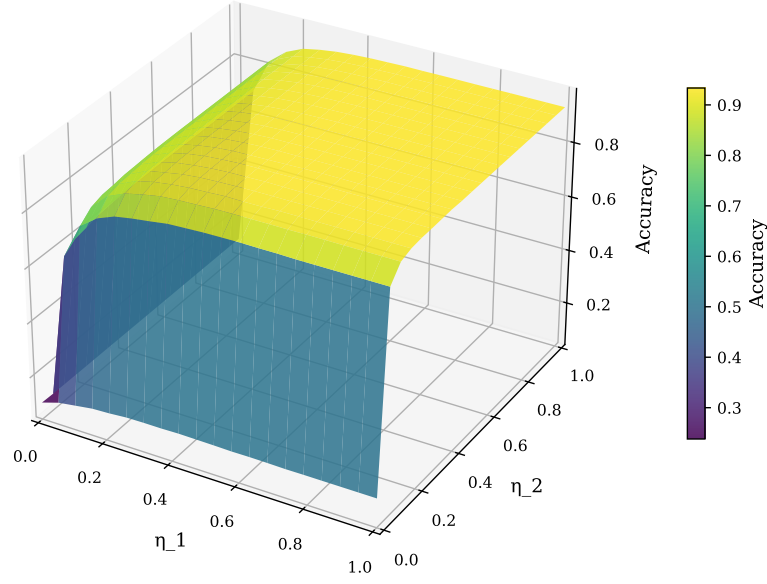


Figure 8: KNN-smoothed accuracy surface ($k = 8$, inverse-distance weighting). Fitted on N^2 observations; the smooth surface suppresses per-point noise while preserving the monotone concave structure of the informative regime.

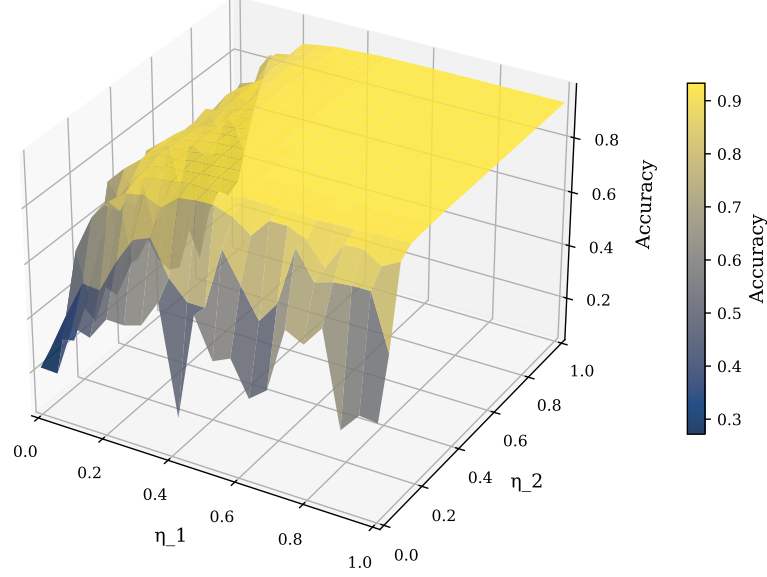


Figure 9: Stein-smoothed accuracy surface $\tilde{A}(\eta)$ (equation (33)), estimated by Monte Carlo with $M = 10$ perturbation samples per point and bandwidth $\sigma = 0.05$. This is the surface effectively maximised by the gradient oracle during optimisation.

function may vary across tasks and scenarios, we evaluate several surrogate families, including *linear_monotonic*, *poly2*, *poly3*, *gbm*, *rf*, *mlp*, and *mlp_small*.

Linear monotonic (*linear_monotonic*) This surrogate uses a linear regressor with nonnegative coefficients, enforcing a monotonic relationship between the compression features and the predicted accuracy.

Table 7: Test accuracy and surrogate inference speed for different fitting models. Inference speed is reported in milliseconds per sample.

Task	Codec	<i>linear_mono</i>			<i>poly2</i>			<i>poly3</i>			<i>rf</i>			<i>gbm</i>			<i>mlp_small</i>			<i>mlp</i>		
		RMSE	R^2	Inf. (ms)	RMSE	R^2	Inf. (ms)	RMSE	R^2	Inf. (ms)	RMSE	R^2	Inf. (ms)	RMSE	R^2	Inf. (ms)	RMSE	R^2	Inf. (ms)	RMSE	R^2	Inf. (ms)
RN56-CF10	<i>topk</i>	0.0940	0.6937	0.1752	0.0441	0.9327	0.2709	0.0226	0.9822	0.2794	0.0009	1.0000	13.36	0.0019	0.9999	0.4332	0.1661	0.0422	0.2276	0.0269	0.9749	0.2038
	<i>quantization</i>	0.3487	0.1638	0.1752	0.2999	0.3816	0.2892	0.2331	0.6265	0.2868	0.0016	1.0000	13.64	0.0018	1.0000	0.4536	0.4368	-0.3122	0.2029	0.4289	-0.2649	0.2243
	<i>llmint8_fp16_int2</i>	0.0292	0.5688	0.1757	0.0100	0.9501	0.2707	0.0053	0.9859	0.2852	0.0061	0.9811	13.28	0.0101	0.9487	0.4338	0.0239	0.7124	0.1981	0.1110	-5.2093	0.2075
	<i>llmint8_fp16_int4</i>	0.0006	0.3058	0.1755	0.0007	0.2523	0.2793	0.0006	0.2668	0.2815	0.0007	0.2017	13.33	0.0008	-0.2244	0.4522	0.2463	-105348	0.2032	0.1451	-36545	0.2081
	<i>llmint8_int8_int4</i>	0.0007	0.0495	0.1746	0.0007	0.1081	0.2716	0.0006	0.3141	0.2795	0.0006	0.2561	13.30	0.0007	-0.0463	0.4319	0.0217	-895.3	0.1999	0.0912	-15853	0.2087
	<i>llmint8_int8_int2</i>	0.0276	0.6158	0.1759	0.0092	0.9569	0.2705	0.0041	0.9915	0.2818	0.0025	0.9968	13.11	0.0005	0.9999	0.4468	0.1159	-5.7756	0.1984	0.1045	-4.5031	0.2062
FT5-SST2	<i>topk</i>	0.0133	0.4869	0.1731	0.0098	0.7201	0.2693	0.0094	0.7401	0.2766	0.0067	0.8695	13.15	0.0070	0.8565	0.4196	0.0923	-23.8149	0.1971	0.0827	-18.9063	0.2040
	<i>quantization</i>	0.0791	0.3047	0.1794	0.0522	0.6971	0.2764	0.0360	0.8560	0.2800	0.0303	0.8978	13.17	0.0359	0.8570	0.4296	0.1110	-0.3695	0.1967	0.1193	-0.5817	0.2044
	<i>llmint8_fp16_int8</i>	0.0000	1.0000	0.1763	0.0000	1.0000	0.2709	0.0000	1.0000	0.2901	0.0000	-1520	13.21	0.0000	1.0000	0.4052	0.9300	-inf	0.2014	0.7418	-inf	0.2068

Polynomial regression (*poly2*, *poly3*) These surrogates use polynomial feature expansion followed by ridge regression to capture smooth nonlinear accuracy-compression relationships. The polynomial degree is set to 2 for *poly2* and 3 for *poly3*, with ridge regularization strength fixed at 0.1. Because the fitted functions are smooth, their gradients can be computed directly.

Multilayer perceptron (*mlp_small*, *mlp*) These surrogates use feedforward neural networks to model more flexible nonlinear mappings. The *mlp_small* model uses two hidden layers of sizes 64 and 32 with up to 2000 training iterations, whereas *mlp* uses three hidden layers of sizes 128, 64, and 32 with up to 3000 iterations. Both models use early stopping to reduce overfitting.

Random forest (*rf*) This method uses a random forest regressor, which aggregates the predictions of multiple decision trees to improve robustness and capture nonlinear effects. In our implementation, the model uses 200 trees with a maximum depth of 15.

Gradient boosting (*gbm*) This method uses a gradient boosting regressor, which builds an ensemble of trees sequentially so that each stage reduces the residual error of the previous one. Its main hyperparameters are 300 boosting stages, a maximum depth of 8, and a learning rate of 0.05.

We evaluate each fitting method using RMSE, R^2 , and the inference latency of a single prediction call, since online optimization is sensitive to prediction overhead. Based on preliminary experiments that jointly consider fitting quality and prediction speed, we select the final surrogate for each task and scenario. Taking RN56-CF10 and FT5-SST2 as representative examples, and following the model setup and partition strategy used in the scenarios **J-V-S** and **J-L-S** on the Jetson platform, we evaluate the fitting quality and inference overhead of different surrogate models. The corresponding results for the two tasks are reported in Table 7. The results show that the fitting quality differs significantly across surrogate families, suggesting that the fitting method should be selected task-dependently. Meanwhile, the inference latency of all fitted models remains far below the corresponding scenario-specific τ , demonstrating the efficiency advantage of fitting-based accuracy estimation in terms of runtime overhead.

We also evaluate the structural properties of all fitted models through one-dimensional slices, which allow us to examine characteristics such as monotonicity and convexity with respect to $\eta_{i,k}$. The Figure 10 provides an illustrative surrogate model $\hat{A}_k(\eta_k)$ example for the scenario **J-V-S**.

We further evaluate the fitted models through the behavior of the proposed algorithm in both offline and online experiments. If Algorithms 1 and 2 consistently satisfy the delay constraints and achieve accuracy advances relative to the baselines in the test scenarios, this indicates that the fitting-based accuracy estimator provides high-quality estimates. Referring to the results under scenario V-ST in Figures 12 and 13, we observe that the fitting-based method achieves better performance than Stein’s method in this scenario.

I ADDITIONAL EXPERIMENTAL DETAILS

I.1 Tasks and Datasets

For vision, we consider a 2-layer MLP on MNIST and ResNet-56 on CIFAR-10. For language, we consider Gemma-2 2B and 7B on ShareGPT, Llama-3.1 8B on MMLU and WikiText, and Flan-T5-Base on SST-2. MMLU is evaluated with a fixed multiple-choice log-likelihood protocol. For the generative tasks (ShareGPT and WikiText), we define a reference-normalized next-token prediction score as the ratio of the compressed model’s per-token log-likelihood to that of the uncompressed model, so that a score of 1 indicates lossless quality relative to the full-precision baseline. On ShareGPT, this score is computed over assistant response tokens conditioned on the conversation context; on WikiText, it is computed over fixed-length text windows. SST-2 is a binary sentiment classification subset of the Stanford Sentiment Treebank dataset. It contains sentences labeled as either positive or negative. Therefore, for Flan-T5-Base on SST-2, we use classification accuracy as the metric for evaluating inference quality.

I.2 Compression Rate Selection Algorithms

In this section, we provide the detailed formulations and underlying mechanics for the established baseline policies and our proposed optimization frameworks.

I.2.1 Single-Task. For isolated single-task environments, the optimization objective is restricted to finding the optimal data compression ratios along the inference pipeline. We evaluate the following policies:

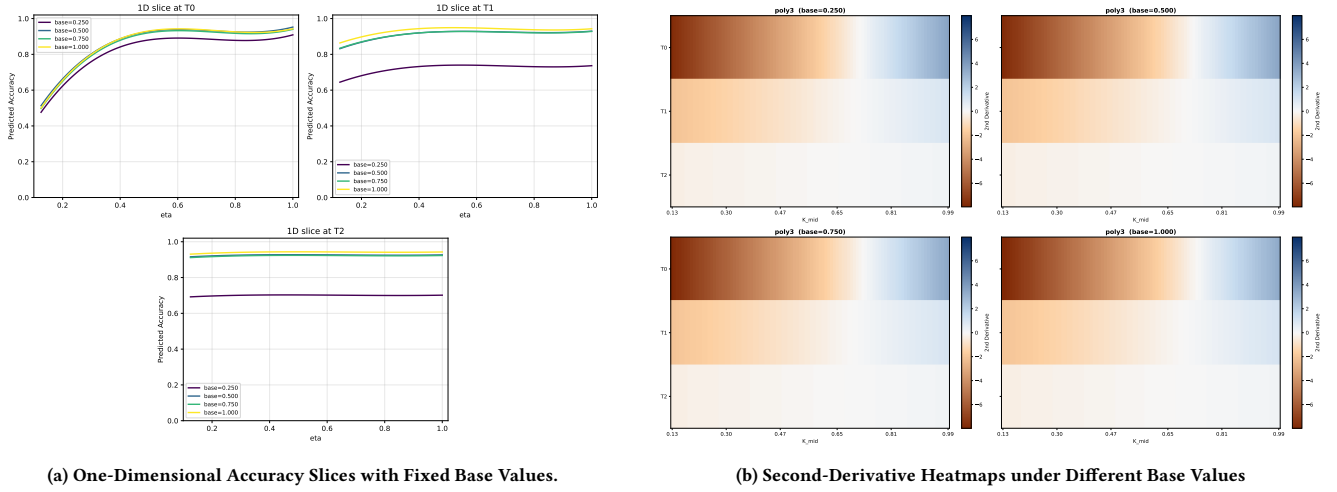


Figure 10: Senerino J-V-S Poly3 Surrogate Visualization. The base value denotes the fixed value assigned to the non-scanned transfer ratios while one transfer ratio is varied to generate a one-dimensional slice.

Max-compression (MAX) Operating as a strict lower bound for network delay, this baseline forcefully applies the absolute minimum admissible compression factor $\eta_i = \eta_i^{\min}$ across all communication links. While it aggressively minimizes transmission latency, it establishes the empirical lower bound on downstream inference accuracy.

No-compression (NONE) Prioritizing maximum inference accuracy without regard for network congestion, this policy transmits the intermediate layer representations entirely uncompressed ($\eta_i = 1$) over every link. It serves as an upper bound for accuracy at the severe risk of violating throughput constraints in bandwidth-limited environments.

Uniform-compression (UNI) This CSI-aware baseline enforces a strictly homogeneous compression rate η across all transmission hops along the pipeline. Because the rate must satisfy the end-to-end constraint uniformly, it is inherently bottlenecked by the most restrictive instantaneous link capacity, taking the form:

$$\eta = \max \left(\max_i \eta_i^{\min}, \min \left(1, \min_i \frac{c_i(t)}{R(t) a_i} \right) \right). \quad (34)$$

Myopic-capacity (MYO) This No-CSI policy employs the exact CSI-aware optimal formulation but operates under the naive assumption that the network is entirely static. Specifically, it relies on the most recent channel observation as a deterministic proxy for current capacity, mapping $\hat{c}_i(t) = c_i(t - 1)$ at every time slot.

Conservative-capacity (CONS) Sharing the optimal formulation of the myopic policy, this No-CSI baseline adopts a highly pessimistic approach to channel estimation. To safeguard against transient network collapses, it substitutes $\hat{c}_i(t)$ with the running minimum of historically observed capacities: $\hat{c}_i(t) = \min_{\tau < t} c_i(\tau)$.

Moving-average (MA) To mitigate the impact of high-frequency channel noise, this No-CSI baseline utilizes the myopic optimal policy but replaces the true channel capacity with a moving average of past observations spanning a finite temporal window W , such that $\hat{c}_i(t) = \frac{1}{W} \sum_{w=1}^W c_i(t - w)$.

CSI-Aware Optimal (OURS-CSI) Our proposed framework computes the exact closed-form solution to the single-task program. It assumes oracle access to perfect channel knowledge ($c_i(t)$), yielding the theoretical Pareto frontier between delay and accuracy.

Alg. 1 with LCB (OURS-NOCSI) Our proposed solver relies entirely on causal channel estimates. To robustly handle channel uncertainty, it employs a Lower Confidence Bound (LCB) estimator for the network capacity, formulated as:

$$\hat{c}(t) = \hat{\mu}_c(t) - \beta \hat{\sigma}_c(t) \quad (35)$$

where $\hat{\mu}_c(t)$ and $\hat{\sigma}_c(t)$ represent the empirically predicted mean and standard deviation of the channel state at time t , and β is a tunable parameter controlling the conservatism of the estimate. By deliberately under-predicting the available capacity, this pessimistic estimator creates a safety buffer. The solver leverages this within our Estimated Stochastic Dual Descent formulation to iteratively optimize compression rates, strictly satisfying long-term QoS constraints even under unpredictable network volatility.

1.2.2 Multi-Task. In multi-task environments, resource contention introduces a coupled optimization challenge necessitating policies that jointly govern both physical resource allocation partitions ($s^{\text{comp}}, s^{\text{comm}}$) and task-specific data compression rates ($\eta_{i,k}$):

- Max-compression (MAX)** This baseline strictly enforces uniform partitioning of compute and bandwidth resources among all active tasks on shared nodes and links ($s_{i,k}^{\text{comp}} = s_{i,k}^{\text{comm}} = 1/|\mathcal{K}|$). Concurrently, it enforces maximum compression $\eta_{i,k} = \eta_{i,k}^{\min}$ universally to aggressively minimize queueing and transmission delays.
- No-compression (NONE)** Maintaining the uniform resource partitioning scheme ($s_{i,k}^{\text{comp}} = s_{i,k}^{\text{comm}} = 1/|\mathcal{K}|$), this baseline transmits all intermediate data completely uncompressed ($\eta_{i,k} = 1$), establishing an upper bound on multi-task accuracy while largely ignoring shared link congestion.
- Equal-share (EQ)** This policy enforces a fair-share uniform resource partition across all tasks. Given these static, isolated resource slices, the compression factors $\eta_{i,k}$ are independently optimized using the single-task CSI-aware optimal mapping.
- Proportional-share (PROP)** Rather than splitting resources equally, this CSI-aware baseline allocates fractions proportional to the inherent architectural demands of each task. Specifically, compute shares are allocated relative to base processing loads ($s_{i,k}^{\text{comp}} \propto \tau_{i,k}$), and bandwidth shares are allocated relative to uncompressed activation dimensions ($s_{i,k}^{\text{comm}} \propto a_{i,k}$). Task compression rates subsequently follow the single-task optimal formulation over these weighted partitions.
- Strict-priority (PRIO)** This baseline implements a greedy resource scheduling algorithm governed strictly by the application-level utility weights w_k . The scheduler first secures the minimum-feasible resource allocations required to keep all active tasks viable. Subsequently, it funnels all residual bandwidth and compute capacity entirely to the tasks with the highest utility priorities to maximize their respective accuracy (η_k).
- Decoupled-equal-split (D-EQUAL)** Operating without explicit channel state information, this policy partitions resources equally at every time slot, effectively decoupling the optimization landscape. Each task k independently executes its local stochastic compression subproblem (Alg. 1) using its dedicated effective channel capacity $\hat{c}_{i,k}^{\text{eff}}(t) = s_{i,k}^{\text{comm}} \hat{c}_i(t)$ and effective processing time $\tau_{i,k}^{\text{eff}} = \tau_{i,k} / s_{i,k}^{\text{comp}}$.
- Decoupled- λ -proportional (D-L-PROP)** Operating under the same decoupled framework as Decoupled-equal-split (D-EQUAL), this No-CSI baseline dynamically modulates the resource allocations proportionally to the current values of the dual variables, $s_{i,k} \propto \lambda_k(t)$. This acts as a feedback-driven triage mechanism, explicitly prioritizing resources for tasks that are currently struggling to meet their required throughput thresholds.
- Moving-average (MA)** This predictive baseline constructs a historical temporal mean of the channel capacities. It subsequently feeds these smoothed deterministic estimates into the exact CSI-aware multi-task problem (Problem 2), jointly optimizing the resource splits and compression rates under the assumption that the average reflects the true environment.
- CSI-Aware Optimal (OURS-CSI)** Assuming perfect instantaneous knowledge of the shared channel capacities, this method solves the convex optimization program (Problem 2), performing optimal multi-dimensional water-filling across all tasks and network links.
- Alg. 2 with LCB (OURS-NOCSI)** Our comprehensive multi-task framework leverages block coordinate descent to alternately optimize resource allocation variables and intermediate compression variables on the fly. It uses an LCB channel estimator and dynamically balances fair resource contention with individual task QoS requirements.

I.3 Activation Compression Methods

Throughout the experiments, the compression factor $\eta \in [0, 1]$ is the *retained fraction* relative to the uncompressed activation tensor in its reference floating-point dtype (for language models, activations are taken at selected decoder layers; for vision models, at hooked layers in the convolutional pipeline).

Top- k sparsification keeps a fraction η of coordinates with largest magnitude and zeros the rest: for transformer hidden states we apply this mask *per token* along the hidden dimension, while for image backbones we flatten each sample’s activation vector and keep the top η fraction of entries *per sample*. For Top- k compression, the indices of the retained values must also be conveyed. We use a PackBits mask, in which each value is assigned one bit to indicate whether it is transmitted, and the resulting bit-packed mask is sent together with the compressed activation. The size of this mask is fixed for a given activation tensor shape and does not vary with the value of η , thus introducing a constant overhead.

Uniform quantization uses symmetric uniform quantization with a per-token (per-row) scale along the hidden dimension. By default, the uncompressed representation uses FP32 precision. To reduce quantization loss, we apply several implementation techniques for lower-precision compressors. For FP16, we use direct datatype conversion together with value clipping to avoid overflow. For INT8, INT4, and INT2, activation values are uniformly mapped to the corresponding integer range based on their minimum and maximum values. In addition, INT4 uses stochastic rounding, which reduces systematic bias and makes the quantization error closer to unbiased in expectation. η is mapped to an integer target bit-width between a minimum of 2 bits (INT2) and the native element width (FP32), thereby enabling the transition from continuous compression control to discrete quantization levels.

LLM.int8-style compression [6] implements a mixed-precision payload: the outlier values are stored separately at outlier precision (FP16 or INT8 with a scalar scale), while the remaining mass is quantized to INT8, INT4, or INT2 using row-wise scaling. We still use a PackBits-style mask to transmit the outlier indices, which introduces a fixed transmission overhead. Within a given experiment, the precision pair for outlier and regular values remains fixed, while the outlier ratio serves as the control parameter for the compression level. This compressor requires an offline-measured mapping between each triplet of (outlier precision, regular precision, outlier ratio) and the resulting η . In the optimization algorithm, we still optimize only over η . During compression and decompression, the corresponding triplet is retrieved by table lookup using the selected η .

Table 8: Offline experiment algorithm parameters. $\mathcal{D}_{\text{test}}$ reports the number of dataset evaluation samples (per task for multi-task scenarios). The Estimator column indicates whether the quality metric is estimated via Stein’s identity or a surrogate accuracy model. For Stein-based estimators, N and $\mathcal{D}_{\text{grad}}$ # samples denote the number of random perturbation directions and the number of samples used per direction, respectively. For surrogate accuracy estimators, N_f and $\mathcal{D}_{\text{grad}}$ # samples denote the number of η points evaluated and the number of $\mathcal{D}_{\text{grad}}$ samples used per η point, respectively. For Stein-based estimators, σ denotes the noise standard deviation. See Table 2 for scenario parameters.

Scenario	μ	ε	$\mathcal{D}_{\text{test}}$ samples	Estimator	$\mathcal{D}_{\text{grad}}$ # samples	σ	N	N_f
<i>Single-task</i>								
L-SG2B	4.0	0.1	32	Fit (<i>poly2</i>)	16	–	–	256
L-SG2B7N	6.0	0.1	32	Fit (<i>poly2</i>)	16	–	–	256
L-SG7B	4.5	0.1	32	Fit (<i>poly2</i>)	16	–	–	256
V-TRS	3.0	0.1	10,000	Stein	100	0.05	20	–
V-LOO	3.0	0.1	10,000	Stein	100	0.05	20	–
L-5shot	0.001	0.1	60	Fit (MLP)	30	–	–	256
L-WT8B	4.5	0.1	32	Fit (<i>poly2</i>)	16	–	–	256
V-ST	0.1	0.1	10,000	Fit (<i>poly3</i>)	10,000	–	–	125
L-ST	0.1	0.1	100	Fit (<i>poly3</i>)	100	–	–	125
<i>Multi-task</i>								
L-SGPT3	4.0	0.1	[16, 16, 16]	Fit (<i>poly2</i>)	[8, 8, 8]	–	–	[256, 256, 256]
V3-LOO	3.0	0.1	[10,000, 10,000]	Stein	[100, 100]	[0.05, 0.05]	[10, 10]	–
LV-MT	0.1	0.1	[10,000, 100]	Fit (<i>poly3</i>)	[10,000, 100]	–	–	[125, 125]

I.4 Scenarios

This section complements the scenario summaries in the main paper—the offline overview in Table 2 and the testbed overview in Table 3—by providing additional information for each scenario not reported in the main body, including the full algorithm parameter listings used to produce our reported results (Table 4 and Table 10) and brief explanations describing scenario-specific details and insights.

For each scenario, we describe the pipeline topology and the dataset subsets used for both utility evaluation and surrogate fitting. We also report dataset versions and model checkpoints at the task level, so that the same inputs can be regenerated from public sources.

Per-task data is drawn from two disjoint sample constructions. $\mathcal{D}_{\text{test}}$ is the evaluation subset used inside the optimization loop to compute the utility metric $\bar{U}$. $\mathcal{D}_{\text{grad}}$ is a separate set drawn with an independent seed, used *only* to either fit the surrogate utility model or supply samples to the Stein zeroth-order estimator; its rows do not overlap with $\mathcal{D}_{\text{test}}$. The **Estimator** column of Tables 8–9 indicates which estimator is used to produce the headline numbers for that scenario—the fitted surrogate (*Fit*) or the Stein oracle (*Stein*). For Stein-based estimators, N and $\mathcal{D}_{\text{grad}}$ # samples denote the number of random perturbation directions and the number of samples used per direction, respectively; for surrogate accuracy estimators, N_f denotes the number of η points evaluated and $\mathcal{D}_{\text{grad}}$ # samples the number of samples per η point, respectively.

I.4.1 Offline Scenarios. We describe all single-task and multi-task offline scenarios below, with algorithm parameters listed in Table 8. For each scenario, we describe the pipeline topology, estimator choice, and dataset configuration.

L-SG2B. Single-task ShareGPT evaluation with Gemma-2 2B on a four-stage pipeline (three links). Forward hooks compress activations after decoder blocks 3, 8, and 12 (18-layer architecture). ShareGPT rows come from Aeala/ShareGPT_Vicuna_unfiltered (default train split) with prompt/response windows capped at 256 tokens. Utility during the run is evaluated on $\mathcal{D}_{\text{test}}$ using the sample count in Table 8; surrogate training fits on $\mathcal{D}_{\text{grad}}$ with a different seed. Reported results follow the *poly2* estimator.

L-SG2B7N. Same ShareGPT task and model family as L-SG2B, but the pipeline is elongated to *seven* stages (six links) with cut indices 2, 5, 8, 10, 12, 14 so that each stage contains fewer transformer blocks; this stresses deeper device chains and matches the seven-node linear topology used for the Raspberry Pi ShareGPT testbed. The intent is to observe scaling effects when only topology depth and link budgets change while the evaluation objective stays ShareGPT perplexity retention. $\mathcal{D}_{\text{test}}$ / $\mathcal{D}_{\text{grad}}$ conventions are as above. The results use the *poly2* estimator.

L-SG7B. ShareGPT with Gemma-2 7B on four stages (three links), hooks after layers 6, 13, and 20 (28-layer model). We use the same ShareGPT dataset protocol as L-SG2B. Again, we report results obtained with *poly2* as our regression fitting.

V-TRS. This scenario runs ResNet-56 on CIFAR-10 in a *threshold* capacity regime ($c \in [0.06, 0.3]$ MB): feasible compression ratios exist in most channel realizations but are occasionally out of reach, placing the optimizer near the boundary of feasibility. Gradients are estimated using a Stein gradient oracle, which queries the fast accuracy callable at $2N$ antithetic perturbations of the current η ; each query runs inference on $\mathcal{D}_{\text{test}}$ with a reduced sample budget.

Table 9: Testbed experiment algorithm parameters. $\mathcal{D}_{\text{test}}$ reports the number of dataset evaluation samples, respectively (per task for multi-task scenarios). The Estimator column indicates whether the quality metric is estimated via Stein’s identity or a surrogate accuracy model. For Stein-based estimators, N and $\mathcal{D}_{\text{grad}}$ # samples denote the number of random perturbation directions and the number of samples used per direction, respectively; for surrogate accuracy estimators, N_f and $\mathcal{D}_{\text{grad}}$ # samples denote the number of η points evaluated and the number of $\mathcal{D}_{\text{grad}}$ samples per η point, respectively. For Stein-based estimators, σ denotes the noise standard deviation. See Table 3 for scenario parameters.

Scenario	μ	ε	$\mathcal{D}_{\text{test}}$ samples	Estimator	$\mathcal{D}_{\text{grad}}$ # samples	σ	N	N_f
<i>Single-task</i>								
R-L-SG2B	1.0	0.1	7	Fit (<i>poly2</i>)	5	–	–	10
P-V-G	4.0	0.1	1,000	Stein	100	0.05	10	–
P-V-E	4.0	0.1	1,000	Stein	100	0.05	10	–
P-L1-G	4.0	0.01	20	Stein	10	0.05	5	–
P-L1-E	4.0	0.01	20	Fit (<i>poly3</i>)	10	–	–	1,000
P-L2-E	4.0	0.1	60	Fit (<i>poly3</i>)	20	–	–	1,000
J-V-S	0.1	1e-7	10,000	Fit (<i>poly3</i>)	10,000	–	–	125
J-L-S	0.1	1e-7	100	Fit (<i>poly3</i>)	100	–	–	125
<i>Multi-task</i>								
P-VV-G	4.0	0.1	[1,000, 1,000]	Stein	[100, 100]	[0.05, 0.05]	[10, 10]	–
P-VV-E	4.0	0.1	[1,000, 1,000]	Stein	[100, 100]	[0.05, 0.05]	[10, 10]	–
P-VL-E	4.0	0.1	[600, 12]	Stein	[100, 10]	[0.05, 0.05]	[10, 5]	–
P-VL-W	4.0	0.1	[600, 12]	Stein	[100, 10]	[0.05, 0.05]	[10, 5]	–
P-LL-E	4.0	0.1	[12, 12]	Stein	[10, 10]	[0.05, 0.05]	[5, 5]	–
J-LV-M	0.1	0.1	[10,000, 100]	Fit (<i>poly3</i>)	[10,000, 100]	–	–	[125, 125]

V-LOO. Also using ResNet-56 on CIFAR-10, this scenario operates in a *loose* capacity regime ($c \in [0.2, 0.6]$ MB), where the channel is almost always wide enough to accommodate feasible compression ratios. As in V-TRS, a Stein gradient oracle provides gradient estimates by evaluating perturbed accuracy values on $\mathcal{D}_{\text{test}}$.

L-5shot. This scenario evaluates Llama-3.1-8B on MMLU in a 5-shot prompting setting under high-capacity conditions ($c \in [10, 40]$ MB). Unlike the vision scenarios, whose Stein oracles query $\mathcal{D}_{\text{test}}$ at every gradient step, this scenario uses a Predictor gradient oracle: an accuracy predictor surrogate is fitted *offline* by evaluating the full-budget accuracy callable on a grid of (η, A_{true}) samples drawn from $\mathcal{D}_{\text{grad}}$, training a concave monotone network to interpolate the accuracy surface. At runtime, exact gradients are obtained in milliseconds via PyTorch autograd through the fitted surrogate, bypassing the need for any further model inference—a critical advantage given the high cost of Llama evaluation. $\mathcal{D}_{\text{test}}$ is used only for final performance reporting, not during optimization.

L-WT8B. WikiText-2 [27] (raw) language modeling with Llama-3.1 8B on four stages (three links) and decoder hooks after layers 7, 15, and 23 (32-layer stack). By default we load `wikitext/wikitext-2-raw-v1` on the test split, extract non-empty text rows, chunk to fixed length 512 tokens with stride 256, and score next-token perplexity retention. As with ShareGPT. Table 8 lists the *poly2* hyperparameters.

J-V-S, J-L-S, LV-MT These scenarios correspond to the single-task and multi-task offline experiments whose parameters, such as c and τ , are obtained from emulated experiments on the Jetson testbed. Model and dataset details are provided in Appendix J.2. Before running the offline experiments, we first perform a set of preliminary measurements on the Jetson testbed to obtain the required scenario parameters for each model-compressor pair, including c , τ , and α . The collected scenario parameters are then used as inputs to the offline experiments on a high-performance computer. In the offline setting, τ includes only the inference delay and the compressor compression/decompression delay. For utility evaluation, we use the full test set. We evaluate both *fitting* and *kernel-smoothing* under multiple values of R_k , while a representative result is reported in Table 4. For *Top-K*, the single-task tradeoff curves are shown in Figures. 12 and 13, while the multi-task tradeoff curves are shown in Figures. 14 and 15.

L-SGPT3. Three concurrent ShareGPT replicas on Gemma-2 2B ($K=3$), each with its own four-stage pipeline but identical topology parameters. This stress-tests proportional baselines when workloads are symmetric and comparatively slack. $\mathcal{D}_{\text{test}} / \mathcal{D}_{\text{grad}}$ follow the ShareGPT protocol with per-task seeds; reporting is *poly2*.

V3-LOO. A multi-task scenario pairing two ResNet-56/CIFAR-10 inference tasks that share the same channel under loose capacity conditions ($c \in [0.2, 0.6]$ MB). The two tasks are assigned heterogeneous activation sizes, testing the scheduler’s ability to allocate capacity fairly across tasks with different compression footprints. Each task maintains an independent Stein gradient oracle that queries $\mathcal{D}_{\text{test}}$ with $N = 10$ antithetic sample pairs; the scenario runs for $T = 100$ time steps.

1.4.2 Testbed Scenarios. We describe all testbed scenarios below, grouped by platform (PRESCIENT, Jetson, Raspberry Pi), with algorithm parameters listed in Table 9. For each scenario, we describe the pipeline topology, network profile, and dataset configuration.

R-L-SG2B. ShareGPT with Gemma-2 2B on seven stages (six links), hooks after layers 3, 5, 8, 10, 12, and 15 (18-layer model). We use the same ShareGPT dataset protocol as L-SG2B with prompt/response windows capped at 50 tokens. The $\mathcal{D}_{\text{test}}$ and $\mathcal{D}_{\text{grad}}$ counts are indicated in Table 9.

P-V-[G, E]. ResNet-56 partitions pipelined across 3 nodes in a linear topology with the network being the differentiator - effectively unbounded or moderately constrained for ResNet-sized activations with some packet loss. The throughput ceiling is compute bound in 1Gbps LAN while W-Edge enables us to measure the network share of the end-to-end cost for small-activation workloads.

P-L1-G. The unbounded network of before is no longer enough to completely decouple task throughput from network owing to the large Llama model activations. Useful to measure how much a gigabit link actually benefits LLM-scale activations and whether optimizers could be necessary in such deployment scenarios as well.

P-L1-E. Large fp16 hidden states on a constrained link — the strongly bandwidth-bound regime. WikiText perplexity gives a smooth, continuous accuracy signal, which makes the resulting trade-off surface the cleanest to read in the whole set. The first experiment scenario where our fitted *poly3* accuracy model outperforms the Stein oracle. The per-slot λ trace shows the active oscillations as the dual variable constantly adjusts compression when a slot runs slow and relaxes when one runs fast. The fitted *poly3* model captures the llm8 accuracy surface for Llama well enough to guide η precisely into the narrow band ($\eta \approx 0.12$) where both accuracy and throughput are good. The Stein oracle over-compresses at ($\eta \approx 0.052$). On Llama activations, the Stein oracle evaluates accuracy by perturbing η and measuring perplexity changes through the simulation pipeline. The perplexity function may have a flatter gradient in the low- η region making it harder for the finite-difference gradient estimate to pull η back up. The LCB channel estimator also contributes — it systematically underestimates capacity (90%, amplified in testbed deployment), and the combined pessimism results in more aggressive compression than necessary.

P-L2-E. Identical setup with Llama model in a heavily constrained network but with the MMLU dataset instead. MMLU accuracy is a 0/1 per-question signal aggregated to a noisy, step-like curve over compression rate. The direct contrast with the WikiText run — same model, same network, same activations, different metric — shows how much the choice of evaluation metric shapes the apparent sensitivity to compression for LLM inference. Another scenario where the fitted *poly3* model outperforms the Stein oracle Alg. 1 with LCB (OURS-NOCSI) optimizer. Due to the nature of the MMLU accuracy function and the limited number of test samples feasible in this network, our optimizers are unable to exploit the flat accuracy function. In fact, some baselines beat Alg. 1 with LCB (OURS-NOCSI) variants (fitted model and Stein oracle) comfortably.

P-VV-[G, E]. Two identical ResNet-56 pipelines with complete node overlap. This overlap is maintained across all multi-task scenarios to introduce FIFO contention dynamics and compute sharing on nodes. Tested under two network profiles - unconstrained and reasonably constrained for multiple pipelines even with ResNet-sized activations.

P-VL-E. Heterogeneous compute and activation sizes on a shared constrained link. Small ResNet-56 tasks queued behind longer running Llama tasks in a shared FIFO queue means total task throughput is bounded by the slower Llama throughput.

P-VL-W. Same pipeline configurations but on an asymmetric, lossy, jittery link prone to packet losses. Link capacity is stochastic and due to the asymmetry of links, the per-link compression trade-off is no long symmetric across the pipeline. Dropped ResNet-56 packets have significantly less impact than dropping large Llama activation payloads resulting in interest throughput performance over long durations.

P-LL-E. Node contention becomes a significant factor as large Llama workloads fill the FIFO queue whose dynamics dominate throughput in contrast to the P-VV-[G, E] scenarios where the same topology and bandwidth suffered from compute queuing instead of network queuing.

J-V-S, J-L-S, J-LV-M This group of scenarios corresponds to real experiments on the Jetson testbed, where both our algorithm and the baselines are executed entirely on the Jetson platform. Details of the topology, network, models, and datasets are provided in Appendix J.2. Testbed experiment results are reported in Table 5. We also provide a *Top-K* single-task testbed result in Figure 4.

Task processing and transmission are fully pipelined on the testbed. The first node serves both as the control node and as the host of the first inference partition; it fetches input samples, runs the first partition, and updates the control policy. The remaining nodes host only their respective inference partitions. On the first node, inference and optimization run on separate threads. Each inference instance reads the latest available policy before execution, while the optimization thread updates the policy in the background and assigns the corresponding η , $s_{i,k}^{\text{comm}}$, and $s_{i,k}^{\text{comp}}$.

The time-slot interval is dynamic. A slot starts when optimization begins and ends after at least five instance-completion feedbacks of the slower task have been collected for the current policy. If policy updating takes longer, the slot includes more instances and ends when the new policy update finishes. In practice, *kernel-smoothing* is slower, so the slot duration is usually aligned with the optimization time, whereas *fitting* is faster and the slot typically ends after five completed instances. To speed up online updates, we set the iteration number J in Algorithm. 2 to 1.

At each node, transmissions for different tasks are carried out independently and in parallel. The allocation of $s_{i,k}^{\text{comm}}$ is realized by assigning different token-bucket rates to different tasks. Each node also stores multiple task models and runs them in parallel on separate threads. Although $s_{i,k}^{\text{comp}}$ is included in the algorithm, it is not explicitly enforced in the Jetson testbed. In our Jetson scenarios, however, transmission delay is much larger than processing delay based on measurement, so this has little effect on the results.

Because precise probing and control of link bandwidth are difficult in a shared Wi-Fi environment, CSI-aware baselines are not evaluated on the Jetson testbed. Moreover, accuracy evaluation in the online testbed setting differs from that in the offline experiments: rather than evaluating over the full test set, we compute accuracy using only the completed instances for which feedback is received.

Unlike the offline setting, where each task has one instance per slot, the number of completed instances may vary across tasks within the same slot in the online testbed. Therefore, the actual bottleneck delay $D_{\text{act},k}(t)$ is first averaged across completed instances within each task k and slot t , then aggregated across tasks with weights w_k .

J TESTBED DETAILS

We ran our experiments on the following three testbeds:

Raspberry Pi. We deploy the system on a cluster of seven Raspberry Pi 5 nodes (2.4 GHz quad-core ARM, 8 GB RAM), interconnected over 802.11ac Wi-Fi. The inference model is Gemma 2B (18 hidden layers), distributed across the cluster as a pipeline: a dedicated user-equipment node handles embedding and sampling, while the 17 transformer layers are partitioned across the seven Pis. Compression is applied at the seven inter-node cut points where activation tensors cross the wireless link. Multiple requests are served concurrently via pipeline parallelism, each identified by a unique tag routed to per-request KV caches. A central controller on a separate machine periodically collects per-node metrics, runs the optimizer, and broadcasts updated per-link compression ratios without pausing inference.

NVIDIA Jetson. Our Jetson-based testbed consists of four NVIDIA Jetson Orin Nano Developer Kits interconnected through a shared 802.11ac Wi-Fi access point. Each node is equipped with a compact Ampere-architecture GPU, a 6-core Arm Cortex-A78 CPU, and 8 GB of 128-bit LPDDR5 memory. Inference is executed on the GPU, while compression, decompression, and other computations run on the CPU. Nodes communicate over HTTP, and bandwidth is controlled at the application layer using a token-bucket mechanism, yielding only coarse-grained control in practice. On this platform, we evaluate both a vision model, ResNet-56 on CIFAR-10 and a language model Flan-T5-Base on SST-2, each partitioned into pipeline stages across the nodes to balance the workload. In both cases, performance is measured by classification accuracy.

PRESCIENT. An experiment framework deployed on PRESCIENT, a GPU-dense network research testbed, converts pipeline and topology specifications along with scenario and algorithmic configurations into experiments that deploy as “pods” in PRESCIENT’s Kubernetes environment. For each experiment, pods consist an orchestrator and a set of nodes each running a stage of one (single-task) or more (multi-task) pipelines. The orchestrator runs the optimizers, manages configuration updates on node pods, and generates and measures pipeline tasks over the lifetime of an experiment.

We elaborate on each of the three testbed implementations below.

J.1 Raspberry Pi

J.1.1 Hardware and Network. The testbed consists of seven Raspberry Pi 5 single-board computers, each equipped with a 2.4 GHz 64-bit quad-core ARM Cortex-A76 CPU and 8 GB of LPDDR4X RAM. All nodes connect to a shared 802.11ac Wi-Fi access point. An eighth device (laptop) on the same network hosts the central controller process. All inference is CPU-bound; no GPU acceleration is available. Inter-node Wi-Fi links are subject to contention, fading, and interference from other devices on the network.

J.1.2 Model and Layer Assignment. The inference model is Gemma 2B, a decoder-only transformer with $L = 18$ hidden layers. A dedicated user-equipment (UE) node hosts layer 0 and the remaining 17 transformer layers are spread across seven Pis: layers 1–3 on the first node, layers 4–5 on the second, layers 6–8 on the third, layers 9–10 on the fourth, layers 11–12 on the fifth, layers 13–15 on the sixth, and layers 16–17 on the seventh.

J.1.3 Cut Points and Pipeline Parallelism. The model has seven inter-node cut points, located at the output of the last hosted layer on each node: layers 3, 5, 8, 10, 12, and 15. At each cut point the output activation tensor is serialized, compressed according to the optimizer-assigned ratio $\eta_i(t)$, and transmitted to the downstream node over Wi-Fi. The ten remaining layer boundaries are intra-node transfers carried over local memory without compression. Multiple inference requests are served concurrently. Each request carries a unique identifier that routes it to a dedicated KV cache at every pipeline node. Requests are injected with a stagger delay, so that at steady state every node processes a different request’s token simultaneously.

J.1.4 Central Controller. A lightweight controller process runs on a laptop connected to the same Wi-Fi network, operating on a periodic schedule. At each control step, the controller first collects per-node metric reports delivered over UDP, including the computation latency, link throughput, and uncompressed activation size from every pipeline node. It then invokes the selected optimization algorithm to determine the compression ratio for each inter-node link. Finally, it broadcasts the updated compression ratio and codec identifier to every node via UDP. Inference proceeds continuously and the pipeline does not pause between control updates.

J.2 NVIDIA Jetson

J.2.1 Setup. Our Jetson-based testbed consists of four Jetson Orin Nano Developer Kits. Each node is an edge computing device with a compact Ampere-architecture GPU, a 6-core Arm Cortex-A78 CPU, and 8GB of 128-bit LPDDR5 memory. CUDA-supported workloads, such as model inference, are executed on the GPU, while non-CUDA components, such as compression/decompression and optimization, are executed on the CPU. All nodes are connected through a shared 802.11ac Wi-Fi access point. Each node runs an HTTP service, and inter-node communication is carried out over HTTP. Bandwidth is controlled at the application layer on each node using a token-bucket

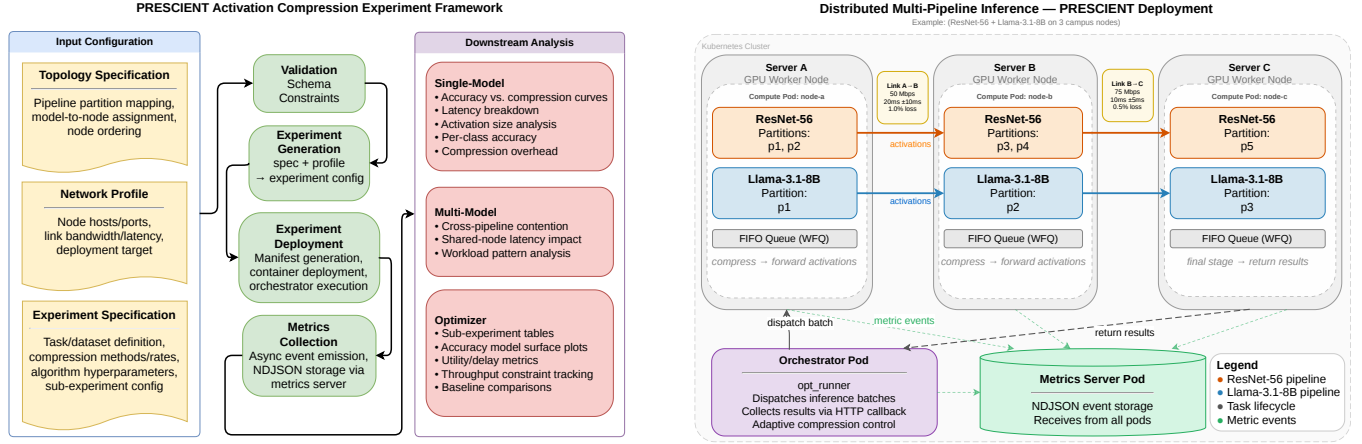


Figure 11: (Left) Experiment framework pipeline from input YAML specifications through validation, generation, deployment, and metrics collection to downstream analysis. (Right) Kubernetes deployment of a multi-pipeline optimizer experiment (ResNet-56 + Llama-3.1-8B) across three campus servers with TC-emulated inter-pod links.

mechanism to avoid exceeding the hardware bandwidth limit of the access point. As this control is implemented above the network stack, the resulting bandwidth shaping is not precise and is susceptible to environmental interference.

J.2.2 Models and Datasets. We use two models and their corresponding datasets on the Jetson platform. For the vision task, we use ResNet-56 on CIFAR-10. The ResNet-56 model we used is a CIFAR-10-specific variant whose architecture is adapted to the input size of CIFAR-10 samples. ResNet-56 is split into four pipeline stages: The four model partitions are deployed sequentially across the four nodes, hosting the stem and layer1.0–layer1.6, layer1.7–layer2.3, layer2.4–layer3.1, and layer3.2–layer3.8 plus the classification head, respectively. This partition is determined based on the computational cost of each layer to balance the inference workload across nodes. For CIFAR-10, we use only the test set, which is divided into 100 batches, thus each batch consists of 100 images. Evaluation is based on classification accuracy; For the language task, we use Flan-T5-Base on SST-2. Flan-T5-Base follows an encoder-decoder architecture and is also split into four stages: Node A hosts encoder layers 0–3; Node B hosts encoder layers 4–7; Node C hosts encoder layers 8–11; and Node D hosts the full decoder together with the final label decision. For SST-2, we use only the first 100 samples from the test set and divide them into 100 batches, each containing one sample. Since SST-2 is a binary classification task, we also use classification accuracy as the evaluation metric. Under SST-2, the output length is only one token. In addition, all SST-2 inputs are padded to a fixed length of 128 tokens to avoid truncation and to keep the inference workload consistent across samples. Note that, for ResNet-56 on CIFAR-10, *LLM.int8* uses the FP16+INT4 precision combination, whereas for Flan-T5 on SST-2, it uses FP16+INT8.

J.3 PRESCIANT

PRESCIANT (Platform for REsearch on Intelligence in Edge-enabled Next-generation Telecommunications) is a wireless network research platform at the Ohio State University. The platform is designed for over-the-air and emulated wireless experiments at scale, providing researchers with programmable radio hardware, dense GPU-accelerated compute nodes, and a Kubernetes-managed software infrastructure for experiment orchestration.

J.3.1 Hardware Configuration.

- **4× multi-GPU nodes.** Multi-GPU x86 nodes equipped with either Nvidia’s H100 or L40s GPUs. A single NVIDIA GH200 Grace Hopper (ARM64) server that supports Nvidia’s GPU-accelerated AERIAL RAN research platform.
- **4× USRP X410 software-defined radios.** High-performance SDR platforms capable of wideband transmission; used for over-the-air 5G NR experiments.
- **5× USRP N300 software-defined radios.** Mid-range SDR platforms used as additional radio access points or for channel sounding.
- **User Equipment (UEs).** COTS UEs attached to the platform for end-to-end RAN experiments.

J.3.2 Experimentation Framework. The platform is managed as a **Kubernetes** cluster with each server accessible as a worker node. Experiment workloads are packaged as containers and scheduled across nodes via standard Kubernetes manifests. Network policies, resource quotas, and inter-pod connectivity can either be expressed through the Kubernetes control plane or deployed per node for experiments requiring higher granularity or specific controls.

An experimentation framework developed to accelerate experiment specification, deployment, and data analysis runs on PRESCIANT and is responsible for generating numerous experiment configurations given topology, profile and algorithmic configuration templates. The

framework generates experiments based on scenario definition and for each optimizer (baselines and ours), runs the optimization loop for T slots. Profiling is performed once per scenario and cached for re-use later by experiments based on the same scenario.

PRESCIENT Experiment Framework - Optimization Algorithm

Input: K pipelines with throughput targets R_k , weights w_k ; profiled latencies $\tau_i^{(k)}$, activation sizes $a_\ell^{(k)}$; compression bounds $[\eta_\ell^{\min}, \eta_\ell^{\max}]$ per link; surrogate accuracy models $\hat{A}_k(\eta_k)$; channel estimator $\mathcal{E}$; slots T , batches B , step size μ , probe interval P
Output: per-slot $\eta_k(t)$, $s(t)$, $\hat{R}_k(t)$

```

1: Initialise  $\lambda_k \leftarrow 0$  for all  $k$ 
2: for  $t = 0, 1, \dots, T - 1$  do
3:   if  $t \bmod P = 0$  then
4:     Probe links  $\rightarrow c_t$ ;   update  $\mathcal{E}$  with  $c_t$ 
5:   end if
6:    $\hat{c}_t \leftarrow \mathcal{E}.\text{estimate}(t)$ 
   // Primal step
7:    $\{\eta^*, s^*\} \leftarrow \arg \max_{\eta, s} \sum_{k=1}^K [w_k \hat{A}_k(\eta_k) - \lambda_k d_k(\eta_k, s_k, \hat{c}_t)]$ 
   s.t.  $\eta_{k,\ell} \in [\eta_\ell^{\min}, \eta_\ell^{\max}]$ ,  $\sum_k s_{k,i} = 1 \forall i$ 
8:   Map each  $\eta_{k,\ell}^*$  to compression method; push config to nodes
9:   Run  $B$  batches; observe achieved throughput  $\hat{R}_k(t)$ 
   // Dual update
10:  for each pipeline  $k$  do
11:     $\lambda_k \leftarrow \max(0, \lambda_k + \mu \cdot \max(0, 1/\hat{R}_k - 1/R_k))$ 
12:  end for
13: end for
```

Preceding offline phases:

- (1) *Profiling* – run inference at $\eta = \eta^{\max}$ to measure $\tau_i^{(k)}$, $a_\ell^{(k)}$, and nominal link capacities.
 - (2) *Accuracy model fitting* – sweep η_k via simulation, fit surrogate $\hat{A}_k$.
-

Similarly, accuracy model oracles are trained off line once per a set of pipeline, dataset, and compression and cached for re-use by subsequent experiments. The training allows for multiple modes of sampling compression rates - along the diagonal and random sampling. For each compression rate sample, accuracy is evaluated over a set of batches of the pipeline model dataset.

For running a complete experiment with multiple baseline and proposed optimizers, the choice of R_k , especially in a real, on-line deployment is important as it is easy to select one which is either trivially satisfied or is infeasible given communication overhead not captured during link bandwidth probing. The framework performs an off line R_k selection that is feasible and appropriate for the network profile being emulated per experiment.

Infrastructure/network profiles.

W-Edge. Both inter-node links (A→B and B→C) are shaped to 100 Mbps with 10ms of delay and 0.2% packet loss. This profile represents a constrained wireless or cellular backhaul link and is the primary baseline for evaluating the benefit of adaptive compression.

1Gbps LAN. Both links are shaped to 1 Gbps with no added delay. This profile models a high-bandwidth local-area or fronthaul network; compression overhead is minimal and end-to-end latency is dominated by compute.

MANET. Asymmetric link conditions modelling a wireless or mobile ad hoc network: the A→B link is constrained to 50 Mbps with 20 ms propagation delay (± 5 ms jitter) and 0.5 % packet loss, while B→C is constrained to 100 Mbps with 5 ms delay (± 1 ms jitter) and 0.1 % packet loss. This profile reflects a scenario in which the first pipeline stage operates at a remote or low-bandwidth site (e.g., an edge device in the field) while later stages are co-located in a slightly higher-capacity facility connected by a cellular backhaul link.

J.3.3 Deployment. The distributed model pipeline is realized as a set of Kubernetes pods scheduled across the available compute nodes with each pod provisioned access to an Nvidia GPU. Each pod runs a container that hosts one or more consecutive model partitions; activations are transmitted between pods over the cluster network. Network emulation via `tc/netem` is applied at each link (pod egress and ingress) to instantiate the infrastructure profiles described below. The optimizer experiments additionally use a GPU provisioned orchestrator pod for Stein-oracle gradient estimation, which requires running multiple forward passes through the simulated model in parallel for each optimization time slot. 11 shows the functional components of the framework and its workflow while also visualizing an example scenario experiment deployed on PRESCIENT.

PRESCIENT Experimentation Framework - Surrogate Accuracy Model Training

Input: pipeline k with L_k inter-node links, compression bounds $[\eta_\ell^{\min}, \eta_\ell^{\max}]$; sweep count N_s , sweep mode $\in \{\text{random}, \text{diagonal}\}$; dataset $\mathcal{D}_k$; compression scheme s ; surrogate type (e.g. poly3, gbm)

Output: fitted surrogate $\hat{A}_k : \mathbb{R}^{L_k} \rightarrow [0, 1]$

```

1: Load simulation pipeline from stored partitions and dataset  $\mathcal{D}_k$ 
2: Configure compression scheme  $s$  on all links

3: if sweep mode = diagonal then
4:    $\alpha \leftarrow \text{linspace}(0, 1, N_s)$ 
5:   for  $j = 1, \dots, N_s$  do
6:      $\eta^{(j)} \leftarrow \eta^{\min} + \alpha_j (\eta^{\max} - \eta^{\min})$ 
7:   end for
8: else // random
9:   for  $j = 1, \dots, N_s$  do
10:     $\eta^{(j)} \sim \text{Uniform}[\eta^{\min}, \eta^{\max}]$ 
11:   end for
12: end if

13: for  $j = 1, \dots, N_s$  do
14:   Apply compression at  $\eta^{(j)}$  under scheme  $s$ 
15:    $y_j \leftarrow$  evaluate task accuracy on  $\mathcal{D}_k$  via simulation
16: end for

17: Fit surrogate  $\hat{A}_k$  on  $\{(\eta^{(j)}, y_j)\}_{j=1}^{N_s}$ 

```

Diagonal sweeps all links at the same relative compression level (useful for low-dimensional characterisation); **random** covers the full L_k -dimensional hypercube. Surrogate types include polynomial ridge regression (poly2, poly3), gradient boosted trees (gbm), and MLPs.

PRESCIENT Experimentation Framework - R_k Selection Algorithm

Input: experiment configuration (pipelines, nodes, links, compression schemes $\mathcal{S}$)

Output: per-pipeline throughput target R_k

```

1: Run profiling at  $\eta = \eta^{\max}$ ; measure  $\tau_i^{(k)}, a_\ell^{(k)}$ , link capacities  $c$ 
2: for each scheme  $s \in \mathcal{S}$  do
3:   Run no-compression baseline ( $\eta = \eta^{\max}$ ); record  $\hat{R}_k^{\text{none},s}$ 
4:   Run max-compression baseline ( $\eta = \eta^{\min}$ ); record  $\hat{R}_k^{\text{max},s}$ 
5: end for

6: for each pipeline  $k$  do
7:    $\underline{R}_k \leftarrow \min_s \hat{R}_k^{\text{none},s}$  // slowest no-compression run
8:    $\overline{R}_k \leftarrow \min_s \hat{R}_k^{\text{max},s}$  // slowest max-compression run
9:    $R_k \leftarrow \underline{R}_k + \alpha (\overline{R}_k - \underline{R}_k), \quad \alpha \in (0, 1)$ 
10: end for

```

α controls difficulty: values near 0 are easily met without compression; values near 1 approach the physical throughput ceiling and risk infeasibility. A moderate α (e.g. 0.4–0.6) ensures baselines that ignore channel or accuracy structure violate the constraint while leaving headroom for adaptive optimizers.

K ADDITIONAL EXPERIMENTS

K.1 Testbed Experiments

Implementing CSI-aware methods and baselines on real testbeds is challenging. On the Jetson testbed, kernel-level limitations prevent accurate network control without external devices, so both traffic shaping and link probing must be done at the application layer, which is coarse and imprecise. In addition, the shared Wi-Fi medium is easily affected by interference. Therefore, we do not evaluate CSI-aware methods on Jetson.

Table 10: Testbed results for single-task (upper block) and multi-task (lower block) settings including the CSI-aware methods. We report aggregate utility $\bar{U}$, average delay $\bar{D}$, and excess delay $\bar{\Delta D}$, across the evaluated methods, grouped into reference baselines, CSI-aware methods, and No-CSI methods. Compression schemes T, Q, and I8 denote Top- k , quantization, and LLM.int8, respectively. Infeasible entries ($\bar{\Delta D}/\bar{D} > 0.05$) are shaded light red. Bold and underlined values denote the best and second-best accuracies, respectively, among feasible entries within the CSI-aware and No-CSI groups.

Single-task Scenarios																																					
Scenario	Metric	Reference				CSI-aware								No-CSI																							
		NONE				UNI				OURS-CSI				MYO				CONS				MA				OURS-NOCSI											
		T	Q	I8		T	Q	I8		T	Q	I8		T	Q	I8		T	Q	I8		T	Q	I8		T	Q	I8									
R-L-SG2B	$\bar{U}$	1.00	0.331	0.242	0.552	0.657	0.449	0.671	0.698	0.490	0.747	0.686	0.470	0.742	0.607	0.443	0.702	0.718	0.506	0.798	0.676	0.469	0.713														
	$\bar{D}$	320	102	91.1	97.0	142	124	166	155	137	159	307	285	293	154	143	162	288	265	284	86.0	81.1	128														
	$\bar{\Delta D}$	120	-97.9	-109	-103	-58.0	-75.9	-33.8	-44.8	-63.1	-41.5	107	84.9	92.7	-45.7	-57.3	-38.0	88.2	65.1	84.3	16.5	-119	-71.8														
P-V-G	$\bar{U}$	0.927	0.120	0.114	0.751	0.897	0.927	0.929	0.899	0.927	0.934	0.900	0.927	0.934	0.880	0.927	0.935	0.907	0.927	0.934	0.855	0.648	0.892														
	$\bar{D}$	236.334	226.023	169.156	221.468	235.713	178.333	230.703	287.965	175.316	278.749	285.990	175.103	278.655	287.929	178.763	278.887	286.529	179.709	278.993	183.209	183.649	187.401														
	$\bar{\Delta D}$	9.061	-1.250	-58.117	-5.805	8.441	-48.940	3.430	60.692	-51.957	51.476	58.718	-52.170	51.382	60.656	-48.510	51.615	59.256	-47.564	51.720	-44.064	-43.624	-39.871														
P-V-E	$\bar{U}$	0.927	0.120	0.114	0.751	0.563	0.928	0.903	0.551	0.933	0.908	0.551	0.933	0.908	0.543	0.933	0.909	0.546	0.933	0.909	0.533	0.684	0.845														
	$\bar{D}$	676.002	197.228	197.498	209.463	210.430	242.690	241.475	274.360	259.501	255.704	285.748	264.321	255.146	284.461	268.449	248.806	285.830	263.583	252.235	235.449	222.658	205.917														
	$\bar{\Delta D}$	437.907	-40.867	-40.597	-28.632	-27.665	4.595	3.380	36.265	21.406	17.609	47.653	26.226	17.051	46.366	30.354	10.711	47.735	25.487	14.139	-2.646	-15.437	-32.178														
P-L1-G	$\bar{U}$	1.000	0.192	0.028	0.682	0.995	1.000	0.999	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.639	0.066	0.942														
	$\bar{D}$	296.553	281.692	236.484	273.836	294.012	254.720	284.012	358.007	329.074	321.507	338.583	294.891	307.931	338.732	287.429	316.593	343.194	294.311	331.681	262.093	260.805	236.245														
	$\bar{\Delta D}$	33.395	18.534	-26.674	10.678	30.854	-8.438	20.855	94.849	65.916	58.349	75.425	31.733	44.773	75.575	24.271	53.435	80.036	31.153	68.523	-1.065	-2.353	-26.913														
P-L1-E	$\bar{U}$	1.000	0.192	0.028	0.677	0.891	0.013	0.981	0.957	0.018	0.989	0.957	0.013	0.989	0.956	0.013	0.987	0.957	0.013	0.989	0.691	0.087	0.857														
	$\bar{D}$	790.384	286.550	335.390	291.654	355.782	451.075	383.187	460.639	473.970	437.899	462.651	457.479	434.263	453.741	453.214	437.973	458.281	448.635	435.632	337.721	314.389	343.769														
	$\bar{\Delta D}$	420.014	-83.820	-34.981	-78.716	-14.589	80.704	12.817	90.269	103.600	67.528	92.281	87.109	63.892	83.370	82.843	67.602	87.911	78.265	65.262	-32.649	-55.982	-26.601														
P-L2-E	$\bar{U}$	0.467	0.267	0.283	0.283	0.387	0.300	0.474	0.413	0.300	0.475	0.408	0.300	0.477	0.417	0.300	0.481	0.417	0.300	0.478	0.269	0.282	0.397														
	$\bar{D}$	901.130	274.393	280.272	297.253	294.294	439.337	819.023	390.254	655.895	867.009	385.275	645.917	874.369	390.544	651.298	862.062	386.328	655.560	863.350	290.246	309.594	358.331														
	$\bar{\Delta D}$	401.130	-225.607	-219.728	-202.746	-205.706	-60.663	319.023	-109.746	155.895	367.009	-114.725	145.917	374.369	-109.456	151.298	362.062	-113.672	155.560	363.350	-209.754	-190.406	-141.669														
J-V-S	$\bar{U}$	0.933	0.414	0.870	0.930	-	-	-	-	-	-	0.894	0.933	0.932	0.856	0.933	0.932	0.893	0.932	0.894	0.925	0.930															
	$\bar{D}$	3772	476	469	510	-	-	-	-	-	-	1955	1954	1960	1620	1780	1173	1941	1942	1992	1900	1270	972														
	$\bar{\Delta D}$	1878	-1410	-1430	-1400	-	-	-	-	-	-	68.3	60.1	54.5	-267	-114	-733	53.4	47.5	86.9	12.5	-624	-934														
J-L-S	$\bar{U}$	0.934	0.863	0.484	0.939	-	-	-	-	-	-	0.939	0.486	0.939	0.942	0.486	0.939	0.939	0.486	0.939	0.939	0.540	0.939														
	$\bar{D}$	237	98.1	95.5	96.2	-	-	-	-	-	-	235	179	249	148	153	129	230	178	243	149	95.9	98.9														
	$\bar{\Delta D}$	65.2	-95.2	-76.1	-156	-	-	-	-	-	-	41.5	7.86	-3.39	-45.0	-18.1	-123	36.2	6.60	-9.04	-44.0	-75.7	-153														
Multi-task Scenarios																																					
Scenario	Metric	Reference				CSI-aware								No-CSI																							
		NONE				EQ				PROP				PRIO				OURS-CSI				D-EQUAL				D-L-PROP				MA				OURS-NOCSI			
		T	Q	I8		T	Q	I8		T	Q	I8		T	Q	I8		T	Q	I8		T	Q	I8		T	Q	I8		T	Q	I8					
P-VV-G	$\bar{U}$	0.924	0.094	0.086	0.758	0.675	0.924	0.913	0.672	0.924	0.912	0.486	0.524	0.844	0.094	0.086	0.758	0.094	0.086	0.758	0.094	0.086	0.758	0.109	0.109	0.759											
	$\bar{D}$	430	415	376	383	417	396	411	416	387	414	365	355	402	384	355	399	390	351	394	388	343	393	393	343	396	286	283	294								
	$\bar{\Delta D}$	193	163	85.6	98.9	167	126	156	165	108	162	63	43.1	137	101	43.9	131	113	34.9	120	109	18.4	120	119	20.2	125	-95.1	-101	-79.5								
P-VV-E	$\bar{U}$	0.924	0.094	0.086	0.758	0.541	0.924	0.895	0.543	0.924	0.895	0.432	0.524	0.843	0.094	0.086	0.763	0.094	0.086	0.763	0.094	0.086	0.763	0.389	0.325	0.765											
	$\bar{D}$	1397	465	412	469	606	531	517	599	529	509	545	540	486	427	437	495	424	432	487	436	433	483	426	428	493	358	344	369								
	$\bar{\Delta D}$	1842	-21.8	-128	-13.6	259	110	82.3	246	107	65.8	139	127	20.3	-98.9	-78	37	-104	-89.1	21.1	-80.6	-86.8	13.2	-101	-95.8	34	-236	-264	-215								
P-VL-E	$\bar{U}$	0.962	0.153	0.0593	0.737	0.812	0.483	0.948	0.812	0.483	0.948	0.56	0.551	0.879	0.153	0.0593	0.737	0.153	0.0593	0.737	0.153	0.0593	0.737	0.45	0.232	0.844											
	$\bar{D}$	1350	492	431	487	668	816	786	661	849	770	756	837	879	443	456	475	475	480	471	471	449	483	474	452	487	388	380	446								
	$\bar{\Delta D}$	1368	-349	-471	-360	2.23	298	239	-12.3	364	208	178	341	424	-448	-422	-384	-384	-374	-392	-391	-436	-368	-384	-429	-360	-557	-574	-441								
P-VL-W	$\bar{U}$	0.947	0.147	0.059	0.707	0.616	0.411	0.901	0.619	0.404	0.901	0.516	0.0846	0.845	0.147	0.059	0.707	0.147	0.059	0.707	0.147	0.059	0.707	0.184	0.0718	0.716											
	$\bar{D}$	6107	803	962	1249	1438	1909	2136	1487	1872	2150	1855	2455	2369	753	974	1237	766	940	1266	767	951	1252	769	968	1228	865	996	1225								
	$\bar{\Delta D}$	9357	-1250	-934	-358	19.3	961	1415	118	888	1444	854	2054	1881	-1350	-909	-382	-1324	-976	-325	-1323	-956	-352	-1318	-922	-401	-1126	-864	-405								
P-LL-E	$\bar{U}$	1.00	0.211	0.0327	0.716	0.994	0.0421	1.00	0.994	0.0421	1.00	0.606	0.516	0.858	0.211	0.0327	0.716	0.211	0.0327	0.716	0.211	0.0327	0.716	0.594	0.0327	0.898											
	$\bar{D}$	1492	554	519	518	769	1134	807	764	1151	804	747	861	793	585	507	515	573	488	512	577	491	510	576	477	511	491	495	518								
	$\bar{\Delta D}$	1555	-321	-390	-393	109	841	185	98.8	875	180	66.1	294	157	-258	-414	-398	-283	-453	-405	-274	-446	-409	-277	-475	-407	-446	-438	-393								
J-LV-M	$\bar{U}$	0.926	0.418	0.860	0.929	-	-	-	-	-	-	-	-	-	-	-	-	0.707	0.915	0.932	0.836	0.915	0.932	0.928	0.927	0.933	0.747	0.913	0.932								
	$\bar{D}$	4330	1090	957	1680	-	-	-	-	-	-	-	-	-	-	-	-	2440	1230	2880	3590	1210	2670	4880	3990	13800	2400	1450	2730								
	$\bar{\Delta D}$	1900	-1350	-1480	-760	-	-	-	-	-	-	-	-	-	-	-	-	9.55	-1210	443	1160	-1230	234	2450	1550	11400	-30.3	-986	285								

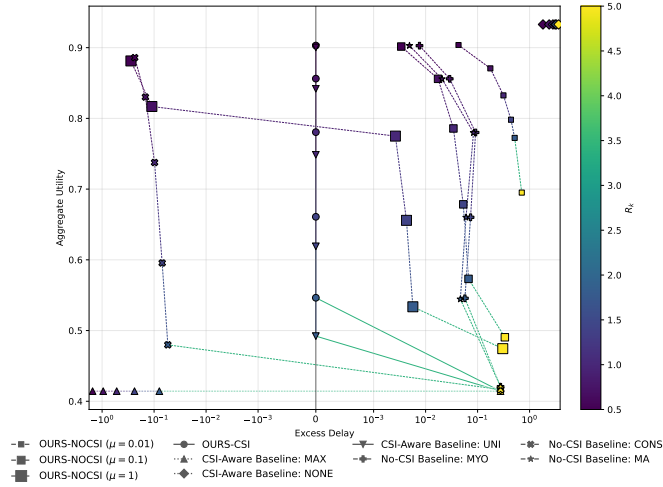


Figure 12: Scenario V-ST , RN56-CF10 single task results (Fitting-Based, Top-K)

This discrepancy reveals a fundamental mismatch in capacity measurement on our real testbed to that in the simulated environment. CSI-awareness in the PRESCIENT testbed is achieved via link probing in every optimization time slot (t) where a payload is transmitted and timed over each link (ℓ) to produce a bandwidth measurement $c_\ell(t)$. This kind of naïve probing measures bulk throughput in isolation and systematically overestimates the effective capacity available to actual activation transfers, which are smaller (100 KiB– 16 MiB) and incur significant per-request HTTP overhead. Moreover, to ensure model pipeline stages are filled to maximize pipeline throughput, there will always be as many inference tasks in-flight, W , as there are stages and hence, activation transfers also need to contend with W in-flight tasks. The testbed CSI measurement attempts to account for some of these losses – a wire overhead factor $\omega_\ell \in [1.15, 1.5]$ corrects the activation size parameter to $a_\ell \cdot \omega_\ell$ and $c_\ell(t)/W$ is the contention-corrected capacity. So the delay model computes

$$\frac{(a_\ell \cdot \omega_\ell) \cdot \eta_\ell}{\hat{c}_\ell / W}$$

which is a tighter measurement but still fails to capture second-order sensitivities that get amplified in the CSI-Aware Optimal (OURS-CS) optimizer.

Probing noise is significant under the W-Edge and MANET emulated network profiles. With jitter and packet loss, each 20 MiB probe produces a noisy $c_\ell(t)$. CSI-aware uses this instantaneous value directly with no smoothing – an upward noise draw produces an inflated η for the entire slot. The Uniform-compression (UNI) is partially shielded by this phenomenon as it computes $\eta = \min_\ell(\hat{c}_\ell / R_k a_\ell)$, where even if one link’s probe comes in high due to noise, the optimizer selects the lowest ratio across links, thus ignoring the upward outlier. If the bottle-necked link gets an upward noise draw it will affect the Uniform-compression (UNI) decision, but other links will have excess capacity to absorb it. The no-CSI optimizers use channel estimators (moving average, LCB) that explicitly smooth this variance. CSI-Aware Optimal (OURS-CS) aggressively adapts η per link capacity thus lacking the cross-link dampening inherent to Uniform-compression (UNI) or Equal-share (EQ) optimizers.

Additionally, the wire overhead factor (ω) is calculated during a no-compression profiling run at maximum activation sizes and applied as a constant through all slots. Wire overhead at those activation sizes is a much smaller fraction of the payload than at lower η . HTTP connection setup, headers, and other such per-request costs can be more efficiently amortized at higher payloads but as η shrinks the fixed cost fraction grows and the true overhead is significantly higher.

Mitigations, therefore, include probing with activation-realistic payloads and with multiple concurrent probe trains which aim to recreate channel contention proportional to the window-size required to keep the pipeline fully loaded (W). Thus, instrumenting robust CSI measurement on a real testbed that matches the simple delay model of the simulation environment can be non-trivial and provide a strong case for Alg. 1 with LCB (OURS-NOCSI) and Alg. 2 with LCB (OURS-NOCSI) optimizers for such deployments.

The Raspberry Pi testbed shares the same Wi-Fi interference sensitivity as the Jetson testbed, yet its full Linux kernel access and lightweight communication stack make CSI-aware operation feasible. Channel probing on this testbed operates in a regime where the probe-to-activation mismatch is minimal. Each non-terminal pipeline node runs a background thread that periodically opens a TCP connection to its downstream neighbor, transmits a fixed-size probe payload B_{probe} sized to match the decode-phase activation, and records the round-trip time Δt_{probe} . The per-link capacity estimate is then $\hat{c}_i^{\text{probe}}(t) = B_{\text{probe}} / \Delta t_{\text{probe}}$. Because the probe payload is deliberately matched to the hidden-state activation size O_i , the payload-dependent overhead discrepancy is eliminated. Furthermore, inter-node communication uses raw TCP sockets with a lightweight serialization protocol rather than HTTP, making per-request wire overhead negligible.

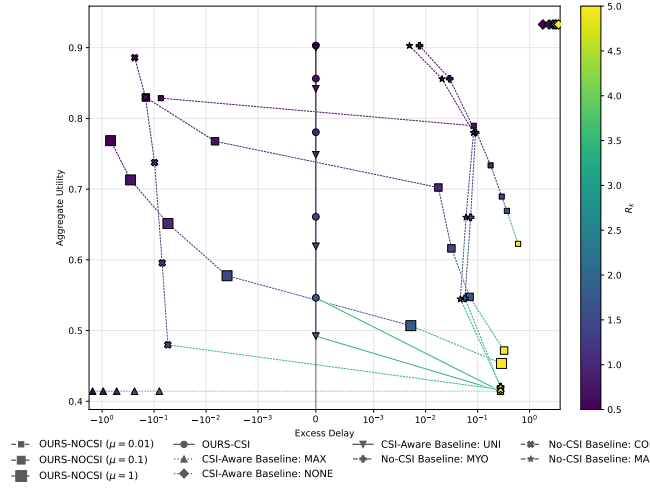
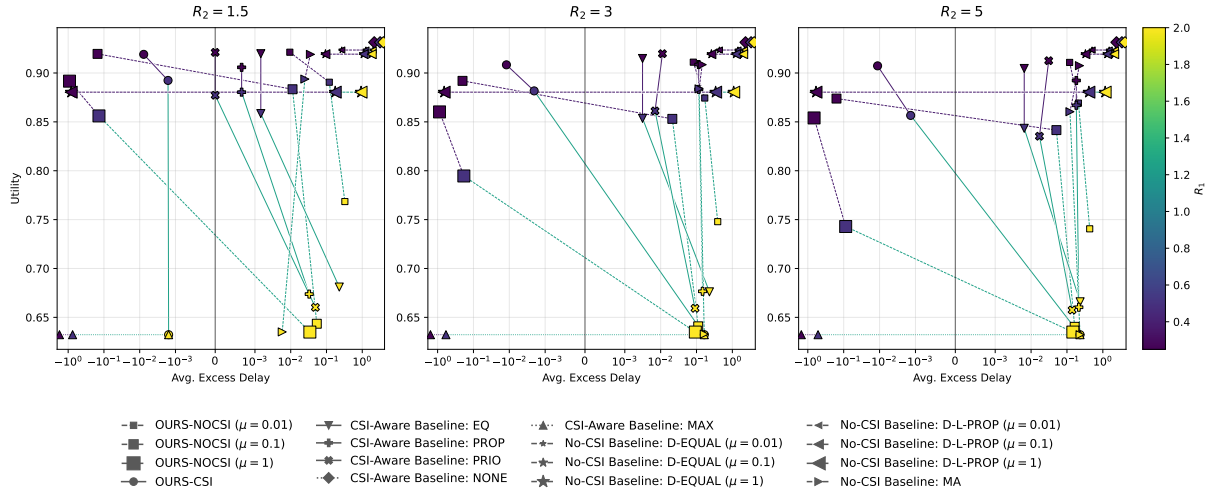


Figure 13: Scenario V-ST , RN56-CF10 single task results (Stein's, Top-K)

Figure 14: Top-K multi-task tradeoff under the fitting-based estimator. Each panel fixes the Task-2 target rate R_2 and sweeps the Task-1 target rate R_1 . The x-axis shows the overall average excess delay, and the y-axis shows the overall utility.

K.2 Accuracy-Delay Tradeoff Trajectory under Different R_k

We sweep both R_k and μ , and further compare two accuracy-estimation methods, *fitting* and *kernel-smoothing* (Stein's method) under scenario V-ST. For the *Top-K* compressor, the resulting tradeoff curves are shown in Figure 12 and 13. We also evaluate the *quantization* and *LLm.int8*. For these compressors, however, the accuracy decreases only mildly as η becomes smaller, and thus no tradeoff plots are included here. These two figures illustrate the single-task accuracy-delay tradeoff. The experiments compare multiple decision policies across six target-rate R_k settings. The x-axis shows the average excess delay, and the y-axis shows the average true accuracy. Each curve corresponds to one policy, and the color of each point indicates the target-rate setting.

First, we observe that our algorithm, especially the No-CSI, performs well across different values of R_k . Compared with the baselines, it is more likely to satisfy the target-rate constraint while maintaining higher accuracy. At the same time, there is a clear upper limit on R_k : once the target rate exceeds the capability of the compression mechanism, the algorithm fails to satisfy the constraint. This limitation also applies to the baselines.

Then, the comparison between *fitting* and Stein's method suggests that, under our No-CSI algorithm, the fitting-based estimator tends to perform better relative to the baselines. In particular, it often achieves higher accuracy while still satisfying the target delay constraint. A likely reason is that, to reduce the computational overhead of Stein-based estimation, we use relatively small values of N and $\mathcal{D}_{\text{grad}}$, which can lead to lower estimation quality for Stein's method.

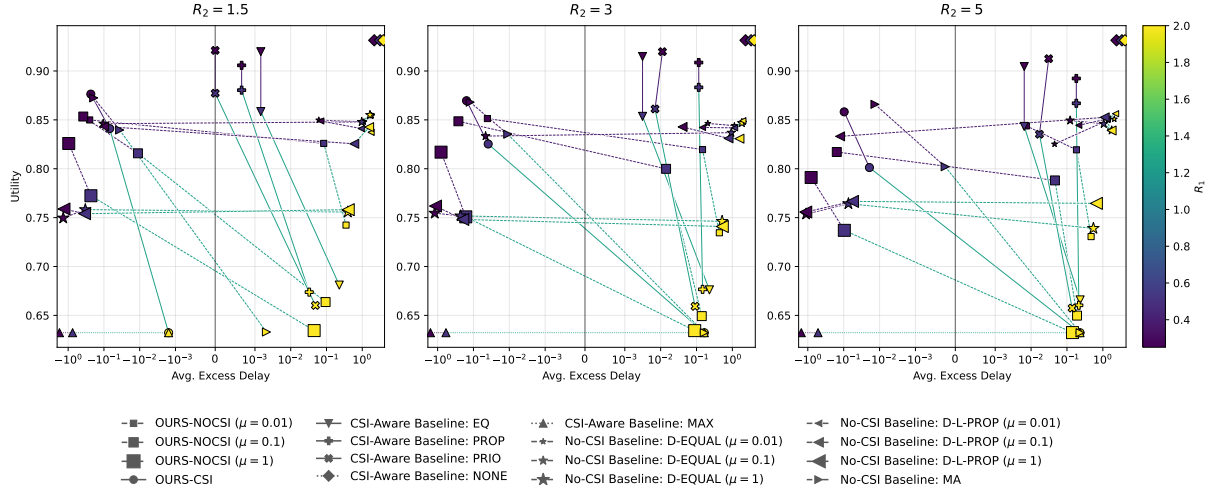


Figure 15: Top-K multi-task tradeoff under Stein's estimator. Each panel fixes the Task-2 target rate R_2 and sweeps the Task-1 target rate R_1 . The x-axis shows the overall average excess delay, and the y-axis shows the overall utility.

By examining the curves under different values of μ , we also observe that a larger μ leads to more aggressive compression, typically at the cost of some accuracy. This is consistent with our theoretical analysis, and can help guide the choice of μ to balance accuracy against the requirement of satisfying the target rate. If stricter control of target-rate violations is desired, a larger μ can be used; if a higher degree of violation is acceptable, a smaller μ can be chosen to achieve higher accuracy.

We further evaluate the multi-task setting with *task1* RN56-CF10 and *task2* FT5-SST2 under scenario LV-MT, sweeping both R_k and μ , and comparing *fitting* with *kernel-smoothing* (Stein's method). For *Top-K*, the resulting tradeoff curves are shown in Figures 14 and 15. In each subfigure, the target rate R_2 of *task2* is fixed, while the trajectory is obtained by sweeping the target rate R_1 of *task1*. The x-axis shows the aggregated excess delay across the two tasks weighted by w_k , and the y-axis shows the aggregated utility weighted by w_k .

The multi-task results are broadly similar with the single-task case. Our method, especially the No-CSI variant, achieves better overall utility under the delay constraint across a range of target-rate settings, while the fitting-based estimator remains more effective than Stein's method in this testbed setting. The curves under different values of μ show the expected tradeoff: a larger μ leads to more conservative behavior and lower violation, while a smaller μ yields higher utility at the cost of looser delay control.